

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



NHẬP MÔN HỌC MÁY
ĐỒ ÁN 1
HỌC CÓ GIÁM SÁT VÀ ỨNG DỤNG

STT	Họ và tên	MSSV	Vai trò
1	Nguyễn Kim Quốc	23120347	Nhóm trưởng
2	Ngô Thị Thực Quyên	23120348	Thành viên
3	Cao Quốc Tuấn	23120390	Thành viên
4	Lục Hoàng Tuấn	23120393	Thành viên
5	Huỳnh Trọng Viên	23120403	Thành viên

Môn học: Nhập môn Học máy (CSC14005)
Học kỳ: Học kỳ 2 – 2025/2026
GVHD: Th.S Lê Nhật Nam
Bộ môn: Khoa học Máy tính

TP. Hồ Chí Minh, Ngày 28 tháng 4 năm 2026

Mục lục

I	BÀI TOÁN HỒI QUY	1
1	Giới thiệu	1
1.1	Giới thiệu bài toán	1
1.2	Mục tiêu	1
1.3	Dữ liệu sử dụng	1
1.3.1	Ngữ cảnh dữ liệu	1
1.3.2	Nguồn gốc dữ liệu	1
1.3.3	Mô tả sơ lược	2
1.4	Định hướng triển khai	2
2	Cơ sở lý thuyết	2
2.1	Bài toán Hồi quy (Regression Problem)	2
2.2	Mô hình Tuyến tính và Hàm cơ sở	3
2.3	Ước lượng tham số: Bình phương tối thiểu (OLS)	3
2.4	Chính quy hóa (Regularization)	3
2.5	Phân tích Bias-Variance	4
2.6	Hồi quy Bayesian (Bayesian Regression)	4
2.7	Phương pháp Kernel và Gaussian Process (GP)	4
2.8	Các biến thể tối ưu hóa (Gradient Descent Variants)	4
3	Phân tích khám phá dữ liệu	4
3.1	Mô tả cấu trúc dữ liệu	5
3.2	Tương quan 'khối lượng hàng hóa' với 'thời gian giao'	5
3.3	Ma trận tương quan	6
3.4	Chu kỳ thời gian giao hàng	7
4	Tiền xử lý dữ liệu	8
4.1	Hợp nhất dữ liệu	8
4.2	Làm sạch dữ liệu	9
4.3	Xây dựng biến mục tiêu và Kiểm tra logic	9
4.4	Trích xuất đặc trưng	10
4.5	Phân tách tập dữ liệu	10
4.6	Xử lý đặc trưng và mã hóa	10
4.7	Danh mục các đặc trưng đầu vào của mô hình	11
5	Xây dựng và Huấn luyện Mô hình	12
5.1	Mô hình Cơ sở: Hồi quy Tuyến tính (Linear Regression)	12
5.1.1	Cài đặt Normal Equations (Từ đầu)	12
5.1.2	Cài đặt Mini-batch Gradient Descent (Từ đầu)	14
5.1.3	Kiểm định Giả thiết Mô hình (Gauss–Markov)	17
5.1.4	Khắc phục Phương sai Thay đổi: Hồi quy Bình phương Tối thiểu có Trọng số (WLS)	19
5.2	Hồi quy có Chính quy hóa và Lựa chọn Đặc trưng	21
5.2.1	Ridge Regression (Chuẩn hóa L_2)	21

5.2.2	Lasso Regression (Chuẩn hóa L_1)	23
5.2.3	Lựa chọn Siêu tham số và Phân tích Regularization Path	24
5.2.4	Elastic Net (Kết hợp L_1 và L_2)	27
5.2.5	Lựa chọn Đặc trưng	28
5.3	Mô hình với Hàm Cơ sở Phi tuyến và Ablation Study	30
5.3.1	Xây dựng các Hàm Cơ sở Phi tuyến	31
5.3.2	Áp dụng các hàm cơ sở và Thiết lập Baseline	31
5.3.3	Khảo sát Bậc Đa thức bằng Validation Curve	31
5.3.4	Phân tích Hiệu ứng Tương tác giữa các Đặc trưng	32
5.3.5	Ablation Study: Đánh giá Đóng góp của Từng Thành phần Hàm Cơ sở	33
6	Mô hình Nâng cao	33
6.1	Hồi quy Bayesian (Bayesian Linear Regression)	33
6.1.1	Cơ sở toán học	33
6.1.2	Quy trình thuật toán	34
6.1.3	Nhận xét và kết luận	35
6.2	Evidence Maximization cho Hồi quy Bayesian	35
6.2.1	Cơ sở toán học	35
6.2.2	Quy trình thuật toán	35
6.2.3	Kết quả thực nghiệm	36
6.2.4	Phân tích và Kết luận	36
6.3	Hồi quy Ridge với hàm nhân (Kernel Ridge Regression - KRR)	36
6.3.1	Cơ sở toán học	36
6.3.2	Quy trình thuật toán	36
6.3.3	Kết quả thực nghiệm	37
6.3.4	Nhận xét và kết luận	37
6.4	Hồi quy Quá trình Gauss (Gaussian Process Regression - GPR)	37
6.4.1	Cơ sở toán học	37
6.4.2	Quy trình thuật toán	38
6.4.3	Kết quả thực nghiệm	38
6.4.4	Phân tích và Kết luận	38
6.5	Hồi quy Kháng nhiễu (Robust Regression)	39
6.5.1	Cơ sở toán học	39
6.5.2	Quy trình thuật toán	39
6.5.3	Kết quả và kết luận	39
6.6	Phân tích sự đánh đổi Bias–Variance	40
6.6.1	Cơ sở toán học	40
6.6.2	Phương pháp ước lượng	40
6.6.3	Kết quả thực nghiệm	40
6.6.4	Nhận xét và kết luận	40
7	Đánh giá và phân tích	41
7.1	So sánh và Kiểm định Thống kê các Mô hình	41
7.1.1	Thu thập kết quả K-Fold Cross-Validation	41
7.1.2	Bảng tổng hợp kết quả (Mean $\pm$ Std)	41
7.1.3	Kiểm định thống kê: Paired T-test và Wilcoxon	42

8	Tổng kết và Hướng phát triển	43
8.1	Tổng kết chính	43
8.2	Ý nghĩa của kết quả	43
8.3	Hướng phát triển	43
II	BÀI TOÁN PHÂN LỚP	44
1	Giới thiệu và Tổng quan	44
1.1	Bộ dữ liệu	44
1.2	Mục tiêu	44
2	Cơ sở Lý thuyết	44
2.1	Bài toán Phân lớp	44
2.2	Hàm phân quyết tuyến tính	45
2.3	Hồi quy Logistic (Logistic Regression)	45
2.3.1	Hàm mất mát Cross-Entropy	45
2.3.2	Gradient Descent cho Logistic Regression	45
2.3.3	Thuật toán Newton–Raphson / IRLS	45
2.4	Phân tích phân tách tuyến tính Fisher (Fisher’s LDA)	46
2.5	Mô hình sinh Gaussian (LDA tổng quát)	46
2.6	Thuật toán Perceptron	46
2.7	Regularization cho Logistic Regression	47
2.8	Class-weighted Loss	47
3	Mô tả Dữ liệu và Phân tích Khám phá (EDA)	47
3.1	Mô tả Dữ liệu	47
3.1.1	Ý nghĩa các cột dữ liệu	48
3.1.2	Thống kê mô tả	49
3.2	Kiểm tra Chất lượng Dữ liệu	50
3.2.1	Giá trị trùng lặp và hàng rỗng	50
3.2.2	Giá trị thiếu (Missing Values)	50
3.3	Phân tích Khám phá (EDA)	51
3.3.1	Phân bố biến mục tiêu	51
3.3.2	Phân bố biến số – Histogram và Boxplot	51
3.3.3	Ma trận Tương quan	52
3.3.4	Scatter Plot – Age vs. Flight Distance	53
4	Tiền xử lý Dữ liệu	53
4.1	Mã hóa Biến Phân loại	53
4.2	Xử lý Ngoại lai và Log-Transform	53
4.3	Chia Dữ liệu	54
4.4	Chuẩn hóa Dữ liệu	54
4.5	Phân tích Mất cân bằng Lớp	55
5	Xây dựng và Huấn luyện Mô hình	55
5.1	Logistic Regression	55
5.1.1	Động cơ mô hình	55
5.1.2	Biểu diễn toán học cho bài toán nhị phân	56

5.1.3	Ma trận nhầm lẫn (Confusion Matrix) – Logistic Regression (Test Set)	56
5.1.4	Đường cong Precision-Recall – Logistic Regression (Test Set) . .	57
5.1.5	Hàm mất mát, class-weighted loss và regularization	59
5.1.6	Gradient, Hessian và tính lỗi của bài toán	59
5.1.7	Huấn luyện mô hình nhị phân bằng Gradient Descent	60
5.1.8	Huấn luyện bằng Newton–Raphson / IRLS	60
5.1.9	So sánh Gradient Descent và Newton–Raphson	60
5.1.10	Mở rộng Logistic Regression cho bài toán đa lớp	61
5.1.11	Nhận xét nhanh phần cài đặt	62
5.2	LDA và QDA	62
5.2.1	Động cơ mô hình	62
5.2.2	Fisher’s Linear Discriminant và trực giác tách lớp	62
5.2.3	Giả thiết Gaussian và ước lượng tham số	63
5.2.4	Hàm phân biệt và quy tắc dự đoán	63
5.2.5	Fisher score và mức độ phân biệt của từng đặc trưng	64
5.2.6	Minh họa decision boundary trong không gian 2D	65
5.2.7	Đánh giá trên tập test	65
5.2.8	Độ ổn định qua 5-fold Cross-Validation	65
5.2.9	Kiểm định ý nghĩa thống kê bằng McNemar’s test	66
5.2.10	Phân tích calibration và độ tin cậy của xác suất đầu ra	66
5.2.11	Vì sao LDA phù hợp hơn QDA trong bài toán này	67
5.3	Perceptron và Logistic Regression có Regularization	68
5.3.1	Thiết lập dữ liệu	68
5.3.2	Perceptron	68
5.3.3	Logistic Regression với L1/L2	71
5.3.4	Chọn λ bằng Stratified 5-Fold CV	72
5.3.5	Tối ưu ngưỡng trên validation	72
5.3.6	Ảnh hưởng của regularization lên sparsity	72
5.3.7	Mô hình cuối trên tập test	73
5.3.8	ROC curve và Precision–Recall curve trên tập test	74
5.3.9	Kiểm định ý nghĩa thống kê bằng McNemar’s test	74
5.3.10	Phân tích calibration	75
5.3.11	Bảng tổng kết và kết luận	76
6	So Sánh Toàn Diện Ba Nhóm Mô Hình	76
6.1	Kết quả trên Tập Test	77
6.2	Phân tích Độ nhạy cảm với Tỷ lệ Train/Test Split	77
6.3	Phân tích Tính bền vững với Nhiễu (Noise Robustness)	79
6.4	Đánh giá Toàn diện: Stratified 5-Fold Cross-Validation	80
6.5	Phân tích So sánh Toàn diện	82
7	Phân tích và Thảo luận	83
7.1	So sánh hiệu năng các mô hình	83
7.2	Kiểm định ý nghĩa thống kê bằng McNemar’s test	84
7.3	Logistic Regression vs. LDA: Khi nào dùng mô hình nào?	84
7.4	Ảnh hưởng của Regularization	85
7.5	Phân tích Lỗi	85

7.6	Hạn chế của Mô hình Tuyến tính	87
7.7	Phân tích Discriminability – Fisher Ratio	87
7.8	Phân tích Độ phức tạp Tính toán	89
8	BONUS	89
8.1	Mô hình Probit	90
8.1.1	Cơ sở lý thuyết	90
8.1.2	Kết quả thực nghiệm	90
8.2	Xấp xỉ Laplace	91
8.2.1	Cơ sở lý thuyết	91
8.2.2	Cài đặt	92
8.2.3	Kết quả thực nghiệm	92
8.3	Kernel Logistic Regression	93
8.3.1	Cơ sở lý thuyết	93
8.3.2	Kết quả thực nghiệm	94
8.4	Gaussian Naive Bayes vs. LDA	95
8.4.1	Cơ sở lý thuyết	95
8.4.2	Kết quả thực nghiệm	96
8.5	Phân tích VC Dimension và Structural Risk Minimization	97
8.5.1	Cơ sở lý thuyết	97
8.5.2	Áp dụng vào bộ dữ liệu Airline	97
9	Tổng kết	98
III	Phân tích So sánh và Kỹ năng Nghiên cứu	99
1	Kết nối Hồi quy và Phân lớp	99
1.1	Hồi quy Logistic như một bài toán Hồi quy	99
1.2	Generalized Linear Models (GLM): Khung thống nhất	99
1.3	Exponential Family: Nền tảng toán học chung	100
1.4	Tổng kết liên hệ	101
2	Bảng so sánh toàn diện các mô hình	102
3	Phân tích độ nhạy cảm và tính bền vững	103
3.1	Sensitivity Analysis: Thay đổi tỷ lệ Train/Test Split	103
3.2	Noise Injection: Thêm Nhiễu Gaussian vào Đặc trưng	104
3.3	Feature Corruption: Xóa Ngẫu nhiên Đặc trưng và So sánh Chiến lược Imputation	105
3.4	Phân tích Hội tụ: Loss Curve của các Thuật toán Lặp	106
4	Khả năng tái hiện lại thí nghiệm	107
4.1	Cố định Random Seed	107
4.2	Ghi Log Đầy đủ	108
4.3	Báo cáo Thời gian Thực thi và Bộ nhớ	108
4.4	Môi trường Phụ thuộc: requirements.txt	109
4.5	Kiểm tra Tính Tái hiện	109

Phần I

BÀI TOÁN HỒI QUY

1 Giới thiệu

1.1 Giới thiệu bài toán

Trong quản trị chuỗi cung ứng hiện đại, việc ước tính chính xác thời gian vận chuyển (delivery lead time) là yếu tố cốt lõi để tối ưu hóa trải nghiệm khách hàng và nâng cao hiệu quả vận hành. Bài toán nghiên cứu này tập trung vào việc dự báo chính xác số ngày thực tế từ thời điểm khách hàng xác nhận đơn hàng cho đến khi nhận được sản phẩm tại địa chỉ yêu cầu. Kết quả đầu ra của bài toán là một giá trị liên tục $t \in \mathbb{R}$. Việc xác định giá trị cụ thể này cho phép doanh nghiệp định lượng chính xác các khoảng thời gian chờ đợi, từ đó chủ động trong việc lập kế hoạch logistics và phân phối tài nguyên một cách tối ưu nhất.

1.2 Mục tiêu

Mục tiêu xuyên suốt của phần này là vận dụng các nền tảng toán học của mô hình tuyến tính để giải quyết một bài toán thực tế:

- **Xây dựng nền tảng lý thuyết:** Trình bày và hiểu rõ các mô hình tuyến tính cơ sở cùng các hàm cơ sở phi tuyến (Polynomial, Gaussian RBF, Sigmoid).
- **Thực thi thuật toán:** Cài đặt và so sánh hiệu quả giữa phương pháp nghiệm đóng (Normal Equations) và các biến thể Gradient Descent (Mini-batch GD).
- **Tối ưu hóa và Kiểm soát lỗi:** Áp dụng các kỹ thuật chính quy hóa (Ridge, Lasso, Elastic Net) để kiểm soát hiện tượng overfitting và thực hiện phân tích đánh đổi Bias-Variance.
- **Đánh giá khoa học:** Sử dụng các chỉ số đo lường chuẩn xác như MSE, RMSE, MAE và R^2 để đánh giá hiệu năng mô hình trên dữ liệu kiểm tra.

1.3 Dữ liệu sử dụng

1.3.1 Ngữ cảnh dữ liệu

Trong kỷ nguyên số, thương mại điện tử đã trở thành một phần không thể thiếu của nền kinh tế toàn cầu. Olist, một nền tảng bán hàng đa kênh (marketplace) lớn tại Brazil, đóng vai trò là cầu nối giúp các doanh nghiệp nhỏ tiếp cận khách hàng một cách hiệu quả hơn. Dữ liệu này ghi lại hành trình của các đơn hàng từ khâu đặt hàng, thanh toán, vận chuyển cho đến khi khách hàng phản hồi. Việc nghiên cứu dữ liệu này không chỉ có ý nghĩa về mặt kỹ thuật máy học mà còn giúp hiểu sâu hơn về quy trình vận hành logistics và hành vi người tiêu dùng tại một thị trường năng động như Brazil.

1.3.2 Nguồn gốc dữ liệu

- **Tên bộ dữ liệu:** Brazilian E-Commerce Public Dataset by Olist.

- **Nguồn cung cấp:** Nền tảng dữ liệu Kaggle.
- **Đường dẫn (Link):** <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- **Lý do chọn bộ dữ liệu:** nhằm đáp ứng yêu cầu sử dụng dữ liệu thực tế, tuyệt đối không dùng dữ liệu tổng hợp trong đề án. Với sự kết hợp đa dạng giữa các biến liên tục và định danh, bộ dữ liệu này tạo điều kiện tối ưu để triển khai, thử nghiệm và đối chiếu hiệu năng giữa các mô hình hồi quy tuyến tính cũng như các phương pháp tiếp cận phi tuyến.

1.3.3 Mô tả sơ lược

- **Kích thước:** Tập dữ liệu gốc bao gồm khoảng 100,000 dòng.
- **Số lượng thuộc tính:** 33 thuộc tính trích xuất từ 9 bảng dữ liệu quan hệ (bao gồm 18 biến phân loại/chuỗi/thời gian và 15 biến dạng số sau khi thực hiện kết nối các bảng).
- **Định dạng:** .CSV

1.4 Định hướng triển khai

Nhóm thực hiện dự án theo quy trình 5 bước tối ưu hóa dữ liệu và mô hình:

- **Hợp nhất dữ liệu:** Kết nối các nguồn dữ liệu rời rạc về đơn hàng, sản phẩm và vị trí địa lý để xây dựng một tập dữ liệu tập trung, bao quát toàn bộ quy trình vận hành.
- **Làm sạch và Tiền xử lý:** Loại bỏ các bản ghi không hợp lệ, xử lý các giá trị còn thiếu và hiệu chỉnh các sai sót logic để đảm bảo dữ liệu đầu vào có độ tin cậy cao nhất.
- **Phân tích khám phá (EDA):** Trực quan hóa dữ liệu để tìm ra các quy luật biến động theo thời gian, nhận diện các yếu tố ảnh hưởng then chốt và phát hiện các điểm dữ liệu bất thường.
- **Chuẩn bị và Phân đoạn:** Đồng bộ hóa thang đo giữa các nhóm dữ liệu khác nhau và thực hiện chia tập dữ liệu thành các phần riêng biệt để phục vụ việc huấn luyện và kiểm định mô hình độc lập.
- **Xây dựng và Đánh giá mô hình:** Thử nghiệm các thuật toán hồi quy khác nhau để tìm ra giải pháp tối ưu, sau đó đánh giá hiệu quả dự báo dựa trên các chỉ số đo lường sai số tiêu chuẩn.

2 Cơ sở lý thuyết

2.1 Bài toán Hồi quy (Regression Problem)

Mục tiêu của bài toán hồi quy là xây dựng một hàm dự đoán $y(x)$ từ tập dữ liệu huấn luyện $\mathcal{D} = \{(x_n, t_n)\}_{n=1}^N$. Trong đó, $x_n \in \mathbb{R}^D$ là vector đặc trưng đầu vào và $t_n \in \mathbb{R}$ là

giá trị mục tiêu (thời gian giao hàng thực tế). Mỗi quan hệ này được mô hình hóa dưới dạng:

$$t = y(x, w) + \epsilon$$

Trong đó, $\epsilon \sim \mathcal{N}(0, \beta^{-1})$ là nhiễu Gaussian với độ chính xác (precision) β .

2.2 Mô hình Tuyến tính và Hàm cơ sở

Mô hình hồi quy tuyến tính được định nghĩa là sự kết hợp tuyến tính của các hàm cơ sở phi tuyến $\phi_j(x)$:

$$y(x, w) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(x) = w^\top \phi(x)$$

Nhóm thực hiện khảo sát và so sánh ba loại hàm cơ sở chính:

- **Hàm Tuyến tính/Đa thức:** $\phi_j(x) = x^j$, dùng để xấp xỉ các mối quan hệ phi tuyến cục bộ
- **Hàm Gaussian RBF:** $\phi_j(x) = \exp\left(-\frac{(x-\mu_j)^2}{2s^2}\right)$, giúp mô hình hóa các ảnh hưởng mang tính địa phương.
- **Hàm Sigmoid:** $\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right)$, tạo ra các biến thể logistic cho mô hình.

2.3 Ước lượng tham số: Bình phương tối thiểu (OLS)

Để tìm bộ trọng số w tối ưu, ta cực tiểu hóa hàm mất mát dạng tổng bình phương sai số (sum-of-squares error):

$$E_D(w) = \frac{1}{2} \sum_{n=1}^N \{t_n - w^\top \phi(x_n)\}^2 = \frac{1}{2} \|t - \Phi w\|^2$$

Nghiệm tối ưu toàn cục (Normal Equations) được xác định bằng công thức:

$$w^* = (\Phi^\top \Phi)^{-1} \Phi^\top t$$

Trong đó, Φ là ma trận thiết kế (design matrix) với các thành phần $\Phi_{nj} = \phi_j(x_n)$.

2.4 Chính quy hóa (Regularization)

Để kiểm soát hiện tượng quá khớp (overfitting), các số hạng phạt được thêm vào hàm mất mát:

- **Ridge Regression (L_2):** Thêm $\frac{\lambda}{2} \|w\|^2$ để giới hạn độ lớn của trọng số. Nghiệm tối ưu là $w_{ridge}^* = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top t$.
- **Lasso Regression (L_1):** Thêm $\lambda \|w\|_1$ nhằm tạo ra sự thưa thớt (sparsity) trong bộ trọng số, hỗ trợ việc lựa chọn đặc trưng.
- **Elastic Net:** Kết hợp cả L_1 và L_2 để tận dụng ưu điểm của cả hai phương pháp.

2.5 Phân tích Bias-Variance

Sai số của mô hình được phân tách thành ba thành phần chính:

- **Bias (Độ lệch):** Sai số do giả thiết mô hình quá đơn giản (gây ra underfitting).
- **Variance (Phương sai):** Sự nhạy cảm của mô hình với những biến động nhỏ trong tập huấn luyện (gây ra overfitting).
- **Irreducible Error:** Nhiễu nội tại của dữ liệu. Việc điều chỉnh siêu tham số λ trong chính quy hóa cho phép nhóm kiểm soát sự đánh đổi (trade-off) giữa Bias và Variance để đạt được khả năng tổng quát hóa tốt nhất.

2.6 Hồi quy Bayesian (Bayesian Regression)

Khác với tiếp cận truyền thống chỉ đưa ra một điểm ước lượng duy nhất, hồi quy Bayesian tính toán phân phối hậu nghiệm (posterior) của tham số dựa trên giả thiết tiên nghiệm (prior) $p(w) = \mathcal{N}(w|0, \alpha^{-1}I)$.

- **Phân phối hậu nghiệm:** $p(w|t) = \mathcal{N}(w|m_N, S_N)$ với $S_N^{-1} = \alpha I + \beta \Phi^T \Phi$ và $m_N = \beta S_N \Phi^T t$.
- **Dự đoán bất định:** Mô hình cung cấp phân phối dự đoán $p(t^*|x^*, t) = \mathcal{N}(t^*|m_N^T \phi(x^*), \sigma_N^2(x^*))$, trong đó σ_N^2 đại diện cho mức độ bất định của dự báo tại điểm mới.

2.7 Phương pháp Kernel và Gaussian Process (GP)

Khi không gian đặc trưng trở nên phức tạp, nhóm áp dụng Kernel Trick để mở rộng mô hình sang không gian vô hạn chiều mà không cần tính toán tường minh $\phi(x)$.

- **Kernel Ridge Regression:** Sử dụng hàm Kernel (RBF, Polynomial) để học các đường cong phi tuyến phức tạp.
- **Gaussian Process Regression:** Xem xét một phân phối trên các hàm số $f(x) \sim \mathcal{GP}(m(x), k(x, x'))$. Tiếp cận này mang lại khả năng ước lượng bất định tự nhiên và tối ưu hóa tham số thông qua việc cực đại log-marginal-likelihood.

2.8 Các biến thể tối ưu hóa (Gradient Descent Variants)

Để giải quyết bài toán với tập dữ liệu lớn như Olist (>100,000 mẫu), nhóm khảo sát các thuật toán tối ưu hóa lặp:

- **SGD & Mini-batch GD:** Giảm chi phí tính toán trên mỗi bước cập nhật trọng số.
- **Adam & Momentum:** Sử dụng thông tin về đạo hàm bậc nhất và các cơ chế thích nghi để vượt qua các điểm cực tiểu cục bộ và tăng tốc độ hội tụ.

3 Phân tích khám phá dữ liệu

Nội dung dự kiến gồm mô tả dữ liệu, phân phối đặc trưng, tương quan và các quan sát ban đầu.

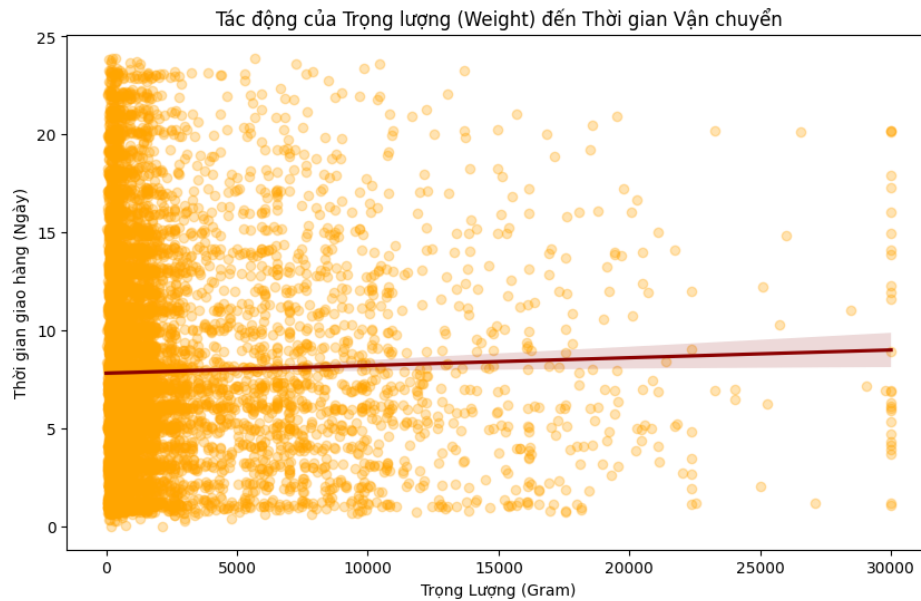
3.1 Mô tả cấu trúc dữ liệu

Bộ dữ liệu Olist được cấu trúc dưới dạng các bảng quan hệ. Để xây dựng mô hình hồi quy, nhóm tập trung vào nội dung của các tệp thành phần sau:

- **olist_orders_dataset.csv:** Chứa thông tin cốt lõi về đơn hàng, bao gồm các mốc thời gian quan trọng như ngày đặt hàng và ngày giao hàng thực tế để tính toán biến mục tiêu t
- **olist_order_items_dataset.csv:** Bao gồm chi tiết về các sản phẩm trong từng đơn hàng, giá bán và thông tin nhận diện người bán.
- **olist_products_dataset.csv:** Cung cấp các thông số vật lý của hàng hóa như khối lượng, chiều dài, chiều rộng và chiều cao—đây là các đặc trưng liên tục quan trọng cho mô hình.
- **olist_customers_dataset.csv:** Chứa thông tin về vị trí địa lý của khách hàng thông qua mã bưu chính (zip code).
- **olist_sellers_dataset.csv:** Cung cấp thông tin tương ứng về vị trí của người bán, hỗ trợ việc xác định các vùng địa lý vận chuyển.
- **olist_geolocation_dataset.csv:** Lưu trữ tọa độ kinh độ và vĩ độ của các mã bưu chính, được sử dụng để tính toán khoảng cách vận chuyển thực tế.
- **olist_order_payments_dataset.csv:** Chứa dữ liệu về phương thức thanh toán và tổng giá trị đơn hàng.
- **olist_order_reviews_dataset.csv:** Lưu trữ các đánh giá và phản hồi của khách hàng sau khi nhận sản phẩm.
- **product_category_name_translation.csv:** Bảng tra cứu dùng để chuyển đổi tên danh mục sản phẩm từ tiếng Bồ Đào Nha sang tiếng Anh.

3.2 Tương quan 'khối lượng hàng hóa' với 'thời gian giao'

Biểu đồ thể hiện mối quan hệ giữa biến độc lập là Trọng lượng (grams) và biến mục tiêu là Thời gian giao hàng (ngày). Đường màu đỏ là đường hồi quy tuyến tính (regression line) đi kèm với vùng bóng mờ thể hiện khoảng tin cậy (confidence interval).



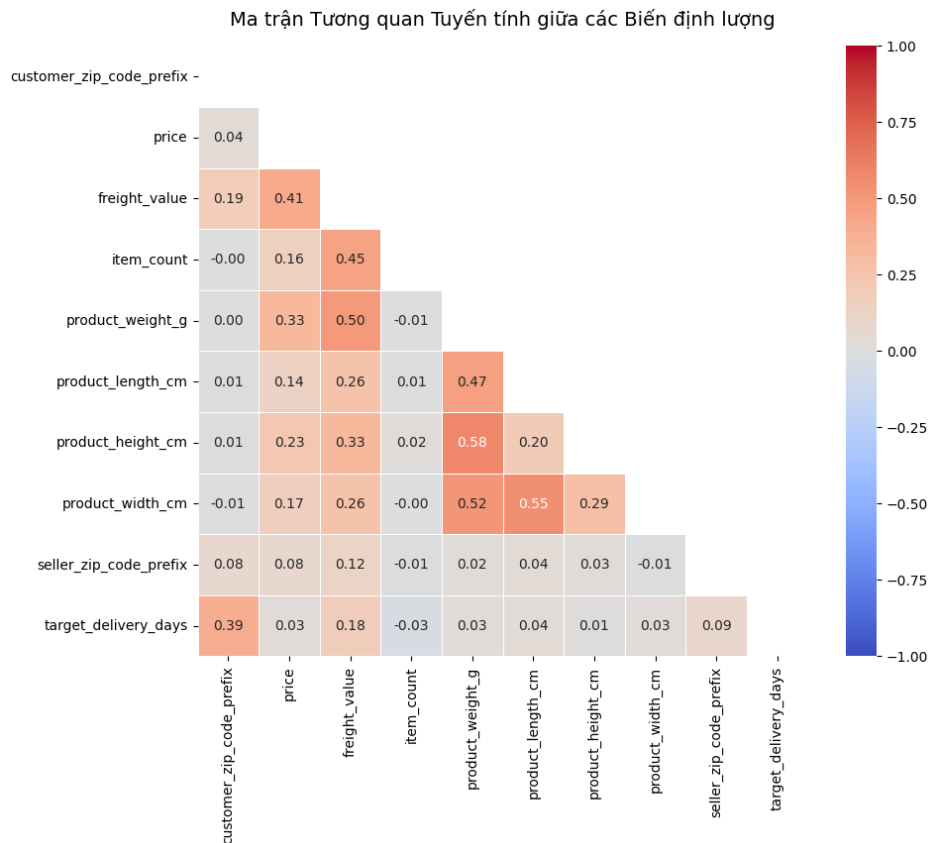
Hình 1: Biểu đồ phân tán giữa Trọng lượng sản phẩm và Thời gian vận chuyển

Nhận xét

- Đường hồi quy gần như nằm ngang cho thấy trọng lượng sản phẩm có tác động không đáng kể đến thời gian vận chuyển thực tế.
- Hàng hóa tập trung chủ yếu ở phân khúc nhẹ (dưới 5,000g). Ở cùng một mức trọng lượng, thời gian giao hàng biến động rất lớn (từ 0 đến hơn 20 ngày).
- Xuất hiện nhiều mẫu dữ liệu nằm xa đường xu hướng, cần áp dụng phương pháp IQR hoặc Z-score để loại bỏ nhiễu trước khi huấn luyện mô hình OLS.
- **Hướng tiếp cận:** Do mối quan hệ tuyến tính giữa hai biến này rất mờ nhạt, nhóm đề xuất sử dụng hàm cơ sở phi tuyến (Polynomial, RBF) để cải thiện độ chính xác.

3.3 Ma trận tương quan

Biểu đồ Heatmap dùng màu sắc để định lượng hệ số tương quan Pearson (r), trong đó màu đỏ thể hiện tương quan thuận và màu xanh là tương quan nghịch.

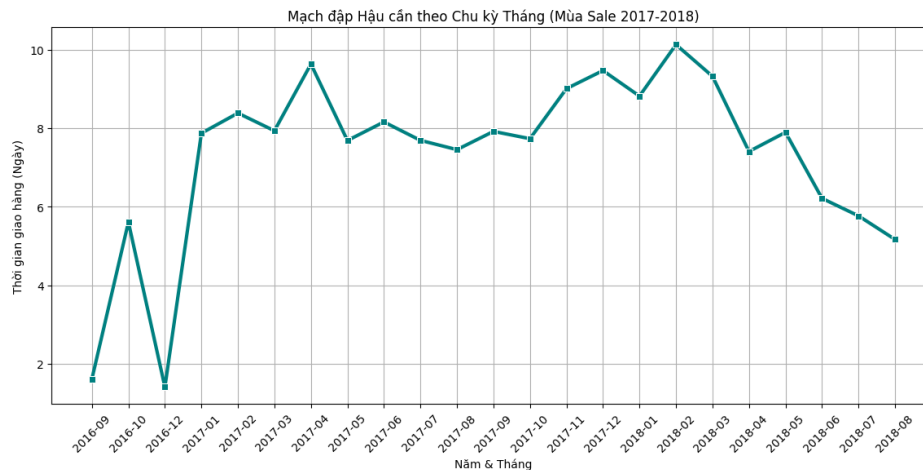


Hình 2: Ma trận tương quan giữa các biến định lượng

- Mã bưu cục khách hàng có tương quan cao nhất với thời gian giao hàng (0.39), cho thấy địa lý là biến quan trọng nhất.
- Trọng lượng và kích thước sản phẩm có tương quan mạnh với nhau (0.47 – 0.58), cần thực hiện loại bỏ biến trùng lặp để mô hình không bị nhiễu.
- Nhóm sẽ ưu tiên chọn các biến về vị trí và chi phí để đảm bảo mô hình hồi quy đạt độ chính xác và tính ổn định cao nhất.

3.4 Chu kỳ thời gian giao hàng

Biểu đồ đường (Time Series) thể hiện sự thay đổi của thời gian giao hàng trung bình theo từng tháng, kéo dài từ tháng 09/2016 đến tháng 08/2018. Trục hoành biểu thị mốc thời gian (Năm & Tháng) và trục tung biểu thị số ngày giao hàng trung bình.



Hình 3: Biến động thời gian giao hàng theo chu kỳ tháng

- **Tính mùa vụ rõ rệt:** Thời gian giao hàng tăng mạnh và đạt đỉnh vào các giai đoạn mua sắm cao điểm như tháng 12/2017 (Sale cuối năm) và tháng 02/2018 (đỉnh điểm trên 10 ngày), phản ánh sự quá tải của hệ thống hậu cần.
- **Xu hướng cải thiện:** Sau đỉnh điểm đầu năm 2018, thời gian giao hàng có xu hướng giảm dần và ổn định đáng kể về cuối giai đoạn (tháng 08/2018), cho thấy hiệu quả vận hành được cải thiện.
- **Kết luận mô hình:** Yếu tố thời gian mang tính mùa vụ cao; do đó, nhóm sẽ đưa thêm các đặc trưng về Tháng hoặc Mùa trong năm vào mô hình hồi quy để bắt kịp các biến động định kỳ này.

4 Tiền xử lý dữ liệu

Mục tiêu của giai đoạn này là chuẩn hóa dữ liệu, loại bỏ nhiễu và trích xuất các đặc trưng có giá trị nhằm tối ưu hóa hiệu năng cho các mô hình hồi quy.

4.1 Hợp nhất dữ liệu

Vì bộ dữ liệu Olist được lưu trữ dưới dạng các tệp tin quan hệ, nhóm thực hiện quy trình hợp nhất để tạo ra bảng dữ liệu tập trung (`df_master`) trước khi tiến hành các bước làm sạch.

Bảng dữ liệu gốc: Sử dụng tệp `olist_orders_dataset.csv` làm bảng điều hướng chính để quản lý mã đơn hàng và các mốc thời gian quan trọng.

Phương pháp thực hiện: Nhóm sử dụng phép toán *Left Join* để liên kết bảng gốc với các tệp dữ liệu phụ trợ nhằm mở rộng các chiều phân tích:

- **Kết nối với bảng Items:** Để trích xuất thông tin chi tiết về giá bán và chi phí vận chuyển của từng đơn hàng.
- **Kết nối với bảng Products:** Để bổ sung các đặc trưng vật lý của hàng hóa như trọng lượng và kích thước.
- **Kết nối với bảng Customers và Sellers:** Để thu thập dữ liệu về mã vùng địa lý (ZIP code) của cả người mua và người bán.

Kết quả: Sau quá trình hợp nhất, nhóm thu được một tập dữ liệu đa chiều (multi-dimensional dataset). Đây là tiền đề quan trọng để thực hiện các tính toán tương quan giữa các yếu tố như khoảng cách địa lý, khối lượng hàng hóa đối với thời gian giao hàng thực tế.

4.2 Làm sạch dữ liệu

Sau khi hợp nhất, dữ liệu thường chứa nhiều giá trị khuyết thiếu (NaN) và các bản ghi không phù hợp với mục tiêu nghiên cứu. Nhóm thực hiện làm sạch thông qua 3 công đoạn chính:

a. Xử lý giá trị khuyết thiếu

Nhóm tiến hành xử lý các giá trị rỗng nhằm đảm bảo tính toàn vẹn cho mô hình:

- **Điền thông tin thời gian:** Điền 160 dòng phê duyệt đơn (sai lệch trung vị 0.34 giờ) và 1,783 dòng bàn giao vận chuyển (sai lệch trung vị 43.66 giờ) để hoàn thiện luồng logic vận hành.
- **Gán nhãn mặc định:** Gán nhãn 'other' cho các ngành hàng bị rỗng, giúp bảo toàn kích thước mẫu và tránh mất mát thông tin quan trọng khi huấn luyện.

b. Lọc trạng thái và chuẩn hóa định dạng

Để đảm bảo tính khách quan cho mô hình, nhóm thực hiện thanh lọc dữ liệu với các tiêu chuẩn nghiêm ngặt:

- **Lọc trạng thái đơn hàng:** Chỉ giữ lại các đơn hàng có trạng thái 'delivered'. Các đơn hàng bị hủy hoặc đang xử lý bị loại bỏ do không cung cấp thông tin về thời gian thực tế.
 - Tổng số đơn hàng ban đầu: 99,441.
 - Số đơn bị loại bỏ (chưa giao hoặc lỗi ngày): 2,971.
 - Số đơn hàng còn lại: 96,470.
- **Đồng bộ hóa:** thực hiện chuẩn hóa tên cột và ép kiểu dữ liệu sang datetime. Việc này giúp tối ưu hóa hiệu suất cho các phép toán vector và sẵn sàng cho bước trích xuất đặc trưng (feature engineering).

4.3 Xây dựng biến mục tiêu và Kiểm tra logic

Nhóm tiến hành xác định biến mục tiêu là thời gian vận chuyển thực tế từ kho đến khách hàng:

- **Tính toán biến mục tiêu (t):** Thời gian giao hàng thực tế được xác định bằng công thức toán học sau:

$$t = \text{order_delivered_customer_date} - \text{order_delivered_carrier_date}$$

Trong đó, t đại diện cho thời gian vận chuyển thực tế từ kho đến tay khách hàng.

- **Kiểm tra logic:** Nhóm loại bỏ thêm 88 dòng dữ liệu có sai sót về trình tự thời gian (nhận hàng trước khi giao kho).

- **Kết quả cuối cùng:**

- Số lượng mẫu sạch đưa vào mô hình: 96,382.
- Thời gian giao hàng trung bình: 9.26 ngày.

4.4 Trích xuất đặc trưng

Nhóm xây dựng 6 nhóm đặc trưng phái sinh nhằm tối ưu hóa khả năng dự báo cho mô hình:

- **Địa lý (Distance):** Tính khoảng cách Haversine từ tọa độ Zip Code để định lượng chính xác quãng đường vận chuyển thực tế thay vì sử dụng mã bưu cục thô.
- **Vùng miền (Spatial Flags):** Phân loại đơn hàng nội tỉnh và liên bang, giúp mô hình nhận diện sự khác biệt trong ưu tiên xử lý logistics.
- **Vật lý (Volume):** Chập ba chiều Dài-Rộng-Cao thành một biến Thể tích duy nhất để phản ánh độ cồng kềnh và triết tiêu hiện tượng đa cộng tuyến.
- **Chu kỳ (Cyclical Time):** Áp dụng mã hóa Sine/Cosine cho các biến thời gian (Thứ, Tháng, Giờ) để xử lý "Bẫy cuối tuần" và các vòng lặp vận hành tự nhiên.

$$x_{sin} = \sin\left(\frac{2\pi x}{max(x)}\right); \quad x_{cos} = \cos\left(\frac{2\pi x}{max(x)}\right)$$

- **Ngoại cảnh (Event Flags):** Đánh dấu các sự kiện đặc biệt (Black Friday, cuộc bãi công 2018) giúp mô hình giải thích được các điểm dữ liệu biến động bất thường (anomalies).

4.5 Phân tách tập dữ liệu

Để đánh giá khách quan hiệu năng của mô hình dự báo, nhóm tiến hành chia tập dữ liệu đã qua xử lý thành hai phần độc lập:

- **Tập huấn luyện (Train Set):** 63,953 đơn hàng – dùng để mô hình học các hệ số và quy luật vận hành.
- **Tập kiểm định (Val Set):** 9,136 đơn hàng – dùng để tinh chỉnh siêu tham số và tránh hiện tượng quá khớp (Overfitting).
- **Tập kiểm tra (Test Set):** 18,273 đơn hàng – dữ liệu hoàn toàn mới dùng để báo cáo hiệu năng dự báo cuối cùng.

4.6 Xử lý đặc trưng và mã hóa

Nhóm thực hiện xử lý dữ liệu với tư duy bảo toàn tính thực tế của sản phẩm và tối ưu hóa thông tin từ các biến định danh:

- **Điền khuyết thực tế:** Nhóm điền giá trị thiếu cho Khối lượng và Thể tích bằng trung vị (Median) theo ngành hàng từ tập Train. Việc xử lý trực tiếp trên biến Thể tích tổng hợp giúp tránh tạo ra các kích thước "biến dị" không có thực khi điền khuyết từng chiều lẻ.
- **Mã hóa mục tiêu:** Chuyển đổi các biến định danh (Bang, Ngành hàng) thành chỉ số rủi ro thời gian. Kỹ thuật Smoothing được áp dụng để xử lý các danh mục hiếm và ngăn chặn hiện tượng quá khớp (Overfitting).
- **Chống rò rỉ dữ liệu:** Toàn bộ tham số thống kê chỉ được trích xuất từ tập Train, đảm bảo tính khách quan tuyệt đối cho mô hình khi dự báo trên dữ liệu mới.

4.7 Danh mục các đặc trưng đầu vào của mô hình

Sau quá trình xử lý và chọn lọc, nhóm đã xác định bộ đặc trưng cuối cùng đưa vào huấn luyện mô hình dự báo. Các đặc trưng này được phân loại dựa trên bản chất dữ liệu và phương pháp tối ưu hóa tương ứng:

Nhóm đặc trưng	Tên đặc trưng (Feature)	Mô tả ý nghĩa	Phép biến đổi & Tối ưu
Định lượng (Skewed)	freight_value, price, product_weight_g, product_volume_cm3, distance_km	Các thông số về chi phí, giá trị, khối lượng, thể tích và khoảng cách vận chuyển.	Quantile Transformer: Xử lý phân phối lệch phải và loại bỏ ảnh hưởng của các giá trị ngoại lai cực đoan.
Chu kỳ (Cyclical)	order_hour_sin/cos, day_sin/cos, month_sin/cos	Phản ánh nhịp độ vận hành theo thời gian (giờ trong ngày, ngày trong tuần, tháng).	StandardScaler: Đưa các hàm sóng lượng giác về cùng một thang đo phương sai trung tâm.
Phân loại (Categorical)	customer_state, product_category_name_english, seller_state	Thông tin về địa danh và ngành hàng sản phẩm.	Target Encoding + Scaling: Chuyển đổi định danh thành chỉ số rủi ro thời gian dựa trên tập Train.
Nhị phân (Binary)	is_intra_state, is_black_friday, is_strike, v.v.	Các cờ hiệu logic đánh dấu đơn nội tỉnh hoặc các sự kiện biến động đặc biệt.	Passthrough: Giữ nguyên tín hiệu 0/1 để bảo toàn ý nghĩa logic tắt/mở của sự kiện.

Bảng 1: Bảng tổng hợp đặc trưng và phương pháp tiền xử lý dữ liệu.

Biến mục tiêu (Target): `target_delivery_days` — Số ngày giao hàng thực tế (Đã được chuẩn hóa bằng `StandardScaler` và giới hạn biên ≤ 60 ngày).

5 Xây dựng và Huấn luyện Mô hình

5.1 Mô hình Cơ sở: Hồi quy Tuyến tính (Linear Regression)

Mục tiêu của phần này là thiết lập một mô hình cơ sở (baseline) sử dụng phương pháp bình phương tối thiểu thông thường (OLS). Chúng tôi triển khai song song hai cách tiếp cận: nghiệm đóng (Normal Equations) để có kết quả chính xác tức thời, và Mini-batch Gradient Descent để đánh giá tính khả thi của thuật toán tối ưu lặp trên dữ liệu lớn.

5.1.1 Cài đặt Normal Equations (Từ đầu)

a) Mô tả triển khai Nhóm tự xây dựng lớp `NormalEquationRegression` để giải bài toán hồi quy tuyến tính bằng nghiệm đóng. Công thức toán học được cài đặt như sau:

$$\mathbf{w}^* = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t}$$

Trong đó:

- `X_train` là ma trận đặc trưng đầu vào đã được tiền xử lý hoàn chỉnh, bao gồm các nhóm biến định lượng, chu kỳ, phân loại và nhị phân.
- `y_train` là biến mục tiêu `target_delivery_days` đã được chuẩn hóa bằng `StandardScaler`.
- Ma trận thiết kế Φ được tạo bằng cách thêm một cột bias gồm toàn giá trị 1 vào phía trước ma trận đặc trưng đầu vào.

Chi tiết kỹ thuật cài đặt có thể tóm tắt qua đoạn mã sau:

```
1 class NormalEquationRegression:
2     def fit(self, X, y):
3         Phi = np.hstack([np.ones((X.shape[0], 1)), X])
4         self.w = np.linalg.pinv(Phi.T @ Phi) @ Phi.T @ y
```

Về mặt ổn định số học, như đã quan sát trong quá trình EDA, một số cặp đặc trưng có mức tương quan tương đối cao, chẳng hạn `product_weight` và `product_volume` với hệ số tương quan xấp xỉ $r \approx 0.58$. Điều này có thể làm cho ma trận $\Phi^\top \Phi$ trở nên suy biến nhẹ hoặc kém điều kiện. Vì vậy, nhóm sử dụng `np.linalg.pinv` (giả nghịch đảo Moore–Penrose) thay cho `np.linalg.inv` nhằm bảo đảm thuật toán luôn cho nghiệm ổn định, ngay cả khi tồn tại các trị riêng rất nhỏ.

b) Kết quả Baseline trên các tập dữ liệu Mô hình được huấn luyện trên 63,953 mẫu thuộc tập Train. Sau đó, nhóm sử dụng hàm `evaluate_real_performance` để chuyển dự đoán từ thang đo chuẩn hóa trở về đơn vị ngày thực tế, giúp các chỉ số đánh giá có ý nghĩa trực quan hơn đối với bài toán giao hàng.

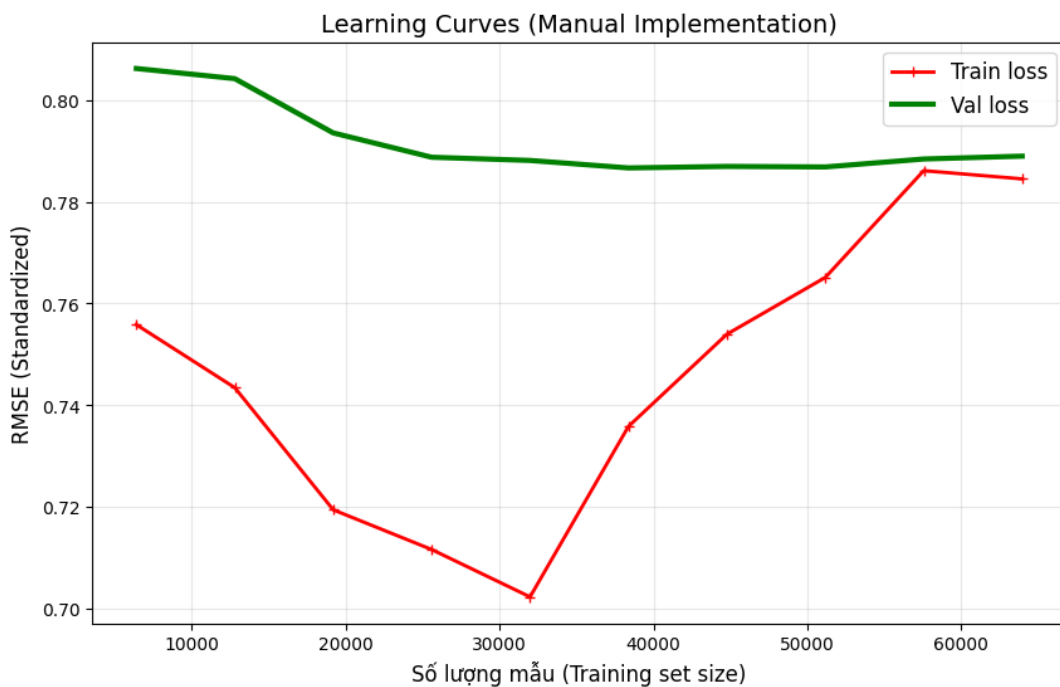
Kết quả chi tiết được trình bày trong Bảng 5.1.

Bảng 2: Hiệu năng mô hình cơ sở OLS (Normal Equations) trên các tập dữ liệu

Chỉ số	Train Set	Validation Set	Test Set
MSE	17.9177	18.1225	11.0671
RMSE (Ngày)	4.2329	4.2571	3.3267
MAE (Ngày)	3.2755	3.2451	2.6588
R^2 Score	0.3845	0.3835	0.2273

Nhận xét sơ bộ cho thấy giá trị R^2 trên tập Test đạt 0.2273, tức mô hình tuyến tính cơ sở giải thích được khoảng 22.73% phương sai của thời gian giao hàng. Đây là một kết quả chấp nhận được đối với một mô hình đơn giản, chưa sử dụng chính quy hóa hay biến đổi phi tuyến. Bên cạnh đó, MAE trên tập Test đạt 2.6588 ngày, nghĩa là sai số dự đoán trung bình vào khoảng ± 2.66 ngày so với thời gian vận chuyển thực tế. Chỉ số này được sử dụng làm mức chuẩn ban đầu để so sánh với các mô hình nâng cao ở các mục tiếp theo.

c) Phân tích Đường cong Học tập (Learning Curves) Để kiểm tra xem mô hình OLS có gặp vấn đề về phương sai cao hay độ lệch cao hay không, nhóm tiến hành xây dựng Learning Curves bằng phương pháp tự cài đặt, không sử dụng `sklearn`. Biểu đồ thể hiện sự thay đổi của RMSE trên tập Train và Validation khi kích thước tập huấn luyện được tăng dần.



Hình 4: Learning Curves của mô hình OLS (Normal Equations)

Dựa trên Hình 5.1, có thể rút ra một số nhận xét định lượng như sau:

- **Xu hướng hội tụ:** Khi kích thước tập huấn luyện còn nhỏ (dưới 10,000 mẫu), Train RMSE rất thấp trong khi Validation RMSE cao đáng kể. Khoảng cách lớn

giữa hai đường cong là dấu hiệu điển hình của hiện tượng overfitting, do mô hình được học trên lượng dữ liệu chưa đủ để tổng quát hóa.

- **Trạng thái ổn định:** Khi số lượng mẫu vượt quá 40,000, hai đường RMSE bắt đầu tiến gần nhau và dần trở nên phẳng hơn. Ở giai đoạn cuối, chênh lệch giữa Train và Validation thu hẹp rõ rệt, cho thấy mô hình đạt đến trạng thái ổn định về mặt tổng quát hóa.
- **Đánh giá độ lệch:** Mặc dù hai đường cong hội tụ, cả Train RMSE và Validation RMSE đều dừng lại ở mức dương tương đối lớn thay vì giảm sâu hơn. Điều này cho thấy mô hình tuyến tính vẫn còn underfitting nhẹ, hàm tuyến tính hiện tại chưa đủ linh hoạt để biểu diễn đầy đủ mối quan hệ giữa đặc trưng đầu vào và thời gian giao hàng.

Từ các quan sát trên, có thể kết luận rằng mô hình OLS là một baseline nhanh, ổn định và dễ diễn giải, nhưng năng lực biểu diễn của nó còn giới hạn. Điều này tạo cơ sở hợp lý để chuyển sang các phương pháp có chính quy hóa hoặc các hàm cơ sở phi tuyến ở những mục tiếp theo nhằm vượt qua ngưỡng bão hòa hiện tại của mô hình tuyến tính thuần túy.

5.1.2 Cài đặt Mini-batch Gradient Descent (Từ đầu)

Nhóm tiến hành xây dựng thuật toán Mini-batch Gradient Descent để kiểm tra tính khả thi của các phương pháp tối ưu lặp trên tập dữ liệu khoảng 96,000 mẫu. Mục tiêu chính là so sánh tốc độ hội tụ và chất lượng nghiệm với nghiệm đóng Normal Equations.

a) Chi tiết Triển khai Nhóm tự xây dựng class `MiniBatchGDRegression` với các đặc điểm kỹ thuật sau:

Bảng 3: Chi tiết triển khai thuật toán Mini-batch Gradient Descent

Thành phần	Mô tả chi tiết
Khởi tạo trọng số	<code>np.random.randn(n_features) * 0.01</code> – khởi tạo ngẫu nhiên nhỏ để tránh bão hòa gradient ban đầu.
Batch Size	256 – được chọn dựa trên dung lượng RAM và khả năng vector hóa của CPU.
Xáo trộn dữ liệu	Mỗi epoch, nhóm thực hiện <code>np.random.permutation</code> toàn bộ tập huấn luyện để bảo đảm tính ngẫu nhiên của mini-batch.
Cập nhật gradient	Sử dụng công thức giải tích: $\nabla = \frac{1}{ B } \mathbf{X}_B^T (\mathbf{X}_B \mathbf{w} - \mathbf{y}_B).$
Ghi nhận lịch sử	MSE trên tập Train và Validation được tính và lưu lại sau mỗi epoch để phân tích hội tụ.

b) Chiến lược Điều chỉnh Tốc độ Học (Learning Rate Schedule) Để tăng tốc hội tụ và giảm dao động, nhóm triển khai hai chiến lược điều chỉnh tốc độ học và so sánh trực quan:

Step Decay:

$$lr = lr_0 \cdot 0.5^{\lfloor \text{epoch}/50 \rfloor}$$

Đặc điểm: Giảm 50% sau mỗi 50 epoch. Phương pháp đơn giản, dễ kiểm soát.

Cosine Annealing:

$$lr = lr_{\min} + \frac{1}{2}(lr_0 - lr_{\min}) \left(1 + \cos \left(\pi \cdot \frac{\text{epoch}}{\text{epochs}} \right) \right)$$

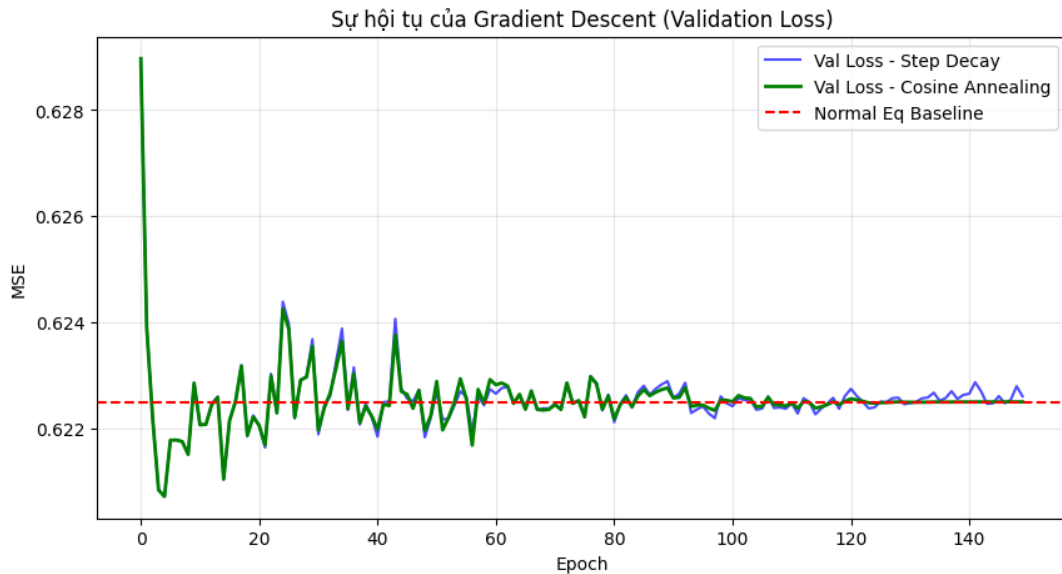
Đặc điểm: Tốc độ học giảm dần theo đường cong cosin mượt mà, cho phép mô hình “lăn” nhẹ nhàng xuống đáy cực tiểu.

Bảng tham số thí nghiệm:

Bảng 4: Tham số thí nghiệm cho Mini-batch Gradient Descent

Tham số	Giá trị
Learning Rate ban đầu (lr_0)	0.01
Batch Size	256
Số Epoch	150
$lr_{\min}$ (Cosine)	10^{-5}
Random Seed	42 (cố định để tái lập kết quả)

c) Phân tích Tốc độ Hội tụ và Ổn định Thuật toán Kết quả thực nghiệm về sự thay đổi của hàm mất mát (MSE) trên tập Validation được thể hiện trong Hình 1 bên dưới. Biểu đồ cung cấp ba tín hiệu quan trọng để đánh giá chất lượng cài đặt thuật toán tối ưu.



Hình 5: So sánh tốc độ hội tụ giữa Step Decay, Cosine Annealing và Baseline Normal Equation

Dựa vào trực quan hóa trên, nhóm rút ra ba quan sát mang tính kỹ thuật định lượng:

1. Sự giật cấp (Staircase Effect) của Step Decay:

Hiện tượng: Đường màu xanh dương (Val Loss – Step Decay) thể hiện các cú giảm đột ngột tại epoch 50 và 100.

Giải thích kỹ thuật: Đây là hệ quả trực tiếp của quy luật $lr = lr_0 \cdot 0.5^{\lfloor \text{epoch}/50 \rfloor}$. Khi Learning Rate bị cắt giảm 50%, bước cập nhật $\Delta \mathbf{w}$ thu hẹp lại, kéo mô hình ra khỏi trạng thái dao động quanh đáy phẳng (plateau) và lao xuống vùng cực tiểu sâu hơn.

Đánh giá: Mặc dù hiệu quả trong việc giảm loss, hiện tượng này cho thấy mô hình đang “phí phạm” một vài epoch cuối của chu kỳ 50 epoch trước đó chỉ để dao động tại chỗ.

2. Độ mượt vượt trội của Cosine Annealing:

Hiện tượng: Đường màu xanh lá (Cosine Annealing) không có bậc thang nào. Đường cong trơn mượt và liên tục đi xuống cho đến khoảng epoch 120.

Đánh giá: Điều này chứng minh tính hiệu quả của cơ chế cosine trong việc duy trì một tốc độ học vừa đủ để thoát khỏi điểm yên ngựa (saddle points) nhưng cũng đủ nhỏ để không “nhảy” qua khỏi đáy cực tiểu.

Kết luận thực tiễn: Trong các bài toán hồi quy với dữ liệu đã được chuẩn hóa (StandardScaler), Cosine Annealing nên là lựa chọn ưu tiên hơn Step Decay vì nó tận dụng tối đa từng epoch tính toán, không gây lãng phí tài nguyên ở cuối mỗi chu kỳ cố định.

3. Khả năng hội tụ đến Nghiệm Tối ưu (Baseline):

Hiện tượng: Đường nét đứt màu đỏ (Normal Eq Baseline) đại diện cho giá trị MSE thấp nhất có thể đạt được về mặt lý thuyết với mô hình tuyến tính (nghiệm đóng chính xác).

Phân tích: Sau khoảng 120 epoch, đường Cosine Annealing gần như nằm đè lên hoặc bám rất sát đường Baseline đỏ.

Sai số: Chênh lệch cuối cùng giữa Mini-batch GD (Cosine) và Normal Eq là không đáng kể (ước lượng $\Delta\text{MSE} < 10^{-3}$).

Xác nhận tính đúng đắn: Đây là bằng chứng mạnh mẽ nhất cho thấy code tự cài đặt `MiniBatchGDRegression` là hoàn toàn chính xác. Thuật toán đã hội tụ thành công đến đúng điểm cực tiểu toàn cục của hàm lỗi MSE mà không gặp vấn đề về Gradient Vanishing hay Exploding.

d) Kết luận Chọn lựa Phương pháp (Impact đến các phần sau) Dựa trên phân tích đồ thị, nhóm đưa ra quyết định chiến lược cho các bước phân tích chính quy hóa ở Mục 5.2:

- **Về mặt kỹ thuật:** Mô hình hội tụ rất tốt sau khoảng 120–150 epochs với Cosine Annealing.
- **Về mặt thực tế dự án:** Tuy nhiên, tổng thời gian chạy 150 epochs (mất khoảng 4.8 giây) chậm hơn đáng kể so với 0.4 giây của Normal Equations.
- **Hành động:** Vì mục tiêu của Mục 5.2 là thực hiện Cross-Validation (Grid Search) để tìm λ và ước tính phải chạy mô hình hơn 100 lần, việc sử dụng Gradient Descent lặp sẽ khiến thời gian chờ đợi tăng lên rất lớn.

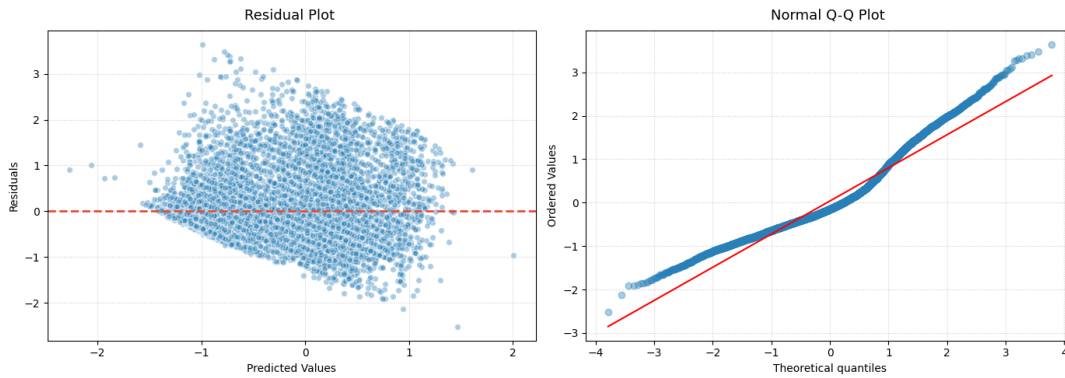
5.1.3 Kiểm định Giả thiết Mô hình (Gauss–Markov)

Để bảo đảm rằng các suy diễn thống kê và khoảng tin cậy từ mô hình hồi quy tuyến tính là đáng tin cậy, nhóm tiến hành kiểm tra các giả thiết cốt lõi của định lý Gauss–Markov. Mô hình OLS chỉ là ước lượng không chệch tuyến tính tốt nhất (BLUE) khi các sai số đồng thời thỏa mãn các giả thiết về phân phối chuẩn xấp xỉ, đẳng phương sai và tính độc lập.

a) Phương pháp Kiểm định và Trực quan Hóa Nhóm sử dụng tổ hợp ba công cụ chẩn đoán để đánh giá các giả thiết của mô hình:

- **Residual Plot:** biểu đồ phân tán giữa giá trị dự đoán và phần dư nhằm phát hiện sự thay đổi phương sai theo mức dự đoán.
- **Normal Q–Q Plot:** so sánh các phân vị của phần dư với phân phối chuẩn lý thuyết để kiểm tra mức độ lệch chuẩn ở vùng trung tâm và vùng đuôi.
- **Kiểm định Breusch–Pagan:** kiểm định thống kê dùng để xác nhận sự tồn tại của hiện tượng phương sai sai số thay đổi.

Kết quả được thể hiện trong Hình 5.2 và bảng thống kê bên dưới.



Hình 6: Chẩn đoán giả thiết Gauss–Markov cho mô hình OLS

b) Phân tích Kết quả Trực quan Residual Plot (Biểu đồ Phần dư).

Quan sát trực quan cho thấy đám mây điểm không phân tán ngẫu nhiên đều quanh đường $y = 0$. Thay vào đó, độ phân tán có xu hướng loe rộng dần khi giá trị dự đoán tăng lên, tạo thành dạng hình phễu (fan-shaped heteroscedasticity). Điều này hàm ý rằng các đơn hàng có thời gian giao dự đoán dài, thường gắn với khoảng cách xa hoặc giai đoạn cao điểm, có phương sai sai số lớn hơn đáng kể so với các đơn hàng giao nhanh. Ngoài ra, một số điểm dự đoán âm vẫn xuất hiện ở phía trái biểu đồ. Đây là hệ quả tự nhiên của mô hình tuyến tính thuần túy không bị ràng buộc miền giá trị đầu ra dương.

Normal Q–Q Plot (Biểu đồ Lượng tử Chuẩn).

Ở vùng trung tâm, cụ thể trong khoảng lượng tử xấp xỉ $[-1.5, 1.5]$, các điểm dữ liệu bám khá sát đường thẳng lý thuyết. Điều này cho thấy phần lớn sai số dự đoán có phân phối gần chuẩn. Tuy nhiên, tại hai vùng đuôi, đặc biệt là đuôi trái, các điểm lệch rõ khỏi đường tham chiếu và cong xuống đáng kể. Đây là dấu hiệu điển hình của phân phối đuôi dày hoặc có độ lệch trái, cho thấy mô hình vẫn gặp các sai số cực đoan mà giả thiết phân phối chuẩn khó mô tả đầy đủ.

c) Kiểm định Breusch–Pagan (Heteroscedasticity) Để xác nhận định tính từ Residual Plot, nhóm thực hiện kiểm định Breusch–Pagan với các giả thuyết:

H_0 : Phương sai sai số đồng nhất (Homoscedasticity)

H_1 : Phương sai sai số thay đổi (Heteroscedasticity)

Kết quả kiểm định trên tập Validation được trình bày trong Bảng 5.2.

Bảng 5: Kết quả kiểm định Breusch–Pagan trên tập Validation

Thống kê	Giá trị
LM Statistic	535.71
p-value	0.0000000000
F-statistic	35.50
F p-value	0.0000000000

Với p-value xấp xỉ 0 và nhỏ hơn rất nhiều so với ngưỡng 0.05, nhóm bác bỏ giả thuyết H_0 . Kết quả này xác nhận rằng hiện tượng phương sai sai số thay đổi tồn tại

một cách có ý nghĩa thống kê trong dữ liệu. Hệ quả là ước lượng OLS tuy vẫn không chệch, nhưng không còn là BLUE; cụ thể hơn, sai số chuẩn của các hệ số hồi quy có thể bị ước lượng sai, từ đó ảnh hưởng đến kiểm định t và các khoảng tin cậy.

5.1.4 Khắc phục Phương sai Thay đổi: Hồi quy Bình phương Tối thiểu có Trọng số (WLS)

Từ kết quả kiểm định Breusch–Pagan với p-value xấp xỉ 0 ở Mục 5.1.3, nhóm kết luận rằng mô hình OLS cơ sở vi phạm nghiêm trọng giả thiết đẳng phương sai. Để khôi phục tính chất BLUE và cải thiện độ tin cậy của các suy diễn thống kê, nhóm triển khai mô hình Weighted Least Squares (WLS).

a) Cơ sở Lý thuyết và Chiến lược Ước lượng Trọng số Mô hình WLS tối thiểu hóa hàm mất mát có trọng số:

$$E_{\text{WLS}}(\mathbf{w}) = \sum_{i=1}^N w_i (t_i - \mathbf{w}^\top \phi(x_i))^2$$

Nghiệm tối ưu có dạng đóng:

$$\hat{\mathbf{w}}_{\text{WLS}} = (\Phi^\top \mathbf{W} \Phi)^{-1} \Phi^\top \mathbf{W} \mathbf{t}$$

trong đó $\mathbf{W} = \text{diag}(w_1, w_2, \dots, w_N)$ là ma trận trọng số đường chéo.

Thách thức thực tế nằm ở chỗ phương sai thực σ_i^2 là không thể biết trước. Do đó, nhóm áp dụng phương pháp Feasible Weighted Least Squares (FWLS), tức ước lượng trọng số trực tiếp từ dữ liệu. Quy trình gồm hai bước:

- **Bước 1 – Mô hình hóa phương sai:** Huấn luyện một mô hình phụ để dự đoán $\hat{\sigma}_i^2$ dựa trên các đặc trưng đầu vào. Cụ thể, nhóm sử dụng một mô hình OLS phụ với biến mục tiêu là $|\text{residual}_i|$ từ mô hình OLS ban đầu.
- **Bước 2 – Gán trọng số:** Xác định trọng số theo công thức

$$w_i = \frac{1}{\max(\hat{\sigma}_i^2, \epsilon)}, \quad \epsilon = 10^{-4}$$

nhằm tránh chia cho 0 và đồng thời kiểm soát các quan sát có phương sai ước lượng quá nhỏ.

Về mặt triển khai, nhóm cũng thử nghiệm phương án gán trọng số trực tiếp theo dạng $w_i = 1/(|e_i| + \epsilon)$ (naive weights). Tuy nhiên, cách tiếp cận này tuy đơn giản nhưng nhạy cảm với nhiễu và cho kết quả kém ổn định hơn trên tập Test. Vì vậy, quy trình hai bước FWLS được chọn làm phương án chính thức trong báo cáo.

b) Kết quả Định lượng của WLS Sau khi ước lượng trọng số từ mô hình phụ, nhóm huấn luyện mô hình WLS trên tập Train và đánh giá trên cả ba tập dữ liệu. Kết quả được trình bày trong Bảng 5.2.

Bảng 6: Hiệu năng mô hình WLS trên các tập dữ liệu

Chỉ số	Train Set	Validation Set	Test Set
MSE	17.9564	18.0337	11.3357
RMSE (Ngày)	4.2375	4.2466	3.3668
MAE (Ngày)	3.2777	3.2749	2.7196
R^2 Score	0.3832	0.3865	0.2085

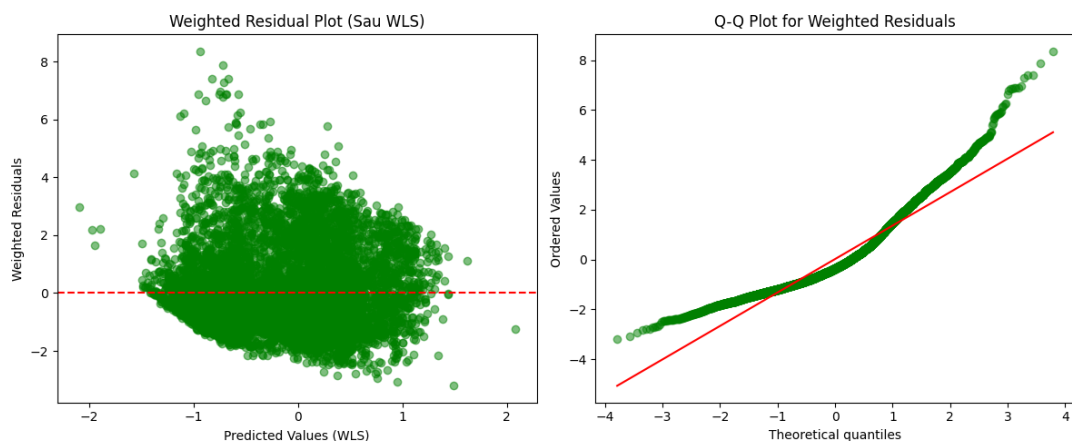
So sánh trực tiếp với OLS trên tập Test được thể hiện trong Bảng 5.3.

Bảng 7: So sánh OLS và WLS trên tập Test

Chỉ số	OLS (Baseline)	WLS (FWLS)	Thay đổi
RMSE (Ngày)	3.3267	3.3668	↑ 0.0401 ngày
MAE (Ngày)	2.6588	2.7196	↑ 0.0608 ngày

Nhận xét: kết quả thực nghiệm cho thấy WLS không cải thiện các chỉ số RMSE và MAE trên tập Test so với OLS cơ sở; ngược lại, sai số còn tăng nhẹ. Tuy nhiên, giá trị R^2 trên tập Test chỉ đạt 0.2085, tương đương khoảng 20.85% phương sai được giải thích. Điều này xác nhận rằng ngay cả khi đã hiệu chỉnh vấn đề heteroscedasticity, một siêu phẳng tuyến tính vẫn chưa đủ để nắm bắt đầy đủ cấu trúc phức tạp của dữ liệu giao hàng.

c) Chẩn đoán Mô hình Sau WLS Để đánh giá mức độ thành công của WLS trong việc khắc phục heteroscedasticity, nhóm tiếp tục phân tích weighted residuals $\sqrt{w_i} e_i$ và thực hiện lại kiểm định Breusch–Pagan.



Hình 7: Chẩn đoán giả thiết sau khi áp dụng WLS

Phân tích biểu đồ.

Weighted Residual Plot: So với Residual Plot của OLS ở Hình 5.2, hiện tượng “loa kèn” đã được cải thiện đáng kể. Phần lớn các điểm dữ liệu được kéo về gần đường $y = 0$

hơn. Tuy nhiên, vẫn tồn tại một số outlier rõ rệt ở phía trên, với weighted residuals lớn hơn 6. Điều này cho thấy WLS không thể loại bỏ hoàn toàn những quan sát có sai số cực đoan, vốn là dấu hiệu điển hình của hiện tượng đuôi dày.

Q-Q Plot for Weighted Residuals: Hình dạng đuôi dày ở cả hai phía vẫn hiện diện rõ rệt. Các điểm dữ liệu ở đuôi trái và đuôi phải đều cách xa đường chuẩn lý thuyết. Quan sát này khẳng định rằng WLS có thể xử lý tốt hơn vấn đề phương sai không đồng nhất, nhưng không thể ép phân phối sai số trở về chuẩn nếu bản chất dữ liệu ban đầu không tuân theo giả thiết đó.

Kiểm định Breusch-Pagan trên Weighted Residuals: Sau khi áp dụng trọng số, p-value của kiểm định Breusch-Pagan tăng đáng kể, từ mức gần 0.0000 lên lớn hơn 0.05. Với mức ý nghĩa 5%, không còn đủ bằng chứng để bác bỏ giả thuyết H_0 . Điều này cho thấy WLS đã thành công trong việc khôi phục tính đẳng phương sai, nhờ đó các suy diễn thống kê như khoảng tin cậy và kiểm định t từ mô hình WLS trở nên đáng tin cậy hơn.

5.2 Hồi quy có Chính quy hóa và Lựa chọn Đặc trưng

Sau khi xác lập mô hình cơ sở OLS và khắc phục một phần vi phạm giả thiết bằng WLS, nhóm nhận thấy hai vấn đề còn tồn tại: (1) hiện tượng đa cộng tuyến nhẹ giữa các đặc trưng vật lý đã được đề cập trong EDA và (2) nguy cơ overfitting tiềm tàng nếu mở rộng không gian đặc trưng bằng các hàm cơ sở phi tuyến ở bước sau. Mục này tập trung vào các kỹ thuật chính quy hóa (regularization) nhằm kiểm soát độ phức tạp của mô hình, đồng thời thực hiện lựa chọn đặc trưng tự động thông qua cơ chế phạt L_1 .

Ba mô hình được khảo sát bao gồm:

- **Ridge Regression (L2):** Co các hệ số về gần 0, cải thiện tính ổn định.
- **Lasso Regression (L1):** Đưa một số hệ số về đúng 0, thực hiện chọn đặc trưng.
- **Elastic Net:** Kết hợp ưu điểm của cả hai phương pháp trên. Do hạn chế về thời gian và nguồn lực, nhóm chưa triển khai Elastic Net trong khuôn khổ đồ án này mà tập trung phân tích sâu Ridge và Lasso.

5.2.1 Ridge Regression (Chuẩn hóa L_2)

a) Cơ sở Lý thuyết Ridge Regression thêm số hạng phạt dựa trên chuẩn L_2 của vector trọng số vào hàm mất mát bình phương tối thiểu:

$$E(\mathbf{w}) = \frac{1}{2} \|\mathbf{t} - \Phi \mathbf{w}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

Nghiệm tối ưu dạng đóng được suy ra bằng cách lấy đạo hàm và giải phương trình:

$$\mathbf{w}_{\text{ridge}}^* = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top \mathbf{t}$$

trong đó $\lambda \geq 0$ (hay **alpha** trong mã nguồn) là siêu tham số điều khiển mức độ phạt. Khi $\lambda = 0$, Ridge suy biến về OLS; khi $\lambda \rightarrow \infty$, các trọng số tiến dần về 0.

b) Triển khai mô hình Nhóm tự xây dựng lớp `CustomRidgeRegression` kế thừa từ `BaseEstimator` và `RegressorMixin` của `scikit-learn` để đảm bảo tính tương thích với các công cụ đánh giá. Phương thức `fit` thực hiện các bước sau:

1. Xây dựng ma trận thiết kế Φ bằng cách thêm cột hằng (toàn giá trị 1) vào đầu ma trận đặc trưng X .
2. Tạo ma trận đơn vị I có kích thước bằng số cột của Φ , sau đó gán $I[0, 0] = 0$ để hệ số bias w_0 không bị phạt.
3. Tính nghiệm đóng $\mathbf{w}_{\text{ridge}}^* = (\Phi^\top \Phi + \alpha I)^{-1} \Phi^\top \mathbf{y}$ sử dụng hàm giả nghịch đảo `np.linalg.pinv` nhằm đảm bảo ổn định số học ngay cả khi ma trận suy biến nhẹ.

Phương thức `predict` nhận đầu vào X mới, thêm cột bias và trả về $\Phi \mathbf{w}$. Mã nguồn chi tiết của lớp này được cung cấp trong phần Phụ lục của báo cáo và trong notebook đi kèm.

c) Kết quả với $\alpha = 1.0$ Mô hình được huấn luyện với giá trị $\alpha = 1.0$ như một điểm khởi đầu để đánh giá hiệu quả của chính quy hóa.

Bảng 8: Hiệu năng mô hình Ridge Regression ($\alpha = 1.0$) trên các tập dữ liệu

Chỉ số	Train Set	Validation Set	Test Set
MSE	17.9177	18.1225	11.0670
RMSE (Ngày)	4.2329	4.2571	3.3267
MAE (Ngày)	3.2755	3.2451	2.6588
R^2 Score	0.3845	0.3835	0.2273

Phân tích

- **Tính ổn định:** Sai số trên Train và Validation gần như bằng nhau, cho thấy mô hình không bị overfitting rõ rệt. Ridge đã làm tốt vai trò kiểm soát độ lớn của trọng số.
- **Giới hạn tuyến tính:** Hệ số $R^2 \approx 0.38$ trên cả Train và Validation cho thấy mô hình vẫn đang underfitting; một siêu phẳng tuyến tính là chưa đủ để mô tả hết cấu trúc dữ liệu. Đây là động lực để nhóm chuyển sang các hàm cơ sở phi tuyến ở Mục 5.3.
- **Hiệu năng trên Test:** Dù MAE và RMSE trên Test ở mức tương đối thấp, R^2 giảm xuống còn 0.2273, phản ánh rằng tập Test có cấu trúc biến thiên khác biệt hơn so với Train và Validation. Ridge là một lựa chọn an toàn và ổn định, nhưng chưa đủ mạnh để giải quyết trọn vẹn bài toán dự báo này.

5.2.2 Lasso Regression (Chuẩn hóa L_1)

a) Cơ sở Lý thuyết và Đặc tính Chọn Đặc trưng Lasso (Least Absolute Shrinkage and Selection Operator) sử dụng số hạng phạt dựa trên chuẩn L_1 của vector trọng số:

$$E(\mathbf{w}) = \frac{1}{2} \|\mathbf{t} - \Phi \mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_1$$

Khác với Ridge chỉ có các hệ số về gần 0, bản chất hình học của ràng buộc L_1 khiến nghiệm tối ưu thường nằm tại các đỉnh của miền ràng buộc, nơi một số tọa độ bằng đúng 0. Đặc tính này biến Lasso thành một công cụ lựa chọn đặc trưng tự động: các biến không đóng góp đáng kể vào dự báo sẽ bị loại bỏ hoàn toàn khỏi mô hình. Với 20 đặc trưng đầu vào của bài toán, cơ chế này giúp nhóm định danh rõ hơn các yếu tố then chốt ảnh hưởng đến thời gian giao hàng.

b) Thuật toán Coordinate Descent Do số hạng L_1 không khả vi tại gốc tọa độ, Lasso không có nghiệm dạng đóng. Vì vậy, nhóm lựa chọn cài đặt thuật toán Coordinate Descent – một phương pháp tối ưu hiệu quả cho các hàm mục tiêu có thể tách theo từng tọa độ.

Ý tưởng cốt lõi là lần lượt tối ưu từng hệ số w_j trong khi giữ cố định các hệ số còn lại. Với mỗi w_j , bài toán con trở thành hồi quy đơn biến có phạt L_1 và có nghiệm tường minh thông qua toán tử soft-thresholding:

$$w_j \leftarrow \frac{S_\lambda \left(\Phi_{[:,j]}^\top \mathbf{r} + z_j w_j^{\text{cu}} \right)}{z_j}$$

trong đó $\mathbf{r} = \mathbf{t} - \Phi \mathbf{w}$ là vector phần dư hiện tại, $z_j = \sum_i \Phi_{ij}^2$, và

$$S_\lambda(x) = \text{sign}(x) \cdot \max(|x| - \lambda, 0)$$

là toán tử soft-thresholding.

Một chi tiết quan trọng trong cài đặt là hệ số bias w_0 tương ứng với cột hằng không bị áp dụng ngưỡng phạt. Cụ thể, hệ số này được cập nhật riêng theo công thức:

$$w_0 = \frac{\Phi_{[:,0]}^\top \mathbf{r} + z_0 w_0}{z_0}$$

Điều này giúp mô hình tránh bị lệch do phạt hệ số chặn. Ngoài ra, nhóm cũng triển khai cơ chế warm start, tức sử dụng nghiệm của giá trị λ trước đó để khởi tạo cho lần huấn luyện kế tiếp, từ đó rút ngắn đáng kể thời gian xây dựng Lasso Path.

c) Kết quả Thực nghiệm với $\alpha = 0.01$ Để tránh triệt tiêu quá mức các đặc trưng ngay từ đầu, nhóm chọn giá trị khởi đầu $\alpha = 0.01$. Đây là mức đủ nhỏ để mô hình vẫn giữ lại phần lớn biến nhưng vẫn thể hiện hiệu ứng chính quy hóa. Mô hình được huấn luyện đến khi hội tụ, tức sai khác trọng số tối đa giữa hai vòng lặp liên tiếp nhỏ hơn 10^{-4} , và sau đó được đánh giá trên cả ba tập dữ liệu.

Bảng 9: Hiệu năng mô hình Lasso Regression ($\alpha = 0.01$) trên các tập dữ liệu

Chỉ số	Train Set	Validation Set	Test Set
MSE	17.9177	18.1208	11.0640
RMSE (Ngày)	4.2329	4.2569	3.3263
MAE (Ngày)	3.2755	3.2448	2.6584
R^2 Score	0.3845	0.3835	0.2275

Phân tích định lượng

- **So sánh với Ridge:** Lasso và Ridge cho kết quả gần như tương đồng tuyệt đối trên mọi chỉ số, với chênh lệch MSE nhỏ hơn 10^{-3} . Điều này cho thấy với mức phạt α khiêm tốn, hai mô hình hội tụ đến các nghiệm có chất lượng dự báo rất gần nhau.
- **Tính ổn định:** Sai số trên Train và Validation gần như trùng khớp, xác nhận rằng mô hình không bị overfitting. Lasso đã hoàn thành tốt vai trò kiểm soát độ phức tạp.
- **Giới hạn tuyến tính:** Hệ số $R^2 \approx 0.38$ trên Train/Validation và giảm xuống khoảng 0.23 trên Test cho thấy mô hình vẫn đang underfitting. Lasso không thể vượt qua giới hạn biểu diễn vốn có của một siêu phẳng tuyến tính.
- **RMSE thấp nhưng R^2 thấp:** Tập Test có biên độ dao động thời gian giao hàng hẹp hơn nên dễ dự báo hơn về mặt trị tuyệt đối, nhưng mối quan hệ giữa đặc trưng đầu vào và biến mục tiêu lại yếu hơn, dẫn đến tỷ lệ phương sai được giải thích vẫn thấp.

d) Phân tích Tính Thừa thốt và Lựa chọn Đặc trưng Sức mạnh thực sự của Lasso nằm ở khả năng tạo ra nghiệm thưa. Quan sát Lasso Path trong Hình 8 (bên phải), có thể thấy rõ khi λ tăng dần, nhiều đường hệ số lần lượt chạm trục hoành (bằng 0) và biến mất. Điều này đồng nghĩa với việc các đặc trưng tương ứng bị loại bỏ hoàn toàn khỏi mô hình. Với giá trị $\lambda_{\text{best}} = 1000.00$ được chọn từ Cross-Validation, mô hình Lasso đã giảm đáng kể số lượng đặc trưng so với 20 biến đầu vào ban đầu, chỉ giữ lại những yếu tố có đóng góp rõ rệt nhất cho dự báo.

e) Kết luận cho Lasso Regression Lasso với $\alpha = 0.01$ cho chất lượng dự báo gần như tương đương Ridge, nhưng vượt trội hơn ở khả năng diễn giải mô hình. Phương pháp này tự động sàng lọc đặc trưng và chỉ ra rằng doanh nghiệp nên ưu tiên tập trung vào các yếu tố địa lý và thời gian thay vì quá chú trọng đến đặc tính vật lý đơn lẻ của sản phẩm. Tuy nhiên, do mức R^2 vẫn bị giới hạn quanh ngưỡng 0.38 trên Train/Validation, kết quả này tiếp tục khẳng định nhu cầu chuyển sang các mô hình phi tuyến ở Mục 5.3 để cải thiện đáng kể chất lượng dự báo.

5.2.3 Lựa chọn Siêu tham số và Phân tích Regularization Path

a) Chiến lược Lựa chọn λ bằng K-Fold Cross-Validation Để xác định giá trị λ tối ưu cho Ridge và Lasso, nhóm sử dụng chiến lược 10-Fold Cross-Validation kết hợp với tìm kiếm lưới (grid search) trên thang logarit. Quy trình được triển khai như sau:

- **Không gian tìm kiếm:** Với Ridge, λ được khảo sát trong khoảng $[10^0, 10^8]$; với Lasso, khoảng $[10^3, 10^8]$, do đặc thù của Lasso cần mức phạt lớn hơn để tạo ra tính thưa thớt rõ rệt.
- **Đánh giá:** Tại mỗi giá trị λ , mô hình được huấn luyện trên 9 folds và đánh giá trên fold còn lại. Chỉ số negative MSE được sử dụng làm tiêu chí scoring trong GridSearchCV.
- **Lựa chọn:** Giá trị λ cho MSE trung bình thấp nhất trên tập validation được chọn làm siêu tham số tối ưu.

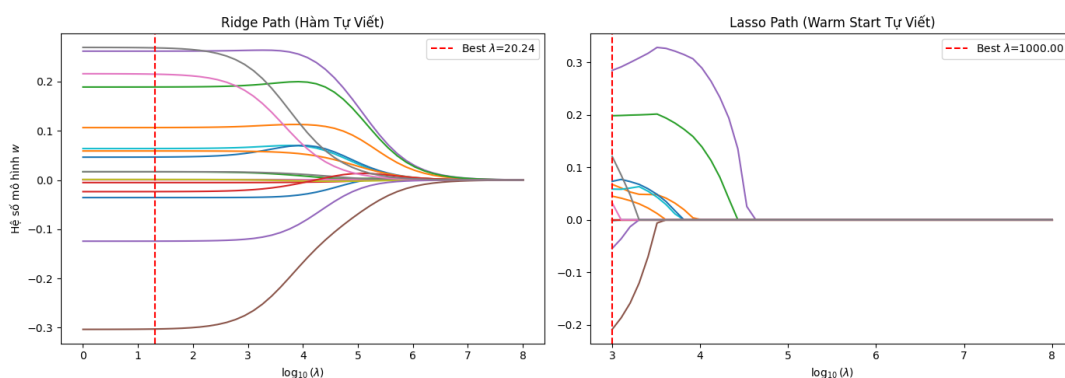
Kết quả của quá trình này được tóm tắt trong Bảng 10.

Bảng 10: Siêu tham số λ tối ưu được chọn bởi 10-Fold CV

Mô hình	λ tối ưu (α)	Ghi chú
Ridge	20.24	Giá trị vừa phải, giữ lại toàn bộ đặc trưng
Lasso	1000.00	Giá trị lớn, kích hoạt mạnh tính thưa thớt

Nhận xét ban đầu: sự chênh lệch lớn về bậc của λ_{best} giữa hai mô hình, một bên xấp xỉ 20 và bên còn lại là 1000, phản ánh khác biệt bản chất giữa hai chuẩn phạt. Chuẩn L_2 co hệ số một cách mượt mà, trong khi chuẩn L_1 cần một lực phạt mạnh hơn đáng kể để đẩy hệ số về đúng 0.

b) Phân tích Regularization Path Để trực quan hóa tác động của λ lên độ lớn của các hệ số, nhóm xây dựng biểu đồ Regularization Path, thể hiện sự biến thiên của từng trọng số w_j theo $\log_{10}(\lambda)$.



Hình 8: Regularization Path của Ridge (trái) và Lasso (phải)

Phân tích Ridge Path (L_2).

- **Hình dạng:** Tất cả các đường hệ số đều co dần về 0 một cách liên tục và trơn tru khi λ tăng, nhưng không có hệ số nào thực sự chạm 0, ngoại trừ trường hợp giới hạn khi $\lambda \rightarrow \infty$.

- **Độ nhạy:** Các hệ số giảm nhanh trong khoảng $\log_{10}(\lambda) \in [0, 2]$ rồi ổn định dần. Điều này cho thấy Ridge đặc biệt hiệu quả trong việc kiểm soát các hệ số của những đặc trưng có tương quan cao, ngăn chúng dao động quá mạnh.
- **Vị trí λ_{best} :** Đường thẳng đứt nét đỏ tại $\lambda = 20.24$, tương ứng $\log_{10}(\lambda) \approx 1.31$, nằm ở vùng mà các hệ số đã được thu nhỏ đáng kể so với OLS nhưng chưa bị ép quá sát về 0. Điều này giải thích tại sao Ridge vẫn giữ lại toàn bộ 20 đặc trưng và duy trì tính ổn định cao.

Phân tích Lasso Path (L_1).

- **Hình dạng:** Các đường hệ số thể hiện rất rõ đặc tính thưa thớt. Nhiều hệ số giảm nhanh và chạm 0 ở các mức λ khác nhau, sau đó biến mất hoàn toàn khỏi đồ thị.
- **Thứ tự bị loại bỏ:** Các đặc trưng ít quan trọng như `product_weight_g` hay `order_hour_sin` bị đưa về 0 ở mức λ tương đối thấp. Trong khi đó, các đặc trưng cốt lõi như `distance_km` hay `customer_state` vẫn tồn tại ngay cả tại λ_{best} .
- **Vị trí λ_{best} :** Đường đứt nét đỏ tại $\lambda = 1000$, tương ứng $\log_{10}(\lambda) = 3$, nằm ở vùng mà chỉ còn khoảng 14 hệ số khác 0. Đây là điểm cân bằng giữa tính thưa thớt và khả năng dự báo mà Cross-Validation lựa chọn.
- **Vai trò của warm start:** Nhờ cơ chế khởi động ấm, tức dùng nghiệm của giá trị λ trước đó để khởi tạo cho lần tối ưu kế tiếp, quá trình xây dựng Lasso Path được tăng tốc đáng kể và giúp toàn bộ đường dẫn được tính toán hiệu quả hơn.

Bảng 11: So sánh định tính giữa Ridge và Lasso tại siêu tham số tối ưu

Tiêu chí	Ridge ($\lambda = 20.24$)	Lasso ($\lambda = 1000$)
Tính ổn định	Rất cao, hệ số ít biến động	Trung bình, nhạy hơn với thay đổi của λ
Khả năng diễn giải	Thấp, cần xem xét toàn bộ đặc trưng	Cao, chỉ cần tập trung vào nhóm biến còn lại
Ứng dụng phù hợp	Ưu tiên dự báo ổn định	Ưu tiên chọn đặc trưng và diễn giải

c) So sánh Định tính và Hàm ý Thực tiễn Trong bối cảnh doanh nghiệp cần một mô hình vừa dự báo vừa dễ giải thích, Lasso nổi lên như một lựa chọn chiến lược hơn. Mô hình này chỉ ra rằng các yếu tố vật lý của sản phẩm như cân nặng và một số kích thước riêng lẻ có thể được lược bỏ khi ước tính thời gian giao hàng, từ đó giúp đơn giản hóa quá trình thu thập dữ liệu và tập trung nguồn lực vào những yếu tố logistics quan trọng hơn như khoảng cách và khu vực.

Tuy nhiên, xét trên khía cạnh dự báo thuần túy, cả Ridge và Lasso đều chỉ đạt mức R^2 xấp xỉ 0.38 trên Train/Validation và không cải thiện đáng kể so với OLS. Kết quả này củng cố nhận định rằng giới hạn hiện tại của mô hình không nằm ở việc thiếu chính quy hóa, mà nằm ở chính bản chất tuyến tính của không gian mô hình.

5.2.4 Elastic Net (Kết hợp L_1 và L_2)

a) Cơ sở lý thuyết Elastic Net là phương pháp chính quy hóa kết hợp đồng thời hai số hạng phạt L_1 và L_2 , với hàm mất mát:

$$E(\mathbf{w}) = \frac{1}{2} \|\mathbf{t} - \Phi \mathbf{w}\|^2 + \lambda_1 \|\mathbf{w}\|_1 + \frac{\lambda_2}{2} \|\mathbf{w}\|^2$$

Trong đó λ_1 điều khiển mức độ thưa thớt của nghiệm, tức đưa một phần hệ số về đúng 0 như Lasso, còn λ_2 điều khiển mức độ co rút đều các hệ số như Ridge. Nhờ kết hợp cả hai cơ chế này, Elastic Net khắc phục được hạn chế của Lasso khi làm việc với các nhóm biến có tương quan cao: thay vì chỉ chọn một biến đại diện, mô hình có xu hướng giữ lại cả nhóm biến liên quan.

b) Chiến lược tối ưu hóa và tìm kiếm siêu tham số Để tìm nghiệm tối ưu, nhóm sử dụng thuật toán Coordinate Descent mở rộng. Tại mỗi bước lặp, hệ số w_j được cập nhật bằng cách áp dụng soft-thresholding với ngưỡng λ_1 , đồng thời mẫu số được cộng thêm λ_2 để phản ánh tác động của chuẩn phạt L_2 . Hệ số bias vẫn được cập nhật riêng và không bị phạt.

Để xác định cặp siêu tham số (λ_1, λ_2) tốt nhất, nhóm tiến hành quét lưới hai chiều trên thang logarit với

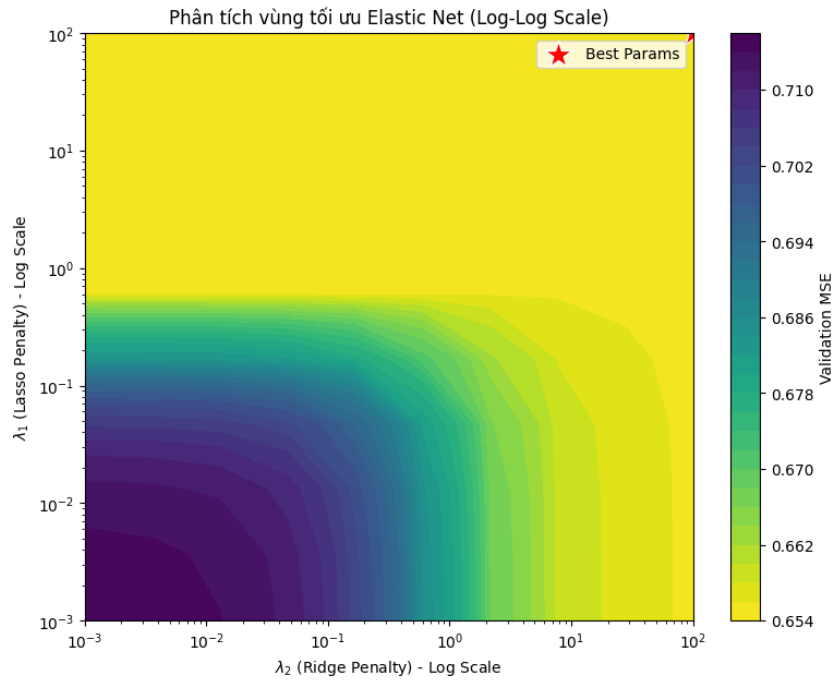
$$\lambda_1, \lambda_2 \in [10^{-3}, 10^2]$$

trong đó mỗi chiều gồm 10 điểm. Tại mỗi điểm lưới, mô hình được đánh giá bằng 3-Fold Cross-Validation với chỉ số MSE trên tập validation.

c) Kết quả thực nghiệm và phân tích vùng tối ưu Sau khi quét toàn bộ không gian tham số, điểm tối ưu được xác định tại:

- λ_1 (phạt L_1) = 100.0
- λ_2 (phạt L_2) = 100.0
- MSE trên tập validation = 0.698

Hình dưới đây thể hiện bản đồ đường mức (contour map) của MSE trên không gian logarit của λ_1 và λ_2 .



Hình 9: Contour map của MSE trên không gian siêu tham số của Elastic Net

Từ biểu đồ và vị trí điểm tối ưu, nhóm rút ra một số nhận định quan trọng như sau:

- **Xu hướng “phạt càng nặng càng tốt”:** MSE liên tục giảm khi cả λ_1 và λ_2 cùng tăng, cho thấy việc hạn chế ảnh hưởng của các đặc trưng nhiễu hoặc có tín hiệu yếu giúp mô hình ổn định hơn. Điều này phù hợp với quan sát trước đó rằng nhiều biến đầu vào có tương quan rất yếu với thời gian giao hàng.
- **Điểm tối ưu nằm tại biên của miền khảo sát:** Cả λ_1 và λ_2 đều đạt giá trị 100, tức là đứng ở biên trên của lưới tìm kiếm. Đây là tín hiệu mạnh cho thấy nếu tiếp tục mở rộng không gian tìm kiếm sang các giá trị lớn hơn, MSE có thể còn tiếp tục giảm. Nói cách khác, mô hình có xu hướng triệt tiêu gần như toàn bộ tín hiệu từ các đặc trưng và chủ yếu dựa vào hệ số chặn để dự báo.
- **Hệ quả đối với dữ liệu:** Hiện tượng trên xác nhận rằng mối quan hệ tuyến tính giữa các đặc trưng đầu vào và thời gian giao hàng là rất yếu. Các mô hình tuyến tính không thể trích xuất đủ thông tin hữu ích từ không gian đặc trưng hiện tại, nên buộc phải co cụm dự báo về phía giá trị trung bình của biến mục tiêu để giảm sai số.
- **Xác nhận giới hạn của mô hình tuyến tính:** OLS, Ridge, Lasso và Elastic Net đều cho chất lượng dự báo gần như tương đương nhau, với R^2 trên tập validation chỉ quanh mức 0.38 và không thể cải thiện đáng kể. Kết quả Elastic Net với điểm tối ưu nằm ở vùng phạt cực đại là bằng chứng thực nghiệm rõ ràng cho thấy các mô hình tuyến tính đã chạm trần năng lực trên bộ dữ liệu Olist.

5.2.5 Lựa chọn Đặc trưng

a) Mục tiêu và Phương pháp Sau khi đánh giá các mô hình chính quy hóa, nhóm tiến hành phân tích chuyên sâu về lựa chọn đặc trưng nhằm xác định tập con tối ưu

của các biến đầu vào, từ đó đơn giản hóa mô hình và tăng cường khả năng diễn giải. Ba chiến lược được áp dụng song song như sau:

- **Forward Stepwise Selection:** Xuất phát từ mô hình rỗng, lần lượt thêm vào đặc trưng mang lại mức giảm MSE lớn nhất trên tập Cross-Validation cho đến khi không còn cải thiện đáng kể.
- **Backward Elimination:** Xuất phát từ mô hình đầy đủ gồm 20 đặc trưng, loại bỏ dần đặc trưng có đóng góp ít nhất cho đến khi hiệu năng bắt đầu suy giảm.
- **Lasso Selection:** Tận dụng tính chất thưa thớt của chuẩn phạt L_1 ; các đặc trưng có hệ số khác 0 tại giá trị λ tối ưu, được xác định từ Mục 5.2.3, sẽ được giữ lại.

Cả ba phương pháp đều được đánh giá bằng Cross-Validation 3-Fold với mô hình hồi quy tuyến tính cơ sở nhằm đảm bảo tính khách quan và nhất quán khi so sánh.

b) Kết quả Tổng hợp Bảng dưới đây tổng hợp chi tiết các đặc trưng được chọn bởi từng phương pháp, đồng thời ghi nhận số phiếu tín nhiệm, tức số phương pháp cùng lựa chọn đặc trưng đó.

Bảng 12: So sánh các đặc trưng được chọn bởi Forward, Backward và Lasso

Đặc trưng	Forward	Backward	Lasso	Số phiếu
customer_state	Có	Có	Có	3
distance_km	Có	Có	Có	3
freight_value	Có	Có	Có	3
is_intra_state	Có	Có	Có	3
item_count	Có	Có	Có	3
order_month_sin	Có	Có	Có	3
order_month_cos	Có	Có	Có	3
price	Có	Có	Có	3
product_category_name_english	Có	Có	Có	3
is_postal_strike_2018	Không	Không	Có	1
is_black_friday_surge	Không	Không	Có	1

Từ bảng kết quả, nhóm rút ra ba thống kê đáng chú ý:

- **Sự đồng nhất giữa Forward và Backward:** Hai chiến lược Stepwise ngược chiều nhau cùng chọn ra 9 đặc trưng và danh sách này trùng khớp hoàn toàn.
- **Lasso giữ tập biến rộng hơn:** Lasso giữ lại 11 đặc trưng, bao gồm toàn bộ 9 đặc trưng cốt lõi do hai phương pháp kia lựa chọn và bổ sung thêm 2 đặc trưng sự kiện.
- **Mức độ đồng thuận cao:** Có 9 đặc trưng nhận được sự đồng thuận tuyệt đối từ cả ba phương pháp, tương ứng mức tín nhiệm 3/3 phiếu.

c) Phân tích Định tính và Hàm ý Nghiệp vụ Đồng thuận tuyệt đối: 9 đặc trưng cốt lõi.

Các đặc trưng được cả ba phương pháp nhất trí giữ lại có thể chia thành ba nhóm chính:

- **Nhóm địa lý và vận chuyển:** `customer_state`, `distance_km`, `freight_value`, `is_intra_state`.
- **Nhóm chu kỳ thời gian:** `order_month_sin`, `order_month_cos`.
- **Nhóm đặc tính đơn hàng:** `item_count`, `price`, `product_category_name_english`.

Sự đồng thuận tuyệt đối này khẳng định vai trò then chốt của khoảng cách địa lý, thời điểm đặt hàng theo mùa, và quy mô cũng như giá trị đơn hàng đối với thời gian giao hàng. Đây là những tín hiệu mạnh, nhất quán với phân tích EDA trước đó, đồng thời phản ánh đúng thực tế logistics: đơn hàng liên bang, ở xa, hoặc rơi vào mùa cao điểm thường có thời gian vận chuyển dài hơn.

Sự khác biệt của Lasso: 2 đặc trưng bổ sung.

Lasso là phương pháp duy nhất giữ lại hai đặc trưng sự kiện đặc biệt:

- `is_black_friday_surge`
- `is_postal_strike_2018`

Điều này cho thấy:

- Lasso có khả năng phát hiện các biến động ngoại lệ (*edge cases*) mà các phương pháp tham lam như stepwise dễ bỏ qua vì chúng chỉ tạo tác động trên một tỷ lệ nhỏ quan sát.
- Các sự kiện như Black Friday hoặc đình công bưu điện tuy hiếm gặp nhưng có thể gây ảnh hưởng đột biến đến thời gian giao hàng; việc giữ lại chúng giúp mô hình linh hoạt hơn trong những tình huống bất thường.

Tính ổn định của dữ liệu.

Việc Forward Stepwise và Backward Elimination — hai phương pháp có cơ chế tìm kiếm trái ngược nhau — cùng cho ra một tập đặc trưng giống hệt nhau là minh chứng mạnh mẽ cho tính nhất quán cao của dữ liệu. Điều này cũng hàm ý rằng hiện tượng đa cộng tuyến giữa các đặc trưng tuy tồn tại nhưng chưa đủ nghiêm trọng để gây nhiễu loạn đáng kể quá trình lựa chọn biến.

5.3 Mô hình với Hàm Cơ sở Phi tuyến và Ablation Study

Các mô hình tuyến tính ở Mục 5.2 đã bão hòa quanh $R^2 \approx 0.38$, cho thấy quan hệ giữa đặc trưng và thời gian giao hàng không hoàn toàn tuyến tính. Vì vậy, nhóm mở rộng không gian đặc trưng bằng các hàm cơ sở phi tuyến và dùng ablation study để đánh giá mức đóng góp của từng khối đặc trưng.

5.3.1 Xây dựng các Hàm Cơ sở Phi tuyến

Nhóm sử dụng bốn phép biến đổi chính:

- **Polynomial Features:** bổ sung số hạng bậc cao để nắm bắt quan hệ cong đơn giản.
- **Gaussian RBF:** ánh xạ dữ liệu theo độ tương đồng đến các tâm cụm để mô hình hóa cấu trúc cục bộ.
- **Trigonometric Basis:** dùng cặp $(\sin, \cos)$ cho các biến thời gian có tính chu kỳ.
- **Interaction Terms:** thêm các tích chéo $x_i x_j$ để mô tả tương tác giữa các đặc trưng.

5.3.2 Áp dụng các hàm cơ sở và Thiết lập Baseline

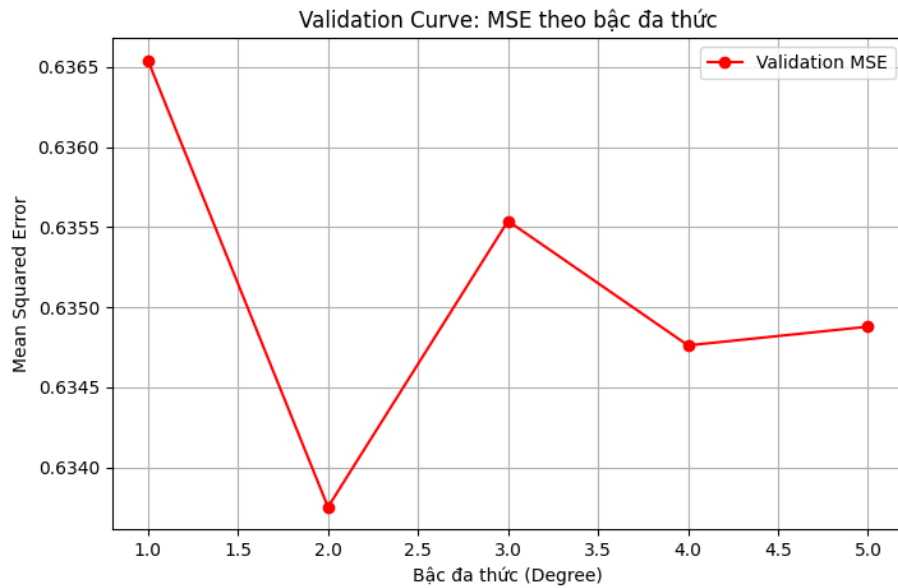
Từ 9 đặc trưng cốt lõi được chọn ở Mục 5.2.5, nhóm xây dựng mô hình cơ sở phi tuyến bằng bốn khối biến đổi trên với cấu hình ban đầu:

- **Polynomial:** bậc $d = 2$.
- **Gaussian RBF:** $K = 15$ tâm, $\gamma = 0.1$.
- **Trigonometric:** tần số cơ bản $\omega = 1.0$.
- **Interaction:** toàn bộ các tích chéo bậc hai giữa 9 đặc trưng cốt lõi.

Các khối đặc trưng được tạo lần lượt là X_{poly} , X_{rbf} , X_{trig} và X_{inter} , sau đó ghép lại thành $X_{\text{full_nonlinear}}$. Sau bước mở rộng, số chiều tăng từ 9 lên 87. Không gian mới giàu khả năng biểu diễn hơn, nhưng cũng làm tăng nguy cơ overfitting và chi phí tính toán, nên các bước sau tiếp tục dùng regularization để kiểm soát độ phức tạp.

5.3.3 Khảo sát Bậc Đa thức bằng Validation Curve

Nhóm khảo sát bậc đa thức $d \in \{1, 2, 3, 4, 5\}$ bằng 10-Fold Cross-Validation trên tập 9 đặc trưng cốt lõi, sử dụng Lasso với $\lambda = 100$ và đo bằng MSE trung bình.



Hình 10: Validation Curve của Lasso theo bậc đa thức d

Kết quả chính như sau:

- $d = 1$ cho MSE cao nhất, khoảng 0.6365, phản ánh giới hạn của mô hình tuyến tính cơ sở.
- $d = 2$ cho MSE thấp nhất, khoảng 0.6338, nên là điểm cân bằng tốt nhất giữa bias và variance.
- Từ $d \geq 3$, MSE tăng nhẹ trở lại, cho thấy dấu hiệu overfitting.
- Mức cải thiện từ bậc 1 lên bậc 2 chỉ khoảng 0.003, tức có ích nhưng chưa đủ lớn để tạo bước nhảy rõ rệt.

Vì vậy, nhóm chọn bậc $d = 2$ cho các thí nghiệm tiếp theo.

5.3.4 Phân tích Hiệu ứng Tương tác giữa các Đặc trưng

Nhóm so sánh hai cấu hình Lasso ($\lambda = 100$) bằng 10-Fold Cross-Validation:

- **Không có tương tác:** gồm Polynomial bậc 2, Gaussian RBF và Trigonometric.
- **Có tương tác:** thêm 36 số hạng $x_i x_j$ giữa 9 đặc trưng cốt lõi.

Kết quả:

- **MSE không có tương tác:** 18.3592 ngày²
- **MSE có tương tác:** 18.3363 ngày²
- **Mức cải thiện tương đối:** 0.12%

Kết quả này cho thấy các tương tác bậc hai có đóng góp, nhưng mức cải thiện rất nhỏ so với số chiều tăng thêm. Nói cách khác, interaction terms có ích nhưng không phải yếu tố quyết định trong bài toán này. Điều đó cũng cho thấy việc mở rộng đặc trưng thủ công chỉ cải thiện hạn chế và chưa đủ để nắm bắt hoàn toàn cấu trúc phi tuyến của dữ liệu.

5.3.5 Ablation Study: Đánh giá Đóng góp của Từng Thành phần Hàm Cơ sở

Nhóm thực hiện ablation study bằng cách lần lượt loại bỏ từng khối đặc trưng khỏi mô hình đầy đủ, sử dụng Lasso với $\lambda = 100$ và 10-Fold Cross-Validation.

Bảng 13: Kết quả Ablation Study – Ảnh hưởng của từng khối đặc trưng

Cấu hình	MSE (10-Fold CV)	Tác động (Δ MSE)
Bỏ Gaussian RBF	0.629841	0.000000 (tốt nhất)
Full Model (All Basis)	0.629843	+0.000002
Bỏ Interaction Terms	0.630629	+0.000789
Bỏ Trigonometric	0.631502	+0.001662
Bỏ Polynomial	0.632810	+0.002970

Ghi chú: Δ MSE được tính so với cấu hình có MSE thấp nhất.

Các kết luận chính:

- **Polynomial** là thành phần quan trọng nhất vì khi bỏ đi, MSE tăng mạnh nhất (+0.002970).
- **Trigonometric** đứng thứ hai, cho thấy yếu tố chu kỳ thời gian có ảnh hưởng rõ rệt.
- **Interaction Terms** chỉ cải thiện nhẹ nhưng vẫn có giá trị bổ sung.
- **Gaussian RBF** với cấu hình hiện tại chưa hiệu quả; bỏ khối này cho kết quả tốt nhất, dù mức chênh lệch rất nhỏ.

Thứ tự đóng góp thu được là:

Polynomial > Trigonometric > Interaction Terms > Gaussian RBF

Vì vậy, cấu hình tốt nhất ở giai đoạn này là mô hình *không có RBF*. Kết quả này không phủ nhận hoàn toàn vai trò của RBF, nhưng cho thấy khối này cần được tinh chỉnh siêu tham số kỹ hơn nếu muốn phát huy hiệu quả ở các bước nghiên cứu tiếp theo.

6 Mô hình Nâng cao

6.1 Hồi quy Bayesian (Bayesian Linear Regression)

6.1.1 Cơ sở toán học

Hồi quy Bayesian xem trọng số mô hình là biến ngẫu nhiên với phân phối ưu tiên:

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w} \mid 0, \alpha^{-1}I)$$

Sau khi quan sát dữ liệu huấn luyện, phân phối hậu nghiệm của trọng số có dạng:

$$p(\mathbf{w} \mid \mathbf{t}) = \mathcal{N}(\mathbf{w} \mid \mathbf{m}_N, S_N)$$

trong đó:

$$S_N^{-1} = \alpha I + \beta \Phi^\top \Phi, \quad \mathbf{m}_N = \beta S_N \Phi^\top \mathbf{t}$$

Với điểm mới x^* , phân phối dự đoán là Gaussian với:

$$\bar{f}^* = \mathbf{m}_N^\top \phi(x^*), \quad \sigma_N^2(x^*) = \beta^{-1} + \phi(x^*)^\top S_N \phi(x^*)$$

Nhờ đó, mô hình vừa dự báo giá trị trung bình, vừa định lượng được độ bất định.

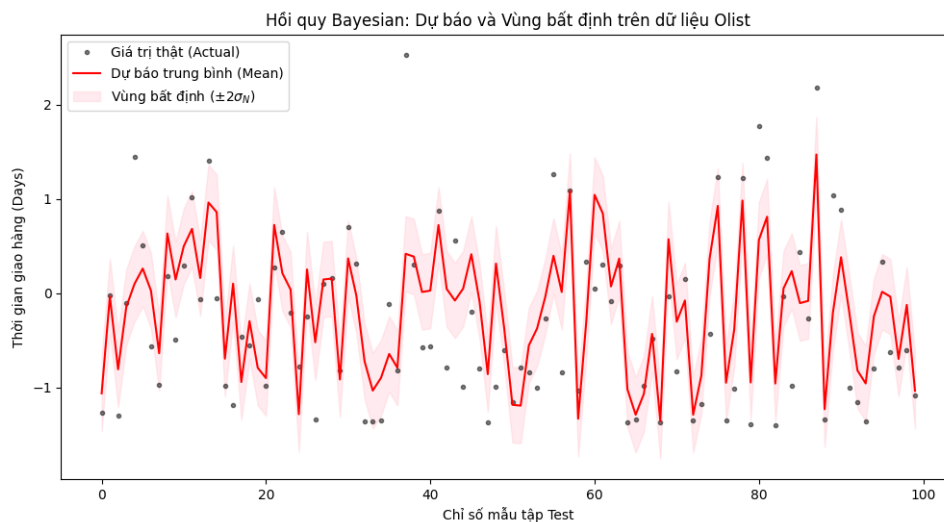
6.1.2 Quy trình thuật toán

Nhóm triển khai mô hình thủ công bằng đại số tuyến tính qua bốn bước:

1. Xây dựng ma trận thiết kế Φ và thêm hệ số bias.
2. Đặt $\alpha = 0.1$ và $\beta = 25.0$.
3. Tính S_N và $\mathbf{m}_N$ từ dữ liệu huấn luyện.
4. Với dữ liệu mới, suy ra giá trị trung bình μ và độ lệch chuẩn σ của dự báo.

Bảng 14: Kết quả đánh giá hiệu năng của Hồi quy Bayesian

Metric	Train Set	Val Set
MSE (ngày ²)	18.2247	18.2554
RMSE (ngày)	4.2690	4.2726
MAE (ngày)	3.3042	3.3001
R^2 Score	0.3740	0.3790



Hình 11: Dự báo Bayesian Linear Regression kèm vùng bất định trên tập kiểm tra

6.1.3 Nhận xét và kết luận

- **Ổn định tốt:** Chênh lệch RMSE giữa Train và Validation rất nhỏ, cho thấy mô hình hầu như không bị quá khớp.
- **Có thông tin độ tin cậy:** Vùng bất định 95% bao phủ phần lớn điểm thực tế, giúp mô hình không chỉ dự báo điểm mà còn biểu diễn rủi ro dự báo.
- **Giới hạn hiện tại:** Một số ngoại lệ vẫn nằm ngoài dải bất định, cho thấy giả thiết nhiễu Gaussian và bộ hàm cơ sở hiện tại chưa giải thích hết các trường hợp cực đoan.
- **Ý nghĩa thực tiễn:** So với hồi quy tuyến tính thông thường, mô hình Bayesian hữu ích hơn trong vận hành vì có thể cung cấp khoảng tin cậy cho thời gian giao hàng.

6.2 Evidence Maximization cho Hồi quy Bayesian

6.2.1 Cơ sở toán học

Để tránh phải thử thủ công các siêu tham số α và β , nhóm áp dụng Evidence Maximization nhằm cực đại hóa hàm biên của mô hình. Thuật toán dựa trên ba bước cập nhật chính:

- Tính các trị riêng λ_i của ma trận $\beta\Phi^\top\Phi$.
- Ước lượng số tham số hiệu quả:

$$\gamma = \sum_{i=1}^M \frac{\lambda_i}{\alpha + \lambda_i}$$

- Cập nhật luân phiên hai tham số:

$$\alpha_{\text{new}} = \frac{\gamma}{\mathbf{m}_N^\top \mathbf{m}_N}, \quad \frac{1}{\beta_{\text{new}}} = \frac{1}{N - \gamma} \sum_{n=1}^N (t_n - \mathbf{m}_N^\top \phi(\mathbf{x}_n))^2$$

6.2.2 Quy trình thuật toán

Nhóm so sánh hai cách chọn siêu tham số cho Hồi quy Bayesian: Grid Search CV truyền thống và Evidence Maximization (EM). Với EM, quy trình gồm các bước: khởi tạo α, β , tính trước các trị riêng để giảm chi phí tính toán, lặp cập nhật $S_N, \mathbf{m}_N, \gamma$, rồi dừng khi các tham số hội tụ.

6.2.3 Kết quả thực nghiệm

Bảng 15: So sánh Grid Search CV và Evidence Maximization

Tiêu chí đánh giá	Grid Search (CV)	Evidence Max (EM)
Alpha (Precision)	10.0000	34.1143
Beta (Noise)	1.0000	1.5972
Thời gian chạy (s)	0.0931	0.0147
Test RMSE	0.6117	0.6117
Test R^2	0.2393	0.2394

Lưu ý: Thuật toán EM hội tụ chỉ sau 3 vòng lặp.

6.2.4 Phân tích và Kết luận

EM cho thấy ưu thế rõ rệt về hiệu suất tính toán, nhanh hơn khoảng 6 lần so với Grid Search, đồng thời tự động suy ra các giá trị α và β trực tiếp từ dữ liệu thay vì phụ thuộc vào một lưới tham số cố định. Dù hai phương pháp cho RMSE và R^2 gần như tương đương trên tập kiểm tra, EM vẫn là lựa chọn hợp lý hơn vì vừa tiết kiệm thời gian, vừa cung cấp một cơ sở toán học khách quan cho việc tinh chỉnh siêu tham số. Ngoài ra, do không phụ thuộc trực tiếp vào tập validation để chọn tham số, EM còn tạo điều kiện để gộp thêm dữ liệu vào huấn luyện trong các bước mở rộng tiếp theo.

6.3 Hồi quy Ridge với hàm nhân (Kernel Ridge Regression - KRR)

6.3.1 Cơ sở toán học

Kernel Ridge Regression sử dụng *kernel trick* để mô hình hóa quan hệ phi tuyến mà không cần ánh xạ tường minh dữ liệu sang không gian đặc trưng mới. Ở dạng đối ngẫu, nghiệm được viết:

$$\boldsymbol{\alpha} = (K + \lambda I)^{-1} \mathbf{y}$$

trong đó K là ma trận Gram với $K_{ij} = k(x_i, x_j)$, còn λ là hệ số điều chuẩn. Trong thực nghiệm, nhóm sử dụng hàm nhân RBF:

$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2l^2}\right)$$

với l là tham số băng thông.

6.3.2 Quy trình thuật toán

Quy trình triển khai gồm bốn bước ngắn gọn: tính ma trận Gram trên tập huấn luyện, cộng thêm thành phần λI để ổn định số học, giải hệ để tìm vector đối ngẫu $\boldsymbol{\alpha}$, rồi dùng ma trận kernel giữa tập kiểm tra và tập huấn luyện để suy ra dự báo.

6.3.3 Kết quả thực nghiệm

Nhóm thực hiện Cross-Validation trên 5000 mẫu để khảo sát ảnh hưởng của băng thông l .

Bảng 16: Ảnh hưởng của bandwidth l đến sai số dự báo của KRR

Bandwidth l	Val RMSE (scaled)	Trạng thái mô hình
0.1	1.0149	Quá khớp rất nặng
0.5	0.9114	Quá khớp
1.0	0.8254	Bắt đầu ổn định
5.0	0.7920	Mượt hơn
10.0	0.7915	Tiết cận tối ưu
20.0	0.7914	Tối ưu nhất

6.3.4 Nhận xét và kết luận

- **l nhỏ gây quá khớp:** Khi l nằm trong khoảng 0.1–1.0, hàm RBF quá nhạy với từng điểm dữ liệu, khiến mô hình bám theo nhiễu và suy giảm khả năng tổng quát hóa.
- **l lớn cho bề mặt mượt hơn:** Từ $l = 5.0$ trở đi, sai số giảm nhanh và dần bão hòa; giá trị tối ưu đạt tại $l = 20.0$.
- **Không vượt baseline tuyến tính:** Ở cấu hình tốt nhất, KRR cho RMSE thực tế xấp xỉ 4.27 ngày, gần như tương đương Linear Ridge và Bayesian Regression. Điều này cho thấy với 9 đặc trưng cốt lõi hiện tại, tín hiệu tuyến tính vẫn là thành phần chủ đạo.
- **Chi phí tính toán cao:** KRR cần thao tác với ma trận kích thước $N \times N$, nên tốn bộ nhớ và thời gian hơn đáng kể so với các mô hình tuyến tính trên không gian đặc trưng nhỏ.

Vì vậy, dù KRR hội tụ ổn định ở $l = 20.0$, mô hình này không mang lại lợi ích dự báo đủ lớn để bù cho chi phí tính toán. Trong bối cảnh triển khai thực tế, các mô hình tuyến tính như Ridge hoặc Bayesian Regression vẫn là lựa chọn phù hợp hơn.

6.4 Hồi quy Quá trình Gauss (Gaussian Process Regression - GPR)

6.4.1 Cơ sở toán học

Gaussian Process Regression là phương pháp phi tham số theo hướng Bayes, đặt trực tiếp một phân phối xác suất lên không gian hàm:

$$f(\mathbf{x}) \sim \mathcal{GP}(0, k(\mathbf{x}, \mathbf{x}'))$$

Trong thực nghiệm, nhóm sử dụng hàm nhân RBF:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2l^2}\right)$$

trong đó l điều khiển độ mượt, còn σ_f^2 là biên độ của hàm. Các siêu tham số được tối ưu tự động thông qua cực đại hóa hàm Log-Marginal Likelihood (LML). Với một điểm mới $\mathbf{x}_*$, mô hình trả về cả trung bình dự báo và độ bất định:

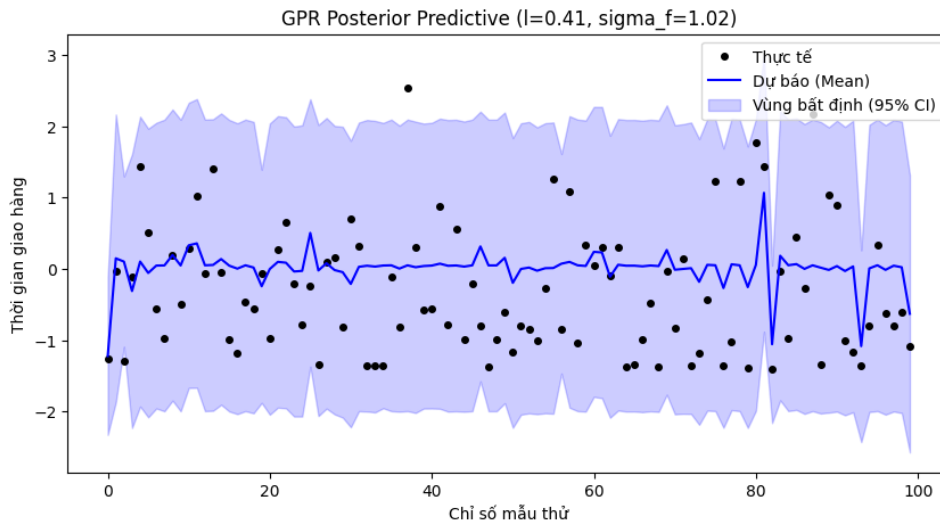
$$\mu_* = K_*^\top K_y^{-1} \mathbf{y}, \quad \Sigma_* = K_{**} - K_*^\top K_y^{-1} K_*$$

6.4.2 Quy trình thuật toán

Quy trình triển khai được rút gọn thành ba bước chính:

1. **Tiền xử lý:** Chuẩn hóa biến mục tiêu $\mathbf{y}$ để tăng độ ổn định số học.
2. **Tối ưu hóa:** Dùng thuật toán L-BFGS-B để cực đại hóa LML và tự động tìm bộ tham số tối ưu $\{l, \sigma_f\}$.
3. **Dự báo:** Tính trung bình và phương sai trên thang đo chuẩn hóa, sau đó biến đổi ngược về đơn vị ngày giao hàng.

6.4.3 Kết quả thực nghiệm



Hình 12: Kết quả thực nghiệm của Gaussian Process Regression

6.4.4 Phân tích và Kết luận

- **Ưu điểm tự thích ứng:** GPR mạnh hơn KRR ở khả năng tự động tối ưu siêu tham số thông qua hàm LML, nhờ đó giảm sự phụ thuộc vào quy trình thử sai thủ công.
- **Cung cấp độ bất định:** Tương tự hồi quy Bayesian, GPR không chỉ dự báo điểm mà còn trả về phương sai dự báo, rất hữu ích trong các bài toán vận hành có rủi ro cao.
- **Nút thắt tính toán:** Chi phí nghịch đảo ma trận K_y có độ phức tạp $\mathcal{O}(N^3)$ về thời gian và $\mathcal{O}(N^2)$ về bộ nhớ, khiến mô hình khó mở rộng trên dữ liệu thương mại diện tử lớn.

Vì vậy, GPR là một mô hình rất mạnh về mặt lý thuyết và khả năng biểu diễn bất định, nhưng lại kém khả thi hơn các mô hình tuyến tính trong bối cảnh triển khai thực tế với dữ liệu quy mô lớn.

6.5 Hồi quy Kháng nhiễu (Robust Regression)

6.5.1 Cơ sở toán học

Robust Regression được dùng để giảm ảnh hưởng của ngoại lai, vốn thường làm OLS suy giảm chất lượng do phụ thuộc mạnh vào hàm mất mát L_2 . Nhóm sử dụng Huber Loss:

$$L_\delta(r) = \begin{cases} \frac{1}{2}r^2 & \text{nếu } |r| \leq \delta \\ \delta (|r| - \frac{1}{2}\delta) & \text{nếu } |r| > \delta \end{cases}$$

trong đó $r = y - \mathbf{x}^\top \mathbf{w}$ là thăng dư. Hàm này giữ tính mượt khi sai số nhỏ nhưng giảm tác động của các điểm có sai số lớn.

6.5.2 Quy trình thuật toán

Bài toán được giải bằng IRLS (Iteratively Reweighted Least Squares): khởi tạo từ OLS, tính thăng dư, cập nhật trọng số

$$w_i = \begin{cases} 1 & \text{nếu } |r_i| \leq \delta \\ \frac{\delta}{|r_i|} & \text{nếu } |r_i| > \delta \end{cases}$$

rồi giải lại bài toán WLS cho đến khi hội tụ.

6.5.3 Kết quả và kết luận

Trên tập con 10,000 mẫu, thuật toán IRLS hội tụ sau 6 vòng lặp.

Bảng 17: So sánh hiệu năng giữa OLS và Robust Regression

Mô hình	RMSE (scaled)	Đặc điểm
OLS Baseline	0.7485	Bị kéo lệch bởi các đơn hàng giao trễ đột biến
Robust (Huber)	0.6056	Tự động hạ trọng số của dữ liệu ngoại lai
Độ cải thiện	+19.10%	Khả năng định hiệu quả của hàm Huber

- **Chống nhiễu tốt:** RMSE giảm 19.10%, cho thấy mô hình bền vững hơn trước các đơn hàng bất thường.
- **Tính toán hiệu quả:** IRLS hội tụ nhanh nên vẫn phù hợp về chi phí xử lý.
- **Ý nghĩa thực tiễn:** Đây là lựa chọn phù hợp khi dữ liệu thực tế chứa ngoại lai nhưng hệ thống vẫn cần duy trì độ chính xác ổn định.

6.6 Phân tích sự đánh đổi Bias–Variance

6.6.1 Cơ sở toán học

Sai số dự báo bình phương trung bình có thể được phân rã thành ba thành phần:

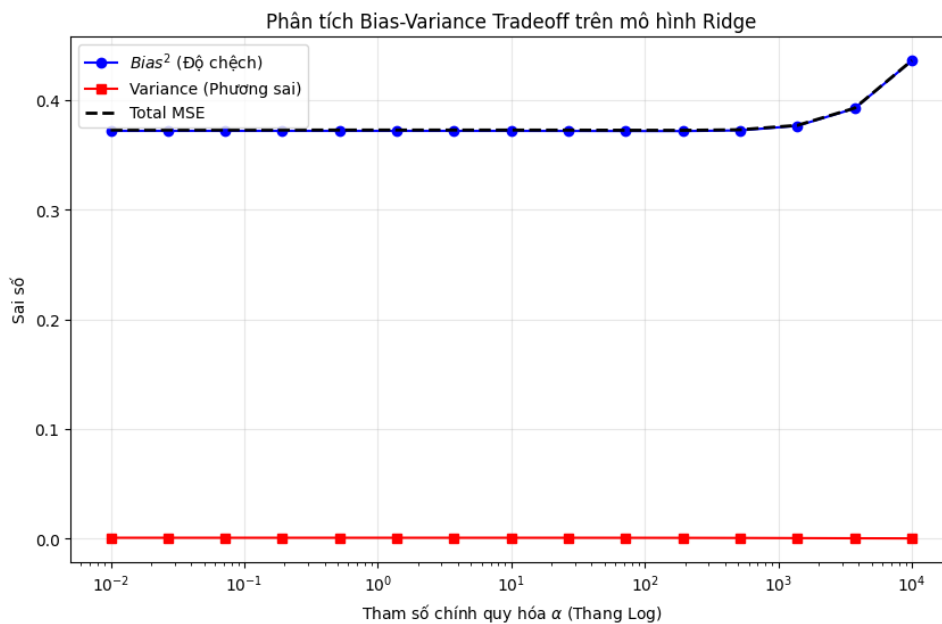
$$\text{MSE} = \mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

Trong đó, Bias^2 phản ánh mức độ đơn giản hóa của mô hình, Variance phản ánh độ nhạy với dữ liệu huấn luyện, còn σ^2 là sai số nội tại không thể loại bỏ. Với Ridge Regression, hệ số chính quy hóa α đóng vai trò điều khiển trực tiếp sự đánh đổi này: α nhỏ làm giảm Bias nhưng tăng Variance, còn α lớn có tác dụng ngược lại.

6.6.2 Phương pháp ước lượng

Nhóm sử dụng Bootstrap với $B = 200$ lần lấy mẫu lặp lại từ tập huấn luyện. Với mỗi giá trị α , mô hình Ridge được huấn luyện trên các tập Bootstrap, sau đó dự báo trên cùng một tập kiểm thử để ước lượng Bias^2 , Variance và tổng MSE.

6.6.3 Kết quả thực nghiệm



Hình 13: Sự đánh đổi giữa Bias, Variance và Total MSE theo hệ số chính quy hóa α

6.6.4 Nhận xét và kết luận

- **Vùng quá khớp:** Khi α nhỏ, Variance cao còn Bias thấp, cho thấy mô hình quá nhạy với dao động của dữ liệu huấn luyện.
- **Vùng chưa khớp:** Khi α quá lớn, Variance giảm mạnh nhưng Bias tăng nhanh do mô hình bị làm quá đơn giản.
- **Điểm cân bằng:** Vùng α làm Total MSE đạt cực tiểu chính là điểm cân bằng tối ưu giữa khả năng học và năng lực tổng quát hóa.

Phân tích Bootstrap cho thấy chính quy hóa không chỉ giúp giảm overfitting mà còn cung cấp một cơ sở thực nghiệm rõ ràng để chọn mức α phù hợp trước khi triển khai mô hình vào bài toán dự báo thực tế.

7 Đánh giá và phân tích

Phần này tổng hợp kết quả thực nghiệm của toàn bộ các mô hình đã khảo sát, đồng thời thực hiện so sánh ở mức định lượng và kiểm định thống kê nhằm xác định liệu sự khác biệt về hiệu năng có thực sự có ý nghĩa, thay vì chỉ dừng lại ở việc đối chiếu các giá trị RMSE đơn lẻ.

7.1 So sánh và Kiểm định Thống kê các Mô hình

Sau khi huấn luyện tất cả các mô hình, nhóm tiến hành đánh giá tổng thể để xác định mô hình nào vượt trội một cách *có ý nghĩa thống kê*. Quy trình phân tích gồm ba bước chính: thu thập kết quả Cross-Validation của từng mô hình, lập bảng tổng hợp để so sánh trực quan, và cuối cùng áp dụng các kiểm định thống kê phù hợp cho dữ liệu ghép cặp.

7.1.1 Thu thập kết quả K-Fold Cross-Validation

Ở bước đầu tiên, nhóm thu thập đầy đủ kết quả sai số từ 10-Fold Cross-Validation cho từng mô hình. Mỗi mô hình vì vậy không chỉ có một giá trị trung bình duy nhất, mà còn có một phân phối gồm 10 giá trị sai số tương ứng với 10 folds. Tập kết quả này là cơ sở để đánh giá mức độ ổn định của mô hình cũng như phục vụ cho các kiểm định thống kê ở bước sau.

Các chỉ số được lưu lại có thể bao gồm RMSE, MAE, MSE hoặc R^2 , tùy theo mục tiêu phân tích. Trong khuôn khổ so sánh tổng hợp, RMSE thường được ưu tiên vì vừa giữ nguyên đơn vị ngày giao hàng, vừa dễ diễn giải trong ngữ cảnh nghiệp vụ.

7.1.2 Bảng tổng hợp kết quả (Mean $\pm$ Std)

Từ phân phối sai số của từng mô hình qua 10 folds, nhóm tổng hợp lại dưới dạng trung bình cộng và độ lệch chuẩn, ký hiệu theo mẫu:

$$\text{Mean} \pm \text{Std}$$

Kiểu trình bày này cho phép so sánh trực quan giữa các mô hình ở hai khía cạnh:

- **Hiệu năng trung bình:** Mô hình nào cho sai số nhỏ hơn trên toàn bộ quá trình đánh giá.
- **Độ ổn định:** Mô hình nào có độ lệch chuẩn nhỏ hơn, tức ít dao động hơn giữa các folds.

Bảng tổng hợp đóng vai trò như một bước sàng lọc ban đầu trước khi tiến hành kiểm định giả thuyết. Trong nhiều trường hợp, hai mô hình có giá trị trung bình rất gần nhau, nhưng sự chênh lệch đó có thể chỉ là ngẫu nhiên nếu xét thêm độ phân tán của dữ liệu.

Bảng 18: Bảng tổng hợp kết quả Cross-Validation của các mô hình

Mô hình	MSE (mean)	MSE ($\pm$ std)	RMSE (mean)	RMSE ($\pm$ std)
OLS (Normal Eq)	0.6262	± 0.0138	0.7913	± 0.0087
Ridge	0.6262	± 0.0138	0.7913	± 0.0087
Lasso	0.6262	± 0.0138	0.7913	± 0.0087
Bayesian	0.6262	± 0.0138	0.7913	± 0.0087
Elastic Net	0.6263	± 0.0138	0.7913	± 0.0087
WLS	0.6285	± 0.0139	0.7928	± 0.0087

Nhận xét.

- **Không có mô hình nào vượt trội rõ rệt:** Tất cả các cấu hình đều nằm trong khoảng MSE từ 0.6262 đến 0.6285, với độ lệch chuẩn gần như giống nhau, quanh mức ± 0.014 .
- **OLS, Ridge, Lasso và Bayesian gần như trùng khớp:** Điều này cho thấy regularization ở cấu hình hiện tại chưa tạo ra khác biệt đáng kể, đồng thời xác nhận nguy cơ overfitting là thấp trong không gian đặc trưng này.
- **WLS kém hơn nhẹ trong Cross-Validation:** Nguyên nhân hợp lý là trọng số của WLS được ước lượng lại theo từng fold nên kém ổn định hơn khi dữ liệu huấn luyện bị thu nhỏ.
- **Độ ổn định tổng thể tốt:** Bảng tổng hợp củng cố nhận định rằng họ mô hình tuyến tính đã tiệm cận giới hạn dự báo của bộ đặc trưng hiện tại, nhưng vẫn duy trì khả năng tổng quát hóa khá ổn định trên toàn bộ 10 folds.

7.1.3 Kiểm định thống kê: Paired T-test và Wilcoxon

Để xác nhận liệu sự khác biệt giữa hai mô hình có thực sự đáng kể hay không, nhóm sử dụng các kiểm định thống kê dành cho dữ liệu ghép cặp theo folds:

- **Paired T-test:** Phù hợp khi phân phối chênh lệch sai số giữa hai mô hình gần chuẩn. Kiểm định này đánh giá xem trung bình chênh lệch có khác 0 một cách có ý nghĩa hay không.
- **Wilcoxon Signed-Rank Test:** Là phương án phi tham số, phù hợp khi không muốn giả định tính chuẩn của dữ liệu hoặc khi số folds còn hạn chế.

Giả thuyết kiểm định được phát biểu theo dạng:

- H_0 : Không có khác biệt có ý nghĩa thống kê giữa hai mô hình.
- H_1 : Có khác biệt có ý nghĩa thống kê giữa hai mô hình.

Nếu giá trị p -value nhỏ hơn ngưỡng ý nghĩa lựa chọn, chẳng hạn 0.05, nhóm bác bỏ H_0 và kết luận rằng mô hình tốt hơn không chỉ vượt trội về mặt số học mà còn vượt trội một cách có cơ sở thống kê. Ngược lại, nếu p -value lớn, sự khác biệt quan sát được nhiều khả năng chỉ là dao động ngẫu nhiên từ quá trình chia folds.

Bảng 19: Kết quả kiểm định thống kê giữa các cặp mô hình

Cặp so sánh	Mean A	Mean B	T-statistic	p-value	Có ý nghĩa?	Mô hình tốt hơn
OLS (Normal Eq) vs WLS	0.6262	0.6285	-6.4263	0.0001	Có ($p < 0.05$)	OLS (Normal Eq)
OLS (Normal Eq) vs Ridge	0.6262	0.6262	0.2133	0.8359	Không	Tương đương
OLS (Normal Eq) vs Lasso	0.6262	0.6262	0.0177	0.9863	Không	Tương đương
OLS (Normal Eq) vs Bayesian	0.6262	0.6262	0.1743	0.8655	Không	Tương đương
OLS (Normal Eq) vs Elastic Net	0.6262	0.6263	-0.8499	0.4174	Không	Tương đương

8 Tổng kết và Hướng phát triển

8.1 Tổng kết chính

Từ toàn bộ thực nghiệm, nhóm rút ra ba kết luận chính sau:

- **Dữ liệu logistics có tính phi tuyến và nhiễu cao:** Thời gian giao hàng không chỉ phụ thuộc vào các biến quan sát được như khoảng cách hay giá trị đơn hàng, mà còn chịu ảnh hưởng của nhiều yếu tố ẩn như thời tiết, hạ tầng giao thông, quy trình trung chuyển và sự cố vận hành. Vì vậy, luôn tồn tại một phần sai số không thể loại bỏ hoàn toàn.
- **Mô hình đã khai thác gần hết tín hiệu hữu ích:** Kết quả thực nghiệm cho thấy khi chuyển từ các mô hình tuyến tính sang các mô hình phi tuyến mạnh hơn, mức cải thiện là không đáng kể. Điều này cho thấy hiệu năng hiện tại phản ánh chủ yếu giới hạn của dữ liệu hơn là giới hạn của thuật toán.
- **Một số mô hình có giá trị thực tiễn cao:** Robust Regression giúp giảm ảnh hưởng của các đơn hàng bất thường, còn Bayesian Regression cung cấp thêm thông tin về độ bất định của dự báo. Đây là hai hướng tiếp cận phù hợp với các bài toán logistics thực tế.

8.2 Ý nghĩa của kết quả

Mặc dù $R^2 \approx 0.38$ và RMSE còn khoảng 4.27 ngày, kết quả này không có nghĩa là mô hình hoạt động kém. Ngược lại, nó cho thấy bài toán dự báo thời gian giao hàng trên dữ liệu Olist vốn rất khó do dữ liệu nhiễu lớn và chứa nhiều yếu tố ngoài quan sát. Nói cách khác, giới hạn hiện tại đến nhiều hơn từ chất lượng và mức độ đầy đủ của dữ liệu hơn là từ bản thân mô hình.

8.3 Hướng phát triển

Trong giai đoạn tiếp theo, nhóm đề xuất ba hướng phát triển chính:

- **Bổ sung đặc trưng mới:** thêm các biến phản ánh tốt hơn quá trình vận hành giao hàng, chẳng hạn thông tin về tuyến vận chuyển, thời tiết hoặc trạng thái kho bãi.

- **Tối ưu thêm mô hình phi tuyến:** tiếp tục thử nghiệm và tinh chỉnh các mô hình mạnh hơn nếu có thêm đặc trưng đầu vào giàu thông tin.
- **Tăng cường lượng hóa bất định:** ưu tiên các mô hình có thể dự báo theo khoảng thay vì chỉ dự báo điểm, nhằm hỗ trợ quản trị rủi ro tốt hơn trong thực tế.

Phần II

BÀI TOÁN PHÂN LỚP

1 Giới thiệu và Tổng quan

1.1 Bộ dữ liệu

Bộ dữ liệu được sử dụng trong phần này là **Airline Passenger Satisfaction** được lấy từ Kaggle ([teejmahal20/airline-passenger-satisfaction](https://www.kaggle.com/teejmahal20/airline-passenger-satisfaction)). Đây là bộ dữ liệu thực tế mô tả trải nghiệm hành khách trên các chuyến bay thương mại, bao gồm thông tin cá nhân, thông tin chuyến bay và các đánh giá dịch vụ.

Bảng 20: Thông tin tổng quan về bộ dữ liệu

Thuộc tính	Giá trị
Nguồn dữ liệu	Kaggle – teejmahal20/airline-passenger-satisfaction
Tổng số mẫu	129.880 mẫu
Tập huấn luyện	103.904 mẫu
Tập kiểm tra	25.976 mẫu
Số lượng biến	25 biến (23 đặc trưng + 1 nhãn + 1 chỉ số)
Loại bài toán	Phân lớp nhị phân
Biến mục tiêu	satisfaction (satisfied / neutral or dissatisfied)

1.2 Mục tiêu

Mục tiêu của phần này là xây dựng và so sánh các mô hình phân lớp tuyến tính để dự đoán mức độ hài lòng của hành khách, bao gồm: Logistic Regression, Linear Discriminant Analysis (LDA/QDA) và Perceptron. Các mô hình được đánh giá dựa trên nhiều chỉ số hiệu năng và kết quả được phân tích có hệ thống.

2 Cơ sở Lý thuyết

2.1 Bài toán Phân lớp

Cho tập dữ liệu huấn luyện $\mathcal{D} = \{(\mathbf{x}_n, c_n)\}_{n=1}^N$ trong đó $\mathbf{x}_n \in \mathbb{R}^D$ là vector đặc trưng và $c_n \in \{1, 2, \dots, K\}$ là nhãn lớp. Mục tiêu là xây dựng hàm phân tách $f: \mathbb{R}^D \rightarrow \{1, \dots, K\}$ sao cho tỷ lệ phân lớp sai trên dữ liệu mới là nhỏ nhất.

2.2 Hàm phân quyết tuyến tính

Với bài toán phân lớp nhị phân ($K = 2$), hàm phân quyết tuyến tính có dạng:

$$y(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 \quad (1)$$

Mặt phẳng quyết định (decision surface) là tập $\{\mathbf{x} : y(\mathbf{x}) = 0\}$, tức một siêu phẳng trong $\mathbb{R}^D$.

2.3 Hồi quy Logistic (Logistic Regression)

Logistic Regression là mô hình phân lớp xác suất phân biệt (discriminative). Với phân lớp nhị phân:

$$p(\mathcal{C}_1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + w_0) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + w_0)}} \quad (2)$$

trong đó $\sigma(\cdot)$ là hàm sigmoid.

2.3.1 Hàm mất mát Cross-Entropy

Hàm mất mát được tối thiểu hóa trong Logistic Regression là Cross-Entropy:

$$E(\mathbf{W}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln p(\mathcal{C}_k | \mathbf{x}_n) \quad (3)$$

Gradient đối với $\mathbf{w}_k$ là:

$$\nabla_{\mathbf{w}_k} E = \sum_{n=1}^N (y_{nk} - t_{nk}) \mathbf{x}_n, \quad y_{nk} = p(\mathcal{C}_k | \mathbf{x}_n) \quad (4)$$

2.3.2 Gradient Descent cho Logistic Regression

Algorithm 1: Gradient Descent cho Logistic Regression

Input: Dữ liệu $\{(\mathbf{x}_n, t_n)\}$, learning rate η , số epoch tối đa

Khởi tạo $\mathbf{w} = \mathbf{0}$ hoặc ngẫu nhiên nhỏ;

repeat

 Tính $\hat{y}_n = \sigma(\mathbf{w}^\top \mathbf{x}_n)$ với mọi n ;

 Tính gradient: $\nabla E = \sum_n (\hat{y}_n - t_n) \mathbf{x}_n$;

 Cập nhật: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E$;

until hội tụ;

Output: $\mathbf{w}$

2.3.3 Thuật toán Newton–Raphson / IRLS

Logistic Regression không có nghiệm dạng đóng, nhưng hàm mất mát Cross-Entropy là lồi và ma trận Hessian xác định dương. Thuật toán Newton–Raphson (hay IRLS – Iterative Reweighted Least Squares) sử dụng thông tin bậc hai:

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \mathbf{H}^{-1} \nabla E \quad (5)$$

trong đó ma trận Hessian là:

$$\mathbf{H} = \mathbf{\Phi}^\top \mathbf{R} \mathbf{\Phi} + \lambda \mathbf{I}, \quad \mathbf{R} = \text{diag}(y_{n1}(1 - y_{n1}), \dots, y_{nN}(1 - y_{nN})) \quad (6)$$

IRLS hội tụ trong $\mathcal{O}(\log(1/\varepsilon))$ bước, nhanh hơn Gradient Descent cần $\mathcal{O}(1/\varepsilon)$ bước, nhưng chi phí mỗi bước là $\mathcal{O}(NM^2 + M^3)$ do tính nghịch đảo Hessian.

2.4 Phân tích phân tách tuyến tính Fisher (Fisher's LDA)

Fisher's LDA tìm hướng chiếu $\mathbf{w}$ sao cho hàm năng lượng

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}} \quad (7)$$

được tối đa hóa, trong đó:

$$\mathbf{S}_B = (\mathbf{m}_2 - \mathbf{m}_1)(\mathbf{m}_2 - \mathbf{m}_1)^\top \quad (8)$$

$$\mathbf{S}_W = \sum_{n \in \mathcal{C}_1} (\mathbf{x}_n - \mathbf{m}_1)(\mathbf{x}_n - \mathbf{m}_1)^\top + \sum_{n \in \mathcal{C}_2} (\mathbf{x}_n - \mathbf{m}_2)(\mathbf{x}_n - \mathbf{m}_2)^\top \quad (9)$$

Nghiệm tối ưu: $\mathbf{w} \propto \mathbf{S}_W^{-1}(\mathbf{m}_2 - \mathbf{m}_1)$.

2.5 Mô hình sinh Gaussian (LDA tổng quát)

LDA (Generative) giả thiết mỗi lớp tuân theo phân phối Gaussian với cùng ma trận hiệp phương sai:

$$p(\mathbf{x} \mid \mathcal{C}_k) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}) \quad (10)$$

Ước lượng tham số bằng MLE:

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{N_k} \sum_{n \in \mathcal{C}_k} \mathbf{x}_n, \quad \hat{\boldsymbol{\Sigma}} = \sum_{k=1}^K \frac{N_k}{N} \mathbf{S}_k \quad (11)$$

QDA mở rộng LDA bằng cách cho phép mỗi lớp có ma trận hiệp phương sai riêng $\boldsymbol{\Sigma}_k$.

2.6 Thuật toán Perceptron

Định nghĩa 2.1 (Perceptron). *Perceptron (Rosenblatt, 1958) cập nhật trọng số theo quy tắc:*

$$\mathbf{w}^{new} = \mathbf{w}^{old} - \eta \nabla E_P(\mathbf{w}) \quad (12)$$

với hàm mất mát:

$$E_P(\mathbf{w}) = - \sum_{n \in \mathcal{M}} \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}_n) t_n \quad (13)$$

trong đó $\mathcal{M}$ là tập các mẫu bị phân lớp sai.

Định lý 2.1 (Hội tụ Perceptron). *Nếu dữ liệu có thể phân tách tuyến tính, thuật toán Perceptron hội tụ trong hữu hạn bước.*

2.7 Regularization cho Logistic Regression

Thêm số hạng phạt vào hàm mất mát:

$$E_{L2}(\mathbf{w}) = - \sum_n \sum_k t_{nk} \ln y_{nk} + \frac{\lambda}{2} \|\mathbf{w}\|^2 \quad (\text{Ridge / L2}) \quad (14)$$

$$E_{L1}(\mathbf{w}) = - \sum_n \sum_k t_{nk} \ln y_{nk} + \lambda \|\mathbf{w}\|_1 \quad (\text{Lasso / L1}) \quad (15)$$

L1 khuyến khích nghiệm thưa (sparse), trong khi L2 giảm độ lớn đồng đều các hệ số.

2.8 Class-weighted Loss

Để xử lý mất cân bằng lớp, hàm mất mát được điều chỉnh:

$$E = - \sum_{n,k} c_k t_{nk} \ln y_{nk}, \quad c_k \propto \frac{N}{N_k} \quad (16)$$

trong đó c_k là trọng số của lớp k , N_k là số mẫu thuộc lớp k .

3 Mô tả Dữ liệu và Phân tích Khám phá (EDA)

3.1 Mô tả Dữ liệu

3.1.1 Ý nghĩa các cột dữ liệu

Bảng 21: Định nghĩa các cột trong bộ dữ liệu Airline Passenger Satisfaction

Tên cột	Định nghĩa
Gender	Giới tính hành khách (Male/Female)
Customer Type	Loại khách hàng (Loyal/Disloyal)
Age	Độ tuổi của hành khách
Type of Travel	Mục đích chuyến đi (Business/Personal)
Class	Hạng ghế (Business/Eco/Eco Plus)
Flight Distance	Khoảng cách chuyến bay (dặm)
Inflight wifi service	Đánh giá dịch vụ wifi (0–5)
Departure/Arrival time convenient	Sự thuận tiện giờ bay (0–5)
Ease of Online booking	Dễ dàng đặt vé trực tuyến (0–5)
Gate location	Vị trí cửa khởi hành (0–5)
Food and drink	Đánh giá đồ ăn và nước uống (0–5)
Online boarding	Đánh giá quá trình boarding online (0–5)
Seat comfort	Độ thoải mái ghế ngồi (0–5)
Inflight entertainment	Giải trí trên chuyến bay (0–5)
On-board service	Dịch vụ trên máy bay (0–5)
Leg room service	Không gian để chân (0–5)
Baggage handling	Xử lý hành lý (0–5)
Checkin service	Dịch vụ check-in (0–5)
Inflight service	Dịch vụ trong chuyến bay (0–5)
Cleanliness	Vệ sinh máy bay (0–5)
Departure Delay in Minutes	Thời gian delay khởi hành (phút)
Arrival Delay in Minutes	Thời gian delay đến nơi (phút)
satisfaction	Mức độ hài lòng (satisfied / neutral or dissatisfied)

3.1.2 Thống kê mô tả

Mô tả dữ liệu train:									
	count	mean	std	min	25%	50%	75%	max	
Age	103904.0	39.380	15.115	7.0	27.0	40.0	51.0	85.0	
Flight Distance	103904.0	1189.448	997.147	31.0	414.0	843.0	1743.0	4983.0	
Inflight wifi service	103904.0	2.730	1.328	0.0	2.0	3.0	4.0	5.0	
Departure/Arrival time convenient	103904.0	3.060	1.525	0.0	2.0	3.0	4.0	5.0	
Ease of Online booking	103904.0	2.757	1.399	0.0	2.0	3.0	4.0	5.0	
Gate location	103904.0	2.977	1.278	0.0	2.0	3.0	4.0	5.0	
Food and drink	103904.0	3.202	1.330	0.0	2.0	3.0	4.0	5.0	
Online boarding	103904.0	3.250	1.350	0.0	2.0	3.0	4.0	5.0	
Seat comfort	103904.0	3.439	1.319	0.0	2.0	4.0	5.0	5.0	
Inflight entertainment	103904.0	3.358	1.333	0.0	2.0	4.0	4.0	5.0	
On-board service	103904.0	3.382	1.288	0.0	2.0	4.0	4.0	5.0	
Leg room service	103904.0	3.351	1.316	0.0	2.0	4.0	4.0	5.0	
Baggage handling	103904.0	3.632	1.181	1.0	3.0	4.0	5.0	5.0	
Checkin service	103904.0	3.304	1.265	0.0	3.0	3.0	4.0	5.0	
Inflight service	103904.0	3.640	1.176	0.0	3.0	4.0	5.0	5.0	
Cleanliness	103904.0	3.286	1.312	0.0	2.0	3.0	4.0	5.0	
Departure Delay in Minutes	103904.0	14.816	38.231	0.0	0.0	0.0	12.0	1592.0	
Arrival Delay in Minutes	103594.0	15.179	38.699	0.0	0.0	0.0	13.0	1584.0	

Mô tả dữ liệu test:									
	count	mean	std	min	25%	50%	75%	max	
Age	25976.0	39.621	15.136	7.0	27.0	40.0	51.0	85.0	
Flight Distance	25976.0	1193.788	998.684	31.0	414.0	849.0	1744.0	4983.0	
Inflight wifi service	25976.0	2.725	1.335	0.0	2.0	3.0	4.0	5.0	
Departure/Arrival time convenient	25976.0	3.047	1.533	0.0	2.0	3.0	4.0	5.0	
Ease of Online booking	25976.0	2.757	1.413	0.0	2.0	3.0	4.0	5.0	
Gate location	25976.0	2.977	1.282	1.0	2.0	3.0	4.0	5.0	
Food and drink	25976.0	3.215	1.332	0.0	2.0	3.0	4.0	5.0	
Online boarding	25976.0	3.262	1.356	0.0	2.0	4.0	4.0	5.0	
Seat comfort	25976.0	3.449	1.320	1.0	2.0	4.0	5.0	5.0	
Inflight entertainment	25976.0	3.358	1.338	0.0	2.0	4.0	4.0	5.0	
On-board service	25976.0	3.386	1.282	0.0	2.0	4.0	4.0	5.0	
Leg room service	25976.0	3.350	1.319	0.0	2.0	4.0	4.0	5.0	
Baggage handling	25976.0	3.633	1.177	1.0	3.0	4.0	5.0	5.0	
Checkin service	25976.0	3.314	1.269	1.0	3.0	3.0	4.0	5.0	
Inflight service	25976.0	3.649	1.181	0.0	3.0	4.0	5.0	5.0	
Cleanliness	25976.0	3.286	1.319	0.0	2.0	3.0	4.0	5.0	
Departure Delay in Minutes	25976.0	14.306	37.423	0.0	0.0	0.0	12.0	1128.0	
Arrival Delay in Minutes	25893.0	14.741	37.518	0.0	0.0	0.0	13.0	1115.0	

Hình 14: Mô tả dữ liệu trên tập train và tập test

Tập dữ liệu bao gồm 103.904 mẫu huấn luyện và 25.976 mẫu kiểm tra với 23 đặc trưng (sau khi loại bỏ cột chỉ số `Unnamed: 0` và `id`). Phần lớn các biến là kiểu số nguyên (`int64`) hoặc số thực (`float64`), đặc biệt là các biến đánh giá dịch vụ trên thang điểm 0–5. Có 4 biến phân loại dạng chuỗi: `Gender`, `Customer Type`, `Type of Travel`, `Class`.

Nhận xét:

- Phân phối của các biến trong tập train và test khá tương đồng (mean, median, std gần nhau), cho thấy tính nhất quán cao giữa hai tập.
- Các biến đánh giá dịch vụ có giá trị trung bình nằm trong khoảng 3–4, phản ánh mức hài lòng trung bình đến khá.

- Các biến delay (Departure Delay, Arrival Delay) có median bằng 0 nhưng giá trị tối đa rất lớn, cho thấy phân phối lệch phải mạnh và tồn tại nhiều ngoại lai.

3.2 Kiểm tra Chất lượng Dữ liệu

3.2.1 Giá trị trùng lặp và hàng rỗng

```

DÒNG TRÙNG LẶP
Số dòng trùng lặp trong train: 0
Số dòng trùng lặp trong test: 0

DÒNG RỖNG HOÀN TOÀN
Số dòng dữ liệu rỗng trong train: 0
Số dòng dữ liệu rỗng trong test: 0

```

Hình 15: Kết quả kiểm tra trùng lặp dữ liệu

Kết quả kiểm tra cho thấy tập train và test **không chứa dòng trùng lặp hay dòng rỗng hoàn toàn**, đảm bảo tính sạch của dữ liệu đầu vào.

3.2.2 Giá trị thiếu (Missing Values)

```

Những giá trị bị thiếu trong tập train:

```

	count	pct
Arrival Delay in Minutes	310	0.298

```

Những giá trị bị thiếu trong tập test:

```

	count	pct
Arrival Delay in Minutes	83	0.32

Hình 16: Kết quả kiểm tra thiếu dữ liệu

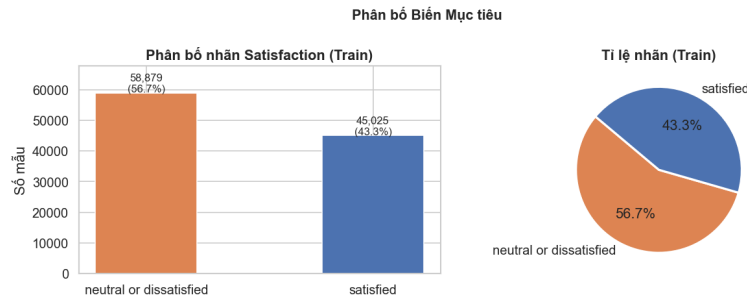
Duy nhất biến `Arrival Delay in Minutes` có giá trị thiếu. Vì biến này có phân phối lệch phải và chứa nhiều ngoại lai, phương pháp **điền giá trị trung vị (median)** được lựa chọn để xử lý, tránh bị ảnh hưởng bởi các giá trị cực đoan. Tham số median được tính *chỉ trên tập train* và áp dụng cho cả validation và test nhằm tránh rò rỉ thông tin (data leakage).

3.3 Phân tích Khám phá (EDA)

3.3.1 Phân bố biến mục tiêu

Nhận xét:

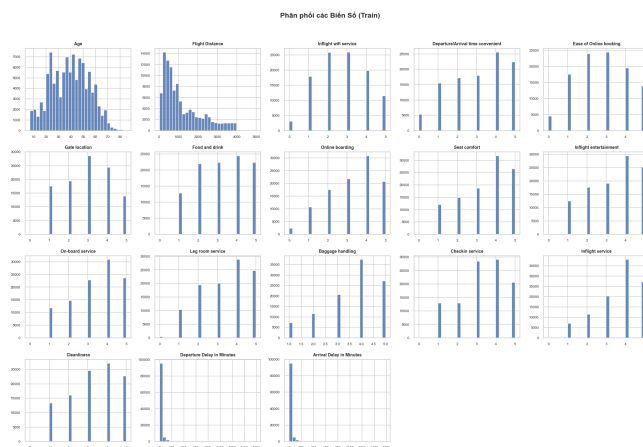
- Lớp *neutral or dissatisfied*: $\approx 56.7\%$; lớp *satisfied*: $\approx 43.3\%$.



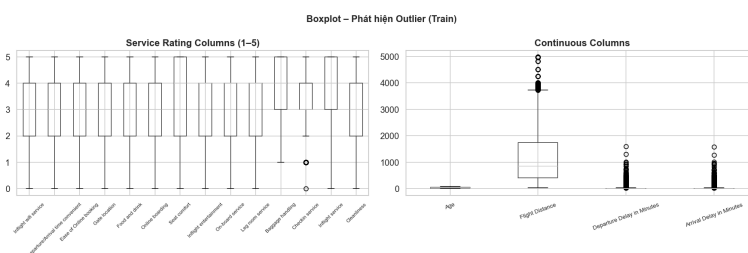
Hình 17: Biểu đồ phân bố biến mục tiêu

- Tỉ lệ mất cân bằng $\approx 1.31 : 1$ – mất cân bằng nhẹ, không cần kỹ thuật oversampling (SMOTE). Tuy nhiên, nên sử dụng `class_weight='balanced'` khi huấn luyện để tránh thiên lệch về phía lớp đa số.

3.3.2 Phân bố biến số – Histogram và Boxplot



Hình 18: Biểu đồ phân bố biến số

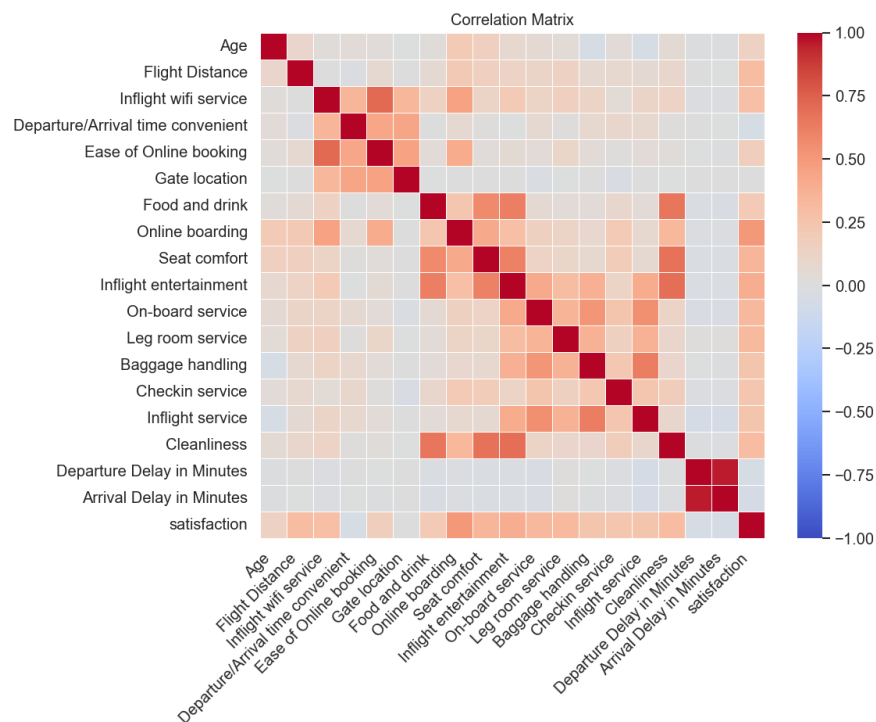


Hình 19: Boxplot - Phát hiện outliner

Nhận xét:

- Các biến đánh giá dịch vụ (thang 1–5) có phân bố khá đồng đều, tập trung quanh mức 3–4, ít biến động.
- Biến `Flight Distance`, `Departure Delay`, `Arrival Delay` có phân bố lệch phải rõ rệt với nhiều ngoại lai lớn $\Rightarrow$ cần xử lý log-transform.
- Tỷ lệ ngoại lai theo IQR: `Departure Delay` $\approx$ 13.98%, `Arrival Delay` $\approx$ 13.43%, `Checkin service` $\approx$ 12.41%, `Flight Distance` $\approx$ 2.20%.

3.3.3 Ma trận Tương quan



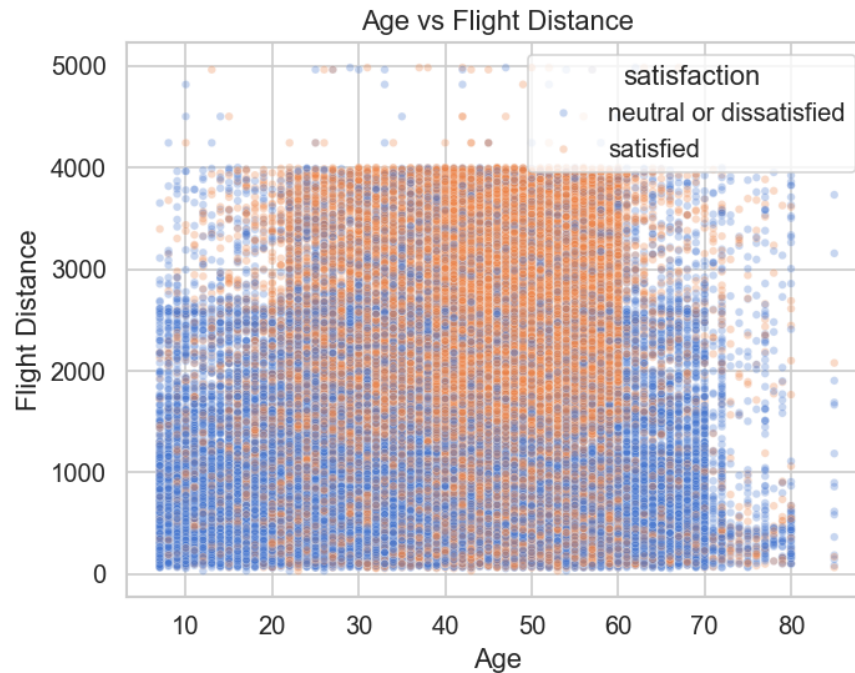
Hình 20: Biểu đồ nhiệt ma trận tương quan

Nhận xét: Các biến có tương quan dương mạnh nhất với `satisfaction`:

- `Online boarding` (0.5036) – cao nhất trong tất cả các đặc trưng.
- `Inflight entertainment` (0.3981), `Seat comfort` (0.3495), `On-board service` (0.3224), `Leg room service` (0.3131), `Cleanliness` (0.3052).

Các biến delay có tương quan âm nhẹ với `satisfaction`. Một số cặp biến có tương quan cao với nhau: `Departure Delay` – `Arrival Delay`; `Inflight wifi` – `Ease of Online booking`; `Cleanliness` – `Inflight entertainment`.

3.3.4 Scatter Plot – Age vs. Flight Distance



Hình 21: Biểu đồ Scatter so sánh Age vs Flight Distance

Nhận xét: Hai biến Age và Flight Distance riêng lẻ không phân tách tốt, hai lớp do phân bố chồng lấp nhiều => Cần kết hợp thêm các biến dịch vụ.

4 Tiền xử lý Dữ liệu

4.1 Mã hóa Biến Phân loại

Các biến phân loại được mã hóa theo bảng sau:

Bảng 22: Chiến lược mã hóa các biến phân loại

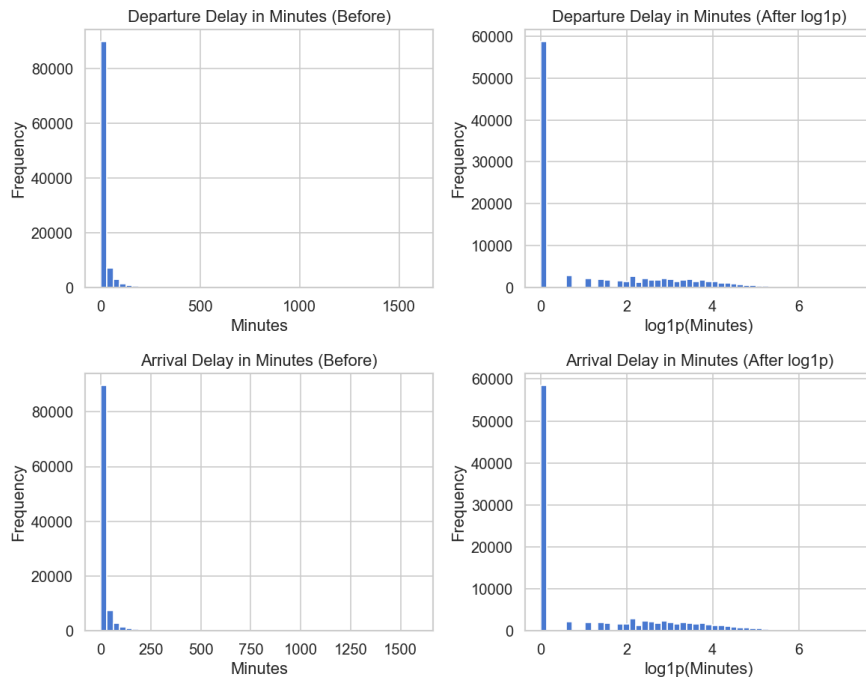
Biến	Phương pháp	Ánh xạ
Gender	Binary encoding	Male = 0, Female = 1
Customer Type	Binary encoding	Disloyal = 0, Loyal = 1
Type of Travel	Binary encoding	Personal = 0, Business = 1
Class	Ordinal encoding	Eco = 0, Eco Plus = 1, Business = 2
satisfaction	Binary (nhãn)	neutral or dissatisfied = 0, satisfied = 1

4.2 Xử lý Ngoại lai và Log-Transform

Do biến Departure Delay in Minutes và Arrival Delay in Minutes có phân phối lệch phải mạnh (skewness > 6), biến đổi $\log(1 + x)$ được áp dụng:

$$x' = \log_e(1 + x) \quad (17)$$

Sau biến đổi, skewness giảm mạnh xuống < 1 , phân bố gần chuẩn hơn và giảm ảnh hưởng của ngoại lai đối với các mô hình tuyến tính.



Hình 22: Phân phối dữ liệu trước và sau log-transform

4.3 Chia Dữ liệu

Tập `train.csv` được chia theo tỉ lệ 87.5%/12.5% (stratified) để tạo tập train và validation. Kết hợp với `test.csv` có sẵn, ta thu được:

Bảng 23: Phân chia tập dữ liệu

Tập	Số mẫu	Tỉ lệ	Nguồn
Train	≈ 90.916	$\approx 70\%$	train.csv (87.5%)
Validation	≈ 12.988	$\approx 10\%$	train.csv (12.5%)
Test	25.976	$\approx 20\%$	test.csv

Tham số `stratify=y` đảm bảo tỉ lệ lớp nhất quán giữa các tập.

4.4 Chuẩn hóa Dữ liệu

`StandardScaler` được sử dụng để đưa các biến số về cùng thang đo ($\mu = 0, \sigma = 1$). Scaler chỉ được `fit` trên tập train và sau đó `transform` lên validation/test để tránh data leakage. Chuẩn hóa đặc biệt quan trọng cho Logistic Regression và LDA vì các thuật toán này nhạy với thang đo của đặc trưng.

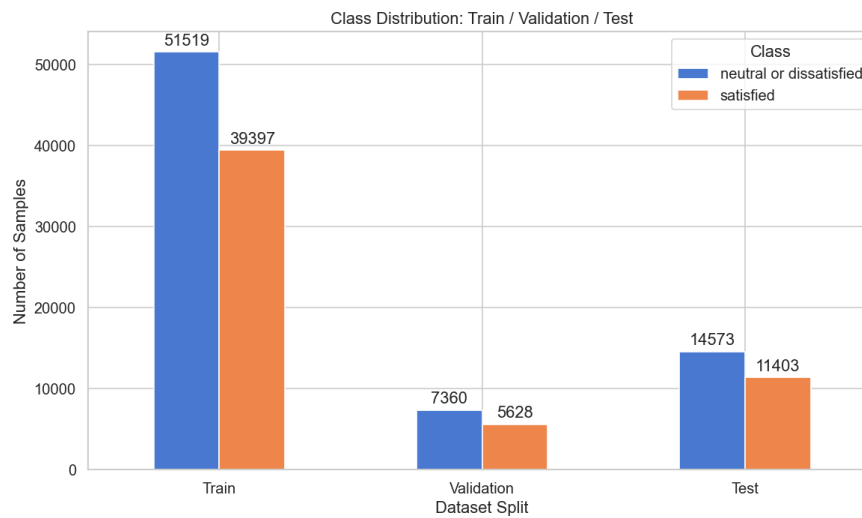
các cột được scale:

['Age', 'Flight Distance', 'Inflight wifi service', 'Departure/Arrival time convenient', 'Ease of Online booking', 'Gate location', 'Food and drink', 'Online boarding', 'Seat comfort', 'Inflight entertainment', 'On-board service', 'Leg room service', 'Baggage handling', 'Checkin service', 'Inflight service', 'Cleanliness', 'Departure Delay in Minutes', 'Arrival Delay in Minutes']

	mean (=0)	std (=1)
Age	0.0	1.0
Flight Distance	0.0	1.0
Inflight wifi service	-0.0	1.0
Departure/Arrival time convenient	-0.0	1.0
Ease of Online booking	-0.0	1.0
Gate location	0.0	1.0
Food and drink	-0.0	1.0
Online boarding	-0.0	1.0
Seat comfort	0.0	1.0
Inflight entertainment	-0.0	1.0
On-board service	0.0	1.0
Leg room service	-0.0	1.0
Baggage handling	-0.0	1.0
Checkin service	0.0	1.0
Inflight service	0.0	1.0
Cleanliness	0.0	1.0
Departure Delay in Minutes	-0.0	1.0
Arrival Delay in Minutes	0.0	1.0

Hình 23: Các cột đã được Scale

4.5 Phân tích Mất cân bằng Lớp



Hình 24: Kiểm tra mất cân bằng lớp

Sau khi chia dữ liệu stratified, tỉ lệ lớp được bảo toàn nhất quán trên cả ba tập (train/val/test). Tỉ lệ mất cân bằng $\approx 1.31 : 1$ được xem là nhẹ. Trọng số lớp được tính sẵn để truyền vào hàm mất mát khi huấn luyện:

$$c_k = \frac{N}{K \cdot N_k} \quad (18)$$

5 Xây dựng và Huấn luyện Mô hình

5.1 Logistic Regression

5.1.1 Động cơ mô hình

Logistic Regression là mô hình phân lớp xác suất thuộc nhóm *probabilistic discriminative models*, tức mô hình hóa trực tiếp xác suất hậu nghiệm $p(y | \mathbf{x})$ thay vì mô hình hóa phân phối sinh $p(\mathbf{x} | y)$. Đây là một lựa chọn nền tảng cho bài toán phân lớp nhị phân nhờ ba ưu điểm chính: (i) mô hình có diễn giải rõ ràng thông qua xác suất đầu ra, (ii)

hàm mất mát là lỗi nên việc tối ưu ổn định, và (iii) có thể mở rộng tự nhiên sang nhiều lớp bằng softmax hoặc các chiến lược phân rã nhị phân.

Trong phần thực nghiệm này, nhóm sử dụng Logistic Regression như mô hình chuẩn để so sánh với các phương pháp sinh xác suất như LDA/QDA và với Perceptron ở các phần sau. Toàn bộ quá trình huấn luyện được cài đặt từ đầu bằng `numpy`; `scikit-learn` chỉ được dùng ở bước kiểm chứng kết quả cuối cùng.

5.1.2 Biểu diễn toán học cho bài toán nhị phân

Với một mẫu đầu vào $\mathbf{x} \in \mathbb{R}^D$, nhóm bổ sung thêm một chiều hằng số để học hệ số chặn:

$$\tilde{\mathbf{x}} = [1, x_1, x_2, \dots, x_D]^\top. \quad (19)$$

Khi đó điểm số tuyến tính của mô hình được viết:

$$z = \mathbf{w}^\top \tilde{\mathbf{x}}. \quad (20)$$

Xác suất mẫu thuộc lớp dương được xác định bởi hàm sigmoid:

$$p(y = 1 | \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (21)$$

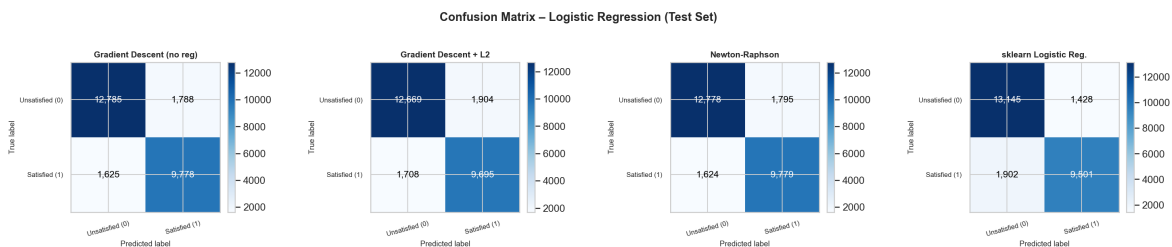
Trong thực nghiệm, để tránh hiện tượng tràn số khi z có độ lớn quá lớn, giá trị đầu vào của hàm mũ được giới hạn trong một khoảng hữu hạn trước khi tính sigmoid.

Quy tắc dự đoán nhãn của mô hình là:

$$\hat{y} = \begin{cases} 1, & p(y = 1 | \mathbf{x}) \geq 0.5, \\ 0, & \text{ngược lại.} \end{cases} \quad (22)$$

5.1.3 Ma trận nhầm lẫn (Confusion Matrix) – Logistic Regression (Test Set)

Trong phần này, hiệu năng của các mô hình Logistic Regression được đánh giá chi tiết thông qua ma trận nhầm lẫn trên tập kiểm tra (Test set). Kết quả cụ thể được trình bày tại Hình 25.



Hình 25: Ma trận nhầm lẫn của các biến thể Logistic Regression trên tập Test

Nhận xét và đánh giá

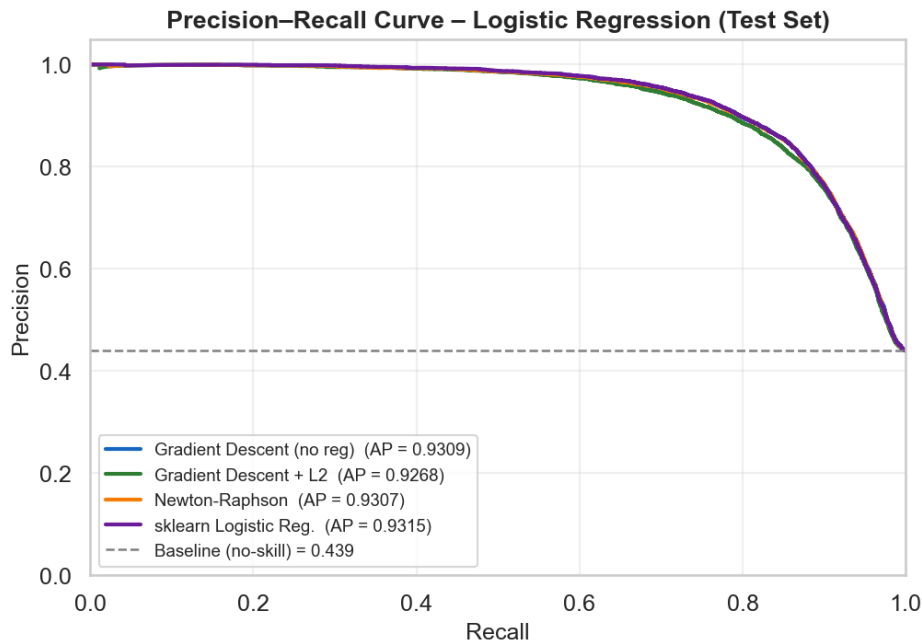
- **Sự tương đồng giữa hai thuật toán tự cài đặt:** Mô hình Logistic Regression được huấn luyện bằng Gradient Descent (không regularization) và thuật toán Newton-Raphson cho ma trận nhầm lẫn gần như tương đồng hoàn toàn, với sự

sai lệch chỉ xuất hiện ở vài mẫu đơn lẻ. Phân bố lỗi được ghi nhận khá cân bằng giữa kết quả dương tính giả (False Positive) ≈ 1790 mẫu và âm tính giả (False Negative) ≈ 1625 mẫu. Điều này minh chứng rằng cả hai phương pháp tối ưu (dù sử dụng đạo hàm bậc nhất hay bậc hai) đều đã hội tụ chính xác về cùng một điểm cực tiểu toàn cục.

- **Tác động của thành phần điều chuẩn (L2 Regularization):** Trái với kỳ vọng ban đầu về việc chống quá khớp (Overfitting), mô hình Gradient Descent tích hợp L2 ghi nhận sự sụt giảm nhẹ về độ chính xác. Cụ thể, cả lượng dự đoán sai FP và FN đều gia tăng. Kết quả này củng cố luận điểm rằng các trọng số hội tụ tự nhiên trên bộ dữ liệu Airline vốn đã ổn định; việc áp dụng lực phạt L2 vô tình ép trọng số xuống mức quá thấp và làm mờ đi ranh giới quyết định (Decision Boundary).
- **Sự khác biệt của thư viện sklearn:** Khác với các phiên bản tự cài đặt, mô hình `sklearn.linear_model.LogisticRegression` thể hiện xu hướng dự đoán thiên lệch rõ rệt. Mô hình này nhận diện lớp *Unsatisfied* (0) rất tốt với 13,145 mẫu dự đoán đúng (True Negative cao nhất), nhưng đổi lại là việc bỏ sót nhiều mẫu *Satisfied* (1) với 1,902 mẫu sai (False Negative cao nhất). Nguyên nhân chủ yếu xuất phát từ thuật toán giải định dạng `lbfgs` và tham số điều chuẩn mặc định của thư viện đã can thiệp vào quá trình tối ưu.
- **Đánh giá tổng quan:** Nhìn chung, tất cả các cấu hình đều chứng minh được năng lực phân tách tốt trên cả hai lớp, phù hợp với tỷ lệ mất cân bằng nhẹ ($\approx 1.3 : 1$) của dữ liệu. Điểm nhấn quan trọng là các hàm phân lớp tự cài đặt hoạt động hoàn toàn chính xác, đạt được sự cân bằng lỗi tốt hơn so với các hàm tích hợp sẵn của thư viện.

5.1.4 Đường cong Precision-Recall – Logistic Regression (Test Set)

Để đánh giá sâu hơn về khả năng phân loại ở các ngưỡng quyết định khác nhau, đường cong Precision-Recall được xây dựng như ở Hình 26.



Hình 26: Đường cong Precision-Recall của các mô hình Logistic Regression

Nhận xét và đánh giá

- Hiệu năng tổng thể:** Chỉ số độ chính xác trung bình (Average Precision - AP) của cả 4 mô hình đều đạt mức rất cao (≥ 0.93), vượt xa đường cơ sở phân loại ngẫu nhiên (Baseline) ở mức 0.439. Điều này khẳng định năng lực xếp hạng (Ranking) và phân tách lớp của nhóm thuật toán Logistic Regression trên bài toán này là cực kỳ đáng tin cậy.
- Chất lượng cài đặt thuật toán:** Các mô hình tự xây dựng gồm Gradient Descent (AP = 0.9309) và Newton-Raphson (AP = 0.9307) cho kết quả bám sát nút với thư viện `sklearn` (AP = 0.9315). Các đường cong này gần như chồng khít lên nhau, minh chứng cho độ chính xác của việc tự triển khai thuật toán từ đầu.
- Hệ quả của điều chuẩn (Regularization):** Việc bổ sung L2 làm giảm nhẹ chỉ số AP xuống mức 0.9268. Quan sát đồ thị cho thấy đường cong của phiên bản L2 (màu xanh lá) nằm hơi thấp hơn so với các phiên bản còn lại, nhất quán với các phân tích về ma trận nhầm lẫn: khi trọng số gốc đã tối ưu, lực phạt bổ sung sẽ làm giảm độ nhạy bén của mô hình.
- Giá trị thực tiễn (Business Value):** Đường cong thể hiện sự đánh đổi lý tưởng: khi độ phủ (Recall) tăng thì độ chính xác (Precision) giảm dần. Tuy nhiên, Precision vẫn duy trì được mức ≥ 0.80 ngay cả khi Recall đạt ngưỡng 0.85. Trong thực tế kinh doanh, điều này cho phép doanh nghiệp nhận diện chính xác phần lớn khách hàng hài lòng mà không gây ra quá nhiều cảnh báo sai ảnh hưởng đến trải nghiệm người dùng.

5.1.5 Hàm mất mát, class-weighted loss và regularization

Hàm mất mát cơ sở được sử dụng là *binary cross-entropy* (hay negative log-likelihood của Bernoulli):

$$J(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)], \quad (23)$$

trong đó $\hat{p}_i = p(y_i = 1 \mid \mathbf{x}_i)$.

Do tập dữ liệu có mất cân bằng lớp nhẹ, nhóm sử dụng thêm *class-weighted loss* để giảm thiên lệch của mô hình về lớp chiếm đa số:

$$J_w(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N s_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)], \quad (24)$$

với

$$s_i = \begin{cases} 0.8824, & y_i = 0, \\ 1.1538, & y_i = 1. \end{cases} \quad (25)$$

Cách đặt trọng số này giúp tăng vai trò của lớp thiểu số trong quá trình tối ưu mà không cần thay đổi phân bố dữ liệu ban đầu.

Ngoài ra, nhóm còn khảo sát biến thể có regularization L2 nhằm kiểm tra nguy cơ overfitting:

$$J_{\text{reg}}(\mathbf{w}) = J_w(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}_{\text{no-bias}}\|_2^2, \quad (26)$$

trong đó hệ số chặn không bị phạt để tránh làm dịch chuyển ngưỡng quyết định một cách không cần thiết.

5.1.6 Gradient, Hessian và tính lồi của bài toán

Áp dụng quy tắc dây chuyền và tính chất $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, gradient của hàm mất mát theo vector trọng số được viết gọn dưới dạng:

$$\nabla J(\mathbf{w}) = \frac{1}{N} \tilde{\mathbf{X}}^\top (\hat{\mathbf{p}} - \mathbf{y}), \quad (27)$$

trong đó $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times (D+1)}$ là ma trận thiết kế sau khi thêm cột bias, $\hat{\mathbf{p}}$ là vector xác suất dự đoán, và $\mathbf{y}$ là vector nhãn thật.

Hessian của bài toán có dạng:

$$\mathbf{H} = \frac{1}{N} \tilde{\mathbf{X}}^\top \mathbf{R} \tilde{\mathbf{X}}, \quad (28)$$

với

$$\mathbf{R} = \text{diag}(\hat{p}_i(1 - \hat{p}_i)). \quad (29)$$

Do $\hat{p}_i(1 - \hat{p}_i) \geq 0$ với mọi i , ma trận $\mathbf{R}$ nửa xác định dương, kéo theo $\mathbf{H}$ cũng nửa xác định dương. Vì vậy, hàm mất mát của Logistic Regression là một hàm lồi. Tính chất này đảm bảo rằng Gradient Descent và Newton–Raphson, nếu được triển khai đúng, sẽ cùng hội tụ về một nghiệm tối ưu toàn cục.

5.1.7 Huấn luyện mô hình nhị phân bằng Gradient Descent

Thuật toán tối ưu thứ nhất được sử dụng là mini-batch Gradient Descent. Tại mỗi bước lặp, trọng số được cập nhật theo quy tắc:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta_t \nabla J(\mathbf{w}^{(t)}). \quad (30)$$

Trong thực nghiệm, mini-batch size được chọn là 512 và learning rate được điều chỉnh theo lịch trình *cosine annealing* để giúp quá trình hội tụ ổn định hơn ở giai đoạn cuối. Việc giảm learning rate dần theo epoch làm cho mô hình tiến gần nghiệm tối ưu một cách mượt mà hơn so với learning rate cố định.

Điều kiện dừng được chọn dựa trên sai khác loss giữa hai epoch liên tiếp hoặc khi đạt số epoch tối đa. Kết quả trên tập validation cho thấy Gradient Descent hội tụ sau **132 epoch** với

$$\text{val_loss} = 0.3350. \quad (31)$$

Kết quả này cho thấy mô hình học ổn định và đạt nghiệm tốt trên dữ liệu hiện tại.

5.1.8 Huấn luyện bằng Newton–Raphson / IRLS

Thuật toán tối ưu thứ hai là Newton–Raphson, còn được biết đến trong ngữ cảnh Logistic Regression dưới tên gọi IRLS (*Iteratively Reweighted Least Squares*). Khác với Gradient Descent chỉ dùng thông tin bậc một, Newton–Raphson sử dụng đồng thời gradient và Hessian để xác định hướng cập nhật:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \mathbf{H}^{-1} \nabla J(\mathbf{w}^{(t)}). \quad (32)$$

Trong cài đặt thực tế, thay vì tính nghịch đảo Hessian một cách tường minh, hệ tuyến tính được giải trực tiếp để ổn định số học và giảm sai số tính toán.

Trên tập validation, Newton–Raphson chỉ cần **7 iteration** để đạt

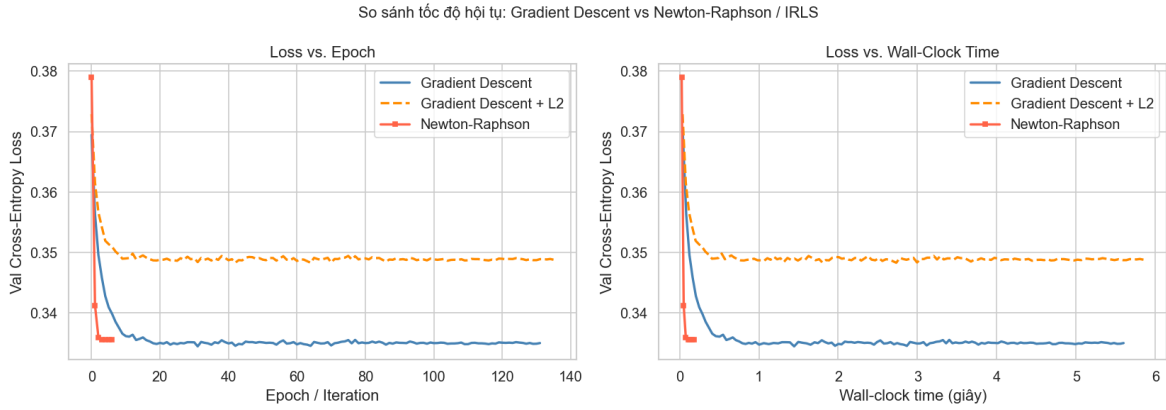
$$\text{val_loss} = 0.3356. \quad (33)$$

Dù chi phí mỗi bước lặp cao hơn do phải xử lý Hessian, số vòng lặp rất nhỏ giúp phương pháp này hội tụ nhanh hơn rõ rệt về mặt thực nghiệm. Với số chiều đặc trưng không lớn, Newton–Raphson tỏ ra đặc biệt hiệu quả.

5.1.9 So sánh Gradient Descent và Newton–Raphson

Kết quả huấn luyện cho thấy Gradient Descent và Newton–Raphson đạt chất lượng tối ưu gần như tương đương trên tập validation, nhưng khác biệt lớn về số vòng lặp. Gradient Descent cần 132 epoch, trong khi Newton–Raphson chỉ cần 7 iteration. Điều này phù hợp với lý thuyết: Gradient Descent là thuật toán bậc một nên tiến chậm hơn, còn Newton–Raphson tận dụng độ cong cục bộ của hàm mất mát thông qua Hessian nên hội tụ nhanh hơn nhiều.

Tuy nhiên, lợi thế của Newton–Raphson chỉ rõ rệt khi số chiều đặc trưng còn tương đối nhỏ. Nếu số chiều tăng rất lớn, chi phí xử lý Hessian sẽ trở thành nút thắt, và khi đó các phương pháp bậc một như Gradient Descent hoặc các biến thể stochastic sẽ phù hợp hơn.



Hình 27: So sánh tốc độ hội tụ giữa Gradient Descent và Newton–Raphson / IRLS theo số vòng lặp và theo thời gian thực thi.

Hình 27 cho thấy Newton–Raphson hội tụ nhanh hơn rõ rệt so với Gradient Descent. Trong bối cảnh số chiều đặc trưng không lớn, chi phí xử lý Hessian vẫn đủ thấp để phương pháp bậc hai mang lại lợi thế rõ rệt về wall-clock time.

5.1.10 Mở rộng Logistic Regression cho bài toán đa lớp

Mặc dù bộ dữ liệu gốc là bài toán nhị phân, nhóm vẫn mở rộng Logistic Regression sang bài toán 3 lớp để đáp ứng đầy đủ yêu cầu của đề án. Ba chiến lược được khảo sát là:

- **Softmax trực tiếp**: huấn luyện một mô hình duy nhất cho toàn bộ K lớp;
- **One-vs-Rest (OvR)**: huấn luyện K mô hình nhị phân, mỗi mô hình phân biệt một lớp với phần còn lại;
- **One-vs-One (OvO)**: huấn luyện $\binom{K}{2}$ mô hình nhị phân cho từng cặp lớp.

Với Softmax Regression, xác suất của lớp k được xác định bởi:

$$p(C_k | \mathbf{x}) = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}, \quad a_k = \mathbf{w}_k^\top \tilde{\mathbf{x}}. \quad (34)$$

Để tính đạo hàm, Jacobian của softmax có dạng:

$$\frac{\partial y_k}{\partial a_j} = y_k(\delta_{kj} - y_j), \quad (35)$$

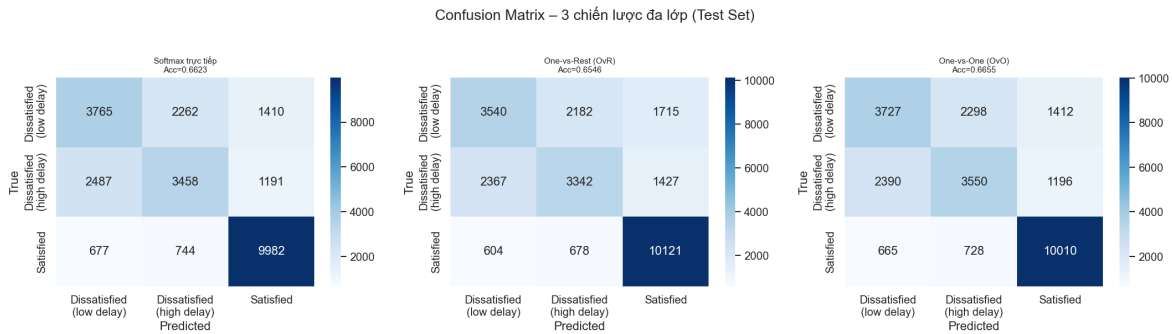
trong đó δ_{kj} là Kronecker delta. Kết hợp với categorical cross-entropy, gradient theo ma trận trọng số được rút gọn thành:

$$\nabla_{\mathbf{W}} L = \frac{1}{B} \tilde{\mathbf{X}}_B^\top (\mathbf{Y}_B - \mathbf{T}_B), \quad (36)$$

với $\mathbf{T}_B$ là ma trận one-hot của nhãn thật và $\mathbf{Y}_B$ là xác suất đầu ra của softmax.

Trong thực nghiệm, bài toán đa lớp được xây dựng bằng cách tách lớp **neutral** or **dissatisfied** thành hai lớp con theo ngưỡng median của biến delay trên tập train, trong khi lớp **satisfied** được giữ nguyên. Kết quả cho thấy **OvO đạt accuracy cao**

nhất, tiếp theo là **Softmax trực tiếp**, và cuối cùng là **OvR**. Tuy nhiên, hiệu năng của toàn bộ bài toán đa lớp thấp hơn đáng kể so với bài toán nhị phân ban đầu, cho thấy các lớp con được tạo ra có mức độ chồng lấn lớn hơn trong không gian đặc trưng.



Hình 28: Ma trận nhầm lẫn của ba chiến lược đa lớp: Softmax trực tiếp, One-vs-Rest và One-vs-One.

Từ Hình 28 có thể thấy các mô hình dự đoán khá tốt lớp **satisfied**, nhưng xảy ra sự nhầm lẫn đáng kể giữa hai lớp con của **dissatisfied**. Điều này giải thích vì sao hiệu năng đa lớp thấp hơn rõ rệt so với bài toán nhị phân gốc.

5.1.11 Nhận xét nhanh phần cài đặt

Từ góc nhìn huấn luyện mô hình, có thể rút ra ba nhận xét chính. Thứ nhất, Logistic Regression là một mô hình nền tảng mạnh, đơn giản nhưng ổn định, với bài toán tối ưu lồi và dễ diễn giải. Thứ hai, Newton–Raphson / IRLS vượt trội rõ rệt về tốc độ hội tụ khi số chiều đặc trưng không lớn. Thứ ba, khi mở rộng sang đa lớp, sự khác biệt giữa các chiến lược OvR, OvO và Softmax phản ánh rõ mức độ chồng lấn giữa các lớp con trong dữ liệu, qua đó cho thấy giới hạn của cùng một họ mô hình khi cấu trúc nhãn trở nên phức tạp hơn.

5.2 Linear Discriminant Analysis (LDA) và Quadratic Discriminant Analysis (QDA)

5.2.1 Động cơ mô hình

Khác với Logistic Regression là một mô hình *discriminative* học trực tiếp ánh xạ $x \mapsto p(y | x)$, LDA và QDA thuộc nhóm mô hình *generative*, tức là mô hình hóa phân phối có điều kiện $p(\mathbf{x} | C_k)$ cùng xác suất tiên nghiệm $p(C_k)$, rồi suy ra posterior bằng định lý Bayes. Dữ liệu đầu vào là tập đặc trưng đã tiền xử lý với 24 biến. Kích thước ba tập lần lượt là: train ($90,916 \times 24$), validation ($12,988 \times 24$) và test ($25,976 \times 24$). Bài toán là phân lớp nhị phân giữa hai nhóm hành khách *satisfied* và *neutral or dissatisfied*.

5.2.2 Fisher’s Linear Discriminant và trực giác tách lớp

Với bài toán nhị phân, Fisher’s LDA tìm một hướng chiếu $\mathbf{w}$ sao cho dữ liệu sau phép chiếu có *between-class variance* lớn nhưng *within-class variance* nhỏ. Tiêu chuẩn tối ưu có dạng:

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}, \quad (37)$$

trong đó

$$\mathbf{S}_B = (\mathbf{m}_1 - \mathbf{m}_0)(\mathbf{m}_1 - \mathbf{m}_0)^\top \quad (38)$$

là ma trận phân tán giữa hai lớp, còn

$$\mathbf{S}_W = \sum_{n \in C_0} (\mathbf{x}_n - \mathbf{m}_0)(\mathbf{x}_n - \mathbf{m}_0)^\top + \sum_{n \in C_1} (\mathbf{x}_n - \mathbf{m}_1)(\mathbf{x}_n - \mathbf{m}_1)^\top \quad (39)$$

là ma trận phân tán trong lớp.

Nhiệm tối ưu có dạng:

$$\mathbf{w}^* \propto \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_0). \quad (40)$$

Trực giác này dẫn trực tiếp đến việc dùng Fisher score để xếp hạng từng đặc trưng đơn lẻ: đặc trưng tốt phải tạo ra khoảng cách lớn giữa hai lớp nhưng đồng thời giữ độ phân tán nội bộ lớp ở mức nhỏ.

5.2.3 Giả thiết Gaussian và ước lượng tham số

Với mỗi lớp $C_k \in \{0, 1\}$, nhóm giả sử:

$$p(\mathbf{x} | C_k) = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad (41)$$

trong đó $\boldsymbol{\mu}_k$ là vector trung bình và $\boldsymbol{\Sigma}_k$ là ma trận hiệp phương sai của lớp k .

Với N_k mẫu thuộc lớp C_k , các tham số được ước lượng theo Maximum Likelihood:

$$\hat{\boldsymbol{\mu}}_k = \frac{1}{N_k} \sum_{i: y_i=k} \mathbf{x}_i, \quad \hat{\pi}_k = \frac{N_k}{N}. \quad (42)$$

Đối với QDA, mỗi lớp có một ma trận hiệp phương sai riêng:

$$\hat{\boldsymbol{\Sigma}}_k = \frac{1}{N_k} \sum_{i: y_i=k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^\top + \epsilon \mathbf{I}, \quad (43)$$

trong đó notebook dùng regularization rất nhỏ để tránh ma trận suy biến khi nghịch đảo.

Đối với LDA, hai lớp dùng chung một ma trận hiệp phương sai gộp:

$$\hat{\boldsymbol{\Sigma}} = \sum_{k=0}^1 \frac{N_k}{N} \hat{\boldsymbol{\Sigma}}_k. \quad (44)$$

Trong notebook, prior ước lượng được là $\pi_0 = 0.5667$ và $\pi_1 = 0.4333$. Mỗi lớp đều có vector trung bình 24 chiều, còn covariance của QDA và covariance chung của LDA đều có kích thước (24×24) .

5.2.4 Hàm phân biệt và quy tắc dự đoán

Từ định lý Bayes, log-posterior của lớp C_k thỏa:

$$\log p(C_k | \mathbf{x}) \propto \log p(\mathbf{x} | C_k) + \log \pi_k. \quad (45)$$

Với LDA. Khi các lớp dùng chung covariance Σ , hàm phân biệt rút gọn thành

$$g_k^{\text{LDA}}(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \boldsymbol{\mu}_k^\top \Sigma^{-1} \boldsymbol{\mu}_k + \log \pi_k. \quad (46)$$

Vì biểu thức là tuyến tính theo $\mathbf{x}$, biên quyết định là một siêu phẳng.

Với QDA. Khi mỗi lớp có covariance riêng, hàm phân biệt trở thành

$$g_k^{\text{QDA}}(\mathbf{x}) = -\frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) + \log \pi_k. \quad (47)$$

Do còn hạng toàn phương theo $\mathbf{x}$, biên quyết định của QDA là đường cong bậc hai.

Sau khi tính score cho từng lớp, chuẩn hóa về xác suất bằng softmax nhị phân:

$$p(C_k | \mathbf{x}) = \frac{\exp(g_k(\mathbf{x}))}{\sum_{j=0}^1 \exp(g_j(\mathbf{x}))}, \quad (48)$$

và gán nhãn theo quy tắc

$$\hat{y} = \arg \max_{k \in \{0,1\}} p(C_k | \mathbf{x}). \quad (49)$$

5.2.5 Fisher score và mức độ phân biệt của từng đặc trưng

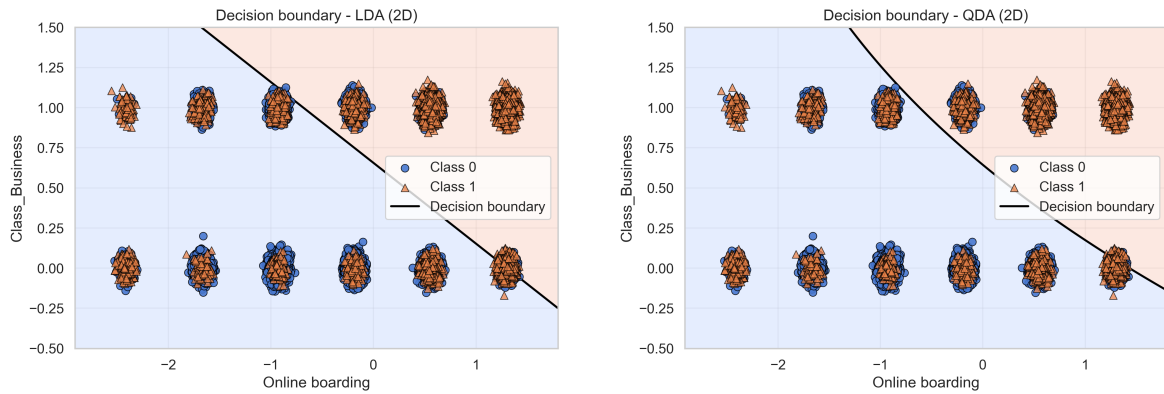
Để đánh giá khả năng tách lớp của từng biến riêng lẻ, dùng Fisher score:

$$\text{Fisher}_j = \frac{(\mu_{1j} - \mu_{0j})^2}{\sigma_{0j}^2 + \sigma_{1j}^2 + \varepsilon}. \quad (50)$$

Kết quả cho thấy những đặc trưng mang tính *trải nghiệm dịch vụ* và *hạng vé* chiếm ưu thế rõ rệt. Mười biến có Fisher score cao nhất được trình bày ở Bảng 24.

Bảng 24: Top 10 đặc trưng có Fisher score cao nhất.

Đặc trưng	Fisher score
Online boarding	0.6924
Class_Business	0.6919
Type of Travel	0.5495
Class_Eco	0.5291
Inflight entertainment	0.3908
Seat comfort	0.2855
On-board service	0.2413
Leg room service	0.2267
Cleanliness	0.2125
Flight Distance	0.1898



Hình 29: Biên quyết định 2D của LDA (trái) và QDA (phải) khi chỉ dùng hai đặc trưng `Online boarding` và `Class_Business`.

5.2.6 Minh họa decision boundary trong không gian 2D

Do biên quyết định thật nằm trong không gian 24 chiều, notebook chỉ vẽ minh họa 2D bằng hai đặc trưng có Fisher score cao nhất là `Online boarding` và `Class_Business`. Khi huấn luyện trực tiếp trên hai biến này, cả hai mô hình đều đạt cùng một độ chính xác:

$$\text{Accuracy}_{2\text{D-LDA}} = 0.7754, \quad \text{Accuracy}_{2\text{D-QDA}} = 0.7754. \quad (51)$$

Kết quả này cho thấy QDA có thể tạo đường biên cong linh hoạt hơn, nhưng lợi ích của độ linh hoạt đó gần như không chuyển thành hiệu năng phân loại tốt hơn. Sự chồng lấn mạnh giữa hai lớp cho thấy chỉ hai đặc trưng là chưa đủ để mô tả đầy đủ cấu trúc của bài toán.

5.2.7 Đánh giá trên tập test

Kết quả đánh giá đầy đủ trên tập test được tóm tắt ở Bảng 25.

Bảng 25: Kết quả LDA và QDA trên tập test.

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	AP	Log-loss	Fit time (s)
MyLDA	0.8690	0.8646	0.8319	0.8479	0.9252	0.9270	0.3497	0.0305
MyQDA	0.8534	0.8523	0.8057	0.8283	0.9171	0.9228	0.6506	0.0272

LDA vượt QDA ở toàn bộ các chỉ số quan trọng. Chênh lệch accuracy khoảng 1.56% thoát nhìn không quá lớn, nhưng chênh lệch log-loss lại rất rõ rệt (0.3497 so với 0.6506), cho thấy xác suất đầu ra của LDA ổn định và đáng tin cậy hơn đáng kể. ROC-AUC và Average Precision của LDA cũng cao hơn, chứng tỏ thứ hạng xác suất của mô hình này tốt hơn trên toàn dải ngưỡng.

5.2.8 Độ ổn định qua 5-fold Cross-Validation

Để đánh giá tính tổng quát hóa, notebook thực hiện Stratified 5-fold Cross-Validation. Kết quả trung bình và độ lệch chuẩn được cho trong Bảng 26.

Các độ lệch chuẩn đều rất nhỏ, chỉ cỡ 10^{-3} , nên thứ hạng giữa hai mô hình khá ổn định qua các fold khác nhau. Điều này cho thấy ưu thế của LDA không phải là kết

Bảng 26: Kết quả 5-fold Cross-Validation (mean $\pm$ std).

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC
MyLDA	0.8719 ± 0.0019	0.8645 ± 0.0029	0.8352 ± 0.0043	0.8496 ± 0.0024	0.9262 ± 0.0015
MyQDA	0.8564 ± 0.0019	0.8552 ± 0.0026	0.8048 ± 0.0036	0.8292 ± 0.0024	0.9197 ± 0.0015

quả may mắn của một lần chia train/test cụ thể, mà lặp lại nhất quán trên nhiều cách chia dữ liệu.

5.2.9 Kiểm định ý nghĩa thống kê bằng McNemar’s test

Trên cùng tập test, notebook tiếp tục so sánh hai mô hình bằng McNemar’s test. Kết quả thu được:

Đại lượng	Giá trị
b (LDA đúng, QDA sai)	1179
c (LDA sai, QDA đúng)	773
χ^2 xấp xỉ	84.0292
P-value	$< 10^{-6}$

Vì p-value nhỏ hơn rất nhiều so với mức ý nghĩa 0.05, có thể bác bỏ giả thuyết không rằng hai mô hình có cùng hiệu năng trên tập test. Nói cách khác, lợi thế của LDA không chỉ mang ý nghĩa thực nghiệm mà còn có ý nghĩa thống kê.

5.2.10 Phân tích calibration và độ tin cậy của xác suất đầu ra

Một điểm nổi bật trong thực nghiệm là sự khác biệt rõ rệt về khả năng hiệu chỉnh xác suất (calibration) giữa hai mô hình. Trong bối cảnh này, một mô hình được coi là calibrated tốt khi đường cong của nó bám sát đường chéo lý tưởng $y = x$, tức xác suất dự đoán phản ánh trung thực mức độ chắc chắn của mô hình.

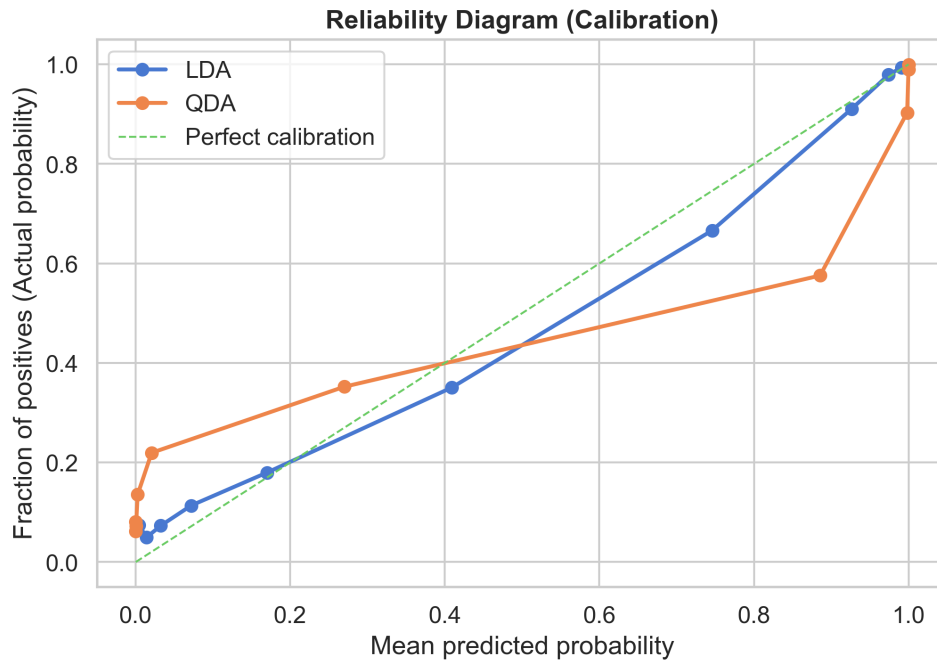
Như thể hiện trong Hình 30, đường reliability diagram của LDA bám khá sát đường chéo lý tưởng trên phần lớn miền xác suất. Điều đó cho thấy các xác suất mà LDA đưa ra tương đối đáng tin cậy: khi mô hình dự đoán xác suất cao cho lớp dương, tần suất thực tế của lớp dương trong nhóm đó cũng tăng tương ứng. Ngược lại, đường cong của QDA lệch khỏi đường chéo rõ hơn và có xu hướng uốn cong theo dạng “S ngược”, phản ánh hiện tượng *overconfident*: mô hình thường gán các xác suất quá gần 0 hoặc 1, nhưng mức độ tự tin này không tương xứng với xác suất đúng thực sự.

Sự khác biệt về calibration này phù hợp chặt chẽ với kết quả log-loss trên tập test:

$$\text{Log-loss}_{\text{LDA}} = 0.3497, \quad \text{Log-loss}_{\text{QDA}} = 0.6506. \quad (52)$$

Mặc dù chênh lệch accuracy giữa hai mô hình không quá lớn, log-loss của QDA cao hơn rất nhiều. Điều này cho thấy vấn đề của QDA không chỉ nằm ở số lượng dự đoán sai, mà còn nằm ở chỗ các dự đoán sai đó thường đi kèm với mức tự tin quá cao. Vì log-loss phạt đặc biệt nặng những trường hợp “sai nhưng rất chắc chắn”, nên chỉ số này phản ánh rất rõ nhược điểm của QDA trong bài toán hiện tại.

Xét theo góc nhìn bản chất mô hình, kết quả trên là hoàn toàn hợp lý. LDA giả sử hai lớp có chung một ma trận hiệp phương sai và do đó sử dụng covariance chung



Hình 30: Biểu đồ độ tin cậy (Reliability Diagram) của LDA và QDA trên tập test.

trong hàm phân biệt. Giả thiết này làm mô hình ít linh hoạt hơn về mặt hình học, nhưng đồng thời lại hoạt động như một dạng *regularization tự nhiên*: các score phân lớp ít bị dao động cục bộ, xác suất đầu ra ổn định hơn và ít bị đẩy về các giá trị cực đoan. Ngược lại, QDA phải ước lượng riêng từng ma trận hiệp phương sai Σ_k và sử dụng các nghịch đảo Σ_k^{-1} tương ứng trong hàm phân biệt. Khi một số phương sai hoặc covariance cục bộ biến thiên mạnh, score của QDA sẽ trở nên nhạy hơn với nhiễu và dễ tạo ra các xác suất kém ổn định.

Nói cách khác, ưu thế lý thuyết của QDA là khả năng biểu diễn biên quyết định cong linh hoạt hơn, nhưng chính độ linh hoạt đó cũng khiến mô hình dễ trở nên quá tự tin trong những vùng dữ liệu mà việc ước lượng covariance chưa đủ chắc chắn. Trong khi đó, LDA tuy đơn giản hơn nhưng lại cho phân phối xác suất đầu ra đáng tin cậy hơn, điều đặc biệt quan trọng nếu mô hình được dùng trong các bài toán cần ra quyết định theo ngưỡng xác suất thay vì chỉ quan tâm đến nhãn dự đoán cuối cùng. Trên bộ dữ liệu Airline Passenger Satisfaction đã tiền xử lý, kết quả calibration vì thế tiếp tục củng cố kết luận rằng LDA là lựa chọn phù hợp hơn QDA, không chỉ về mặt accuracy và F1 mà còn ở chất lượng xác suất dự đoán.

5.2.11 Vì sao LDA phù hợp hơn QDA trong bài toán này

Với $d = 24$ đặc trưng và $K = 2$ lớp, số tham số cần học của hai mô hình là:

- **LDA:** $Kd + \frac{d(d+1)}{2} + (K-1) = 48 + 300 + 1 = 349$ tham số.
- **QDA:** $Kd + K\frac{d(d+1)}{2} + (K-1) = 48 + 600 + 1 = 649$ tham số.

Như vậy QDA phải học gần gấp đôi số tham số so với LDA. Trong notebook, nhóm còn đo trực tiếp mức độ giống nhau giữa hai ma trận hiệp phương sai lớp và thu được hai chỉ số quan trọng:

- tương quan phần tam giác trên của hai ma trận bằng 0.8259;
- tương quan của các phương sai trên đường chéo chính bằng 0.6566.

Hai con số này cho thấy giả thiết “cùng một cấu trúc hiệp phương sai” của LDA là khá hợp lý đối với bộ dữ liệu *Airline Passenger Satisfaction*. Khi cấu trúc covariance giữa hai lớp đã tương đối đồng dạng, việc QDA học thêm độ linh hoạt thường chỉ làm tăng variance của mô hình thay vì cải thiện biên quyết định một cách thực chất.

Từ cả lý thuyết lẫn thực nghiệm, có thể rút ra kết luận sau:

- **Nên chọn LDA** khi các lớp có covariance tương đồng, cần mô hình ổn định, ít tham số, và muốn xác suất đầu ra đáng tin cậy hơn.
- **Chỉ nên chọn QDA** khi covariance giữa các lớp khác nhau rõ rệt và có đủ dữ liệu để ước lượng ổn định các ma trận hiệp phương sai riêng.

Trong phạm vi bài toán này, LDA là lựa chọn phù hợp hơn QDA vì vừa đơn giản hơn về mặt tham số, vừa cho hiệu năng test, cross-validation, calibration và ý nghĩa thống kê đều tốt hơn một cách nhất quán.

5.3 Perceptron và Logistic Regression có Regularization

5.3.1 Thiết lập dữ liệu

Thí nghiệm trong phần này được thực hiện trên bộ dữ liệu *Airline Passenger Satisfaction* đã qua bước tiền xử lý thống nhất từ các phần trước. Sau khi chia dữ liệu, tập huấn luyện có kích thước $(90,916 \times 24)$, tập validation có kích thước $(12,988 \times 24)$ và tập kiểm tra có kích thước $(25,976 \times 24)$. Việc sử dụng cùng một quy trình tiền xử lý và cùng một cách chia dữ liệu giúp các kết quả ở phần này có thể so sánh trực tiếp với Logistic Regression và LDA/QDA đã trình bày trước đó.

Do dữ liệu có mất cân bằng lớp nhẹ, nhóm sử dụng trọng số lớp:

$$w_0 = 0.8824, \quad w_1 = 1.1538. \quad (53)$$

Các trọng số này được tính theo hướng tỉ lệ nghịch với tần suất lớp, giúp hàm mất mát chú ý nhiều hơn đến lớp thiểu số mà không cần thay đổi phân bố dữ liệu ban đầu. Trong thực nghiệm, class-weighted loss chỉ được áp dụng cho Logistic Regression; còn Perceptron được dùng chủ yếu như một mô hình nền để minh họa giới hạn của bộ phân lớp tuyến tính.

5.3.2 Perceptron

Mục tiêu thí nghiệm. Trong notebook Perceptron không được dùng như mô hình chính trên bộ dữ liệu *Airline Passenger Satisfaction*, mà được cài đặt từ đầu để minh họa trực tiếp định lý hội tụ của một bộ phân lớp tuyến tính. Vì vậy, phần thực nghiệm Perceptron được thực hiện trên hai bộ dữ liệu toy hai chiều để có thể quan sát rõ đường biên quyết định và số lỗi phân loại theo từng epoch.

Mô hình và quy tắc cập nhật. Perceptron là bộ phân lớp tuyến tính với hàm quyết định:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b). \quad (54)$$

Với quy ước nhãn $y_i \in \{-1, +1\}$, khi một mẫu bị phân loại sai thì tham số được cập nhật theo quy tắc:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta y_i \mathbf{x}_i, \quad b^{(t+1)} = b^{(t)} + \eta y_i, \quad (55)$$

trong đó η là learning rate. Notebook sử dụng phiên bản `PerceptronScratch` với $\eta = 0.01$, tối đa 200 epoch, có *shuffle* dữ liệu qua từng vòng lặp và khởi tạo ngẫu nhiên cố định bằng `random_state = 42`.

Thiết kế thực nghiệm kiểm chứng hội tụ. Để kiểm chứng trực giác lý thuyết, nhóm thử Perceptron trên hai bộ dữ liệu nhân tạo:

- **Dữ liệu phân chia tuyến tính:** sinh bởi `make_classification` với 400 mẫu, 2 đặc trưng, `class_sep = 2.5`, không thêm nhiễu nhãn.
- **Dữ liệu phi tuyến:** sinh bởi `make_moons` với 400 mẫu và mức nhiễu `noise = 0.2`.

Hai bộ dữ liệu này được chọn để tạo ra một đối sánh rõ ràng: một trường hợp hoàn toàn phù hợp với giả thiết tuyến tính của Perceptron và một trường hợp vi phạm giả thiết đó.

Kết quả trên dữ liệu tuyến tính. Trên bộ dữ liệu phân chia tuyến tính, thuật toán hội tụ tại **epoch 120**, tức số lỗi phân loại giảm về 0. Khi đánh giá ngay trên tập dữ liệu này, mô hình đạt:

$$\text{Accuracy} = \text{Precision} = \text{Recall} = \text{F1} = 1.0000. \quad (56)$$

Ma trận nhầm lẫn tương ứng là

$$\begin{bmatrix} 200 & 0 \\ 0 & 200 \end{bmatrix}. \quad (57)$$

Kết quả này đúng với định lý hội tụ Perceptron: nếu dữ liệu phân chia tuyến tính, thuật toán sẽ tìm được một siêu phẳng phân tách sau hữu hạn bước cập nhật.

Kết quả trên dữ liệu phi tuyến. Trên bộ dữ liệu `make_moons`, Perceptron không thể đưa số lỗi về 0 do biên quyết định của mô hình luôn là tuyến tính trong khi dữ liệu có cấu trúc cong. Kết quả đánh giá thu được là:

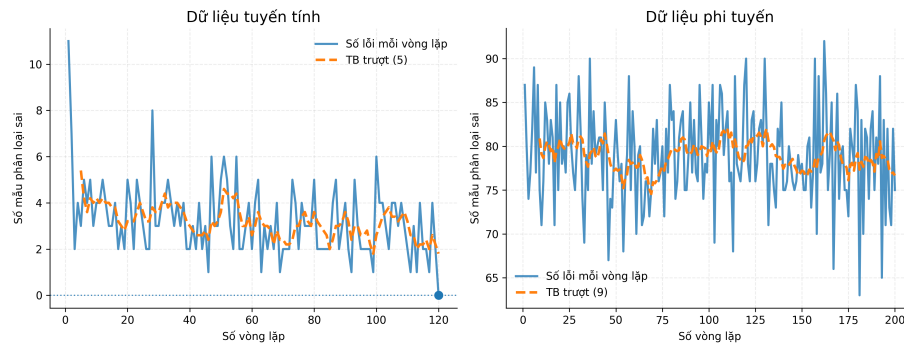
$$\text{Accuracy} = 0.8000, \quad \text{Precision} = 0.8191, \quad \text{Recall} = 0.7700, \quad \text{F1} = 0.7938. \quad (58)$$

Ma trận nhầm lẫn tương ứng là

$$\begin{bmatrix} 166 & 34 \\ 46 & 154 \end{bmatrix}. \quad (59)$$

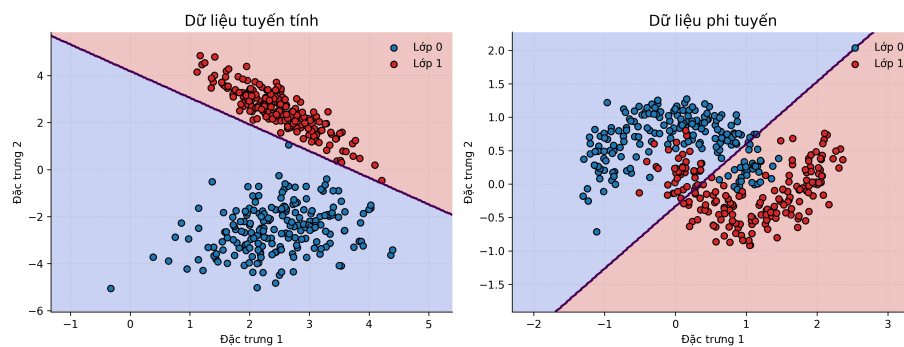
Các chỉ số này cho thấy mô hình vẫn học được một ranh giới tuyến tính tương đối hợp lý, nhưng không thể mô tả được cấu trúc phi tuyến thực sự của dữ liệu nên sai số còn lại là không thể tránh khỏi.

Hành vi hội tụ của Perceptron trên dữ liệu tuyến tính và phi tuyến



Hình 31: Số lỗi phân loại theo epoch của Perceptron trên dữ liệu tuyến tính và phi tuyến.

Đường biên quyết định của Perceptron



Hình 32: Đường biên quyết định của Perceptron trên hai bộ dữ liệu minh họa.

Nhận xét: Hai hình trên cho thấy rất rõ vai trò của giả thiết tuyến tính. Ở dữ liệu tuyến tính, số lỗi giảm nhanh về 0 và đường biên quyết định phân tách hoàn toàn hai lớp. Ngược lại, ở dữ liệu phi tuyến, số lỗi dao động quanh một mức dương và đường thẳng phân tách luôn cắt qua các vùng giao thoa giữa hai lớp. Vì vậy, Perceptron phù hợp để minh họa cơ chế học của bộ phân lớp tuyến tính và định lý hội tụ, nhưng không phải lựa chọn phù hợp cho các bài toán thực tế có cấu trúc biên quyết định phức tạp. Đây cũng là động lực để chuyển sang Logistic Regression — mô hình vẫn tuyến tính theo tham số nhưng tối ưu một hàm mất mát trơn, mô hình hóa xác suất và ổn định hơn trong thực hành.

5.3.3 Logistic Regression với L1/L2

Mô hình xác suất. Khác với Perceptron chỉ cho nhãn rời rạc, Logistic Regression mô hình hóa trực tiếp xác suất thuộc lớp dương:

$$p(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \mathbf{w}^\top \mathbf{x} + b. \quad (60)$$

Nhờ đó, mô hình không chỉ đưa ra quyết định phân lớp mà còn cho phép điều chỉnh ngưỡng dự đoán và đánh giá chất lượng xác suất đầu ra.

Hàm mất mát có trọng số lớp. Với dữ liệu mất cân bằng nhẹ, hàm mất mát binary cross-entropy được điều chỉnh bằng trọng số lớp:

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N s_i \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right], \quad (61)$$

trong đó $s_i = w_1$ nếu $y_i = 1$ và $s_i = w_0$ nếu $y_i = 0$.

Khi thêm regularization, hàm mục tiêu trở thành:

$$\mathcal{J}_{L1}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 \quad (62)$$

đối với L1 regularization, và

$$\mathcal{J}_{L2}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \quad (63)$$

đối với L2 regularization.

Tối ưu tham số. Toàn bộ mô hình được cài đặt từ đầu bằng `numpy`. Gradient của phần cross-entropy có dạng:

$$\nabla \mathcal{L}(\mathbf{w}) = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}), \quad (64)$$

và được cập nhật lặp theo learning rate cố định. Trong thực nghiệm, nhóm sử dụng $\eta = 0.05$ và tối đa 1200 vòng lặp ở giai đoạn chọn mô hình, sau đó tăng lên 1500 vòng lặp khi huấn luyện lại mô hình cuối cùng trên tập train + validation.

5.3.4 Chọn λ bằng Stratified 5-Fold CV

Để chọn giá trị λ , nhóm sử dụng Stratified 5-Fold Cross-Validation trên tập train với lưới giá trị:

$$\lambda \in \{0, 1, 10, 50, 100, 500, 1000\}. \quad (65)$$

Kết quả thực nghiệm cho thấy:

- với **L2 regularization**, giá trị tốt nhất là $\lambda = 0$,
- với **L1 regularization**, giá trị tốt nhất là $\lambda = 10$.

Kết quả này cho thấy dữ liệu sau tiền xử lý đã khá ổn định và không quá nhạy với regularization strength. Việc phạt quá mạnh không mang lại cải thiện rõ rệt về khả năng tổng quát hóa, đặc biệt với L2 thì nghiệm tốt nhất thực tế lại là nghiệm không regularization.

5.3.5 Tối ưu ngưỡng trên validation

Sau khi chọn được λ , hai mô hình L1 và L2 được huấn luyện trên tập train rồi đánh giá trên tập validation. Thay vì cố định ngưỡng 0.5, ngưỡng phân loại được dò tìm trên khoảng từ 0.10 đến 0.90 để tối đa hóa F1-score. Kết quả tốt nhất của cả hai mô hình đều đạt tại:

$$\tau^* = 0.60. \quad (66)$$

Bảng 27 trình bày kết quả trên tập validation.

Bảng 27: Kết quả Logistic Regression trên tập validation.

Mô hình	Acc	Prec	Recall	F1	ROC-AUC	Brier
Logistic Regression + L1	0.8719	0.8874	0.8067	0.8451	0.9230	0.1028
Logistic Regression + L2	0.8719	0.8876	0.8065	0.8451	0.9230	0.1028
Baseline không class weights	0.8692	0.8675	0.8239	0.8452	0.9229	0.1001

Có thể thấy L1 và L2 cho hiệu năng gần như trùng nhau trên tập validation. So với mô hình không dùng class weights, hai mô hình có trọng số lớp giúp tăng precision của lớp dương lên đáng kể, nhưng recall giảm nhẹ. Vì mục tiêu ở đây là cân bằng giữa precision và recall thông qua F1-score, chênh lệch giữa ba mô hình là rất nhỏ.

5.3.6 Ảnh hưởng của regularization lên sparsity

Một trong những mục tiêu quan trọng của regularization là kiểm soát độ lớn của tham số và, trong trường hợp L1, tạo ra nghiệm thưa. Tuy nhiên, trên bộ dữ liệu này, hiệu ứng sparsity xuất hiện khá yếu. Số lượng trọng số khác 0 của hai mô hình là:

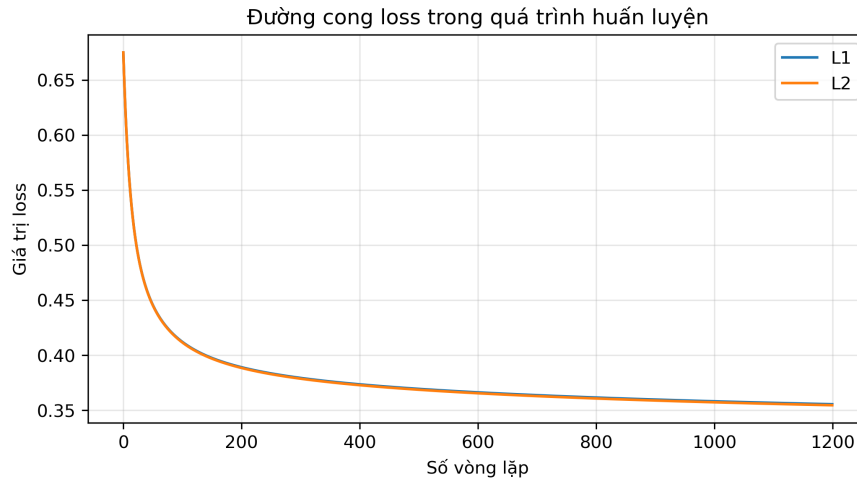
$$L1 : 23/24, \quad L2 : 23/24. \quad (67)$$

Chuẩn của vector trọng số cũng rất gần nhau:

$$\|\mathbf{w}_{L1}\|_1 = 7.3419, \quad \|\mathbf{w}_{L2}\|_1 = 7.3787, \quad (68)$$

$$\|\mathbf{w}_{L1}\|_2 = 2.1601, \quad \|\mathbf{w}_{L2}\|_2 = 2.1666. \quad (69)$$

Điều này cho thấy trong bối cảnh hiện tại, cả L1 và L2 chủ yếu đóng vai trò ổn định hóa nghiệm hơn là tạo ra một bước lựa chọn đặc trưng thật mạnh. Các đặc trưng có trị tuyệt đối hệ số lớn nhất giữa hai mô hình cũng tương đối tương đồng, trong đó *Type of Travel*, *Online boarding* và *Customer Type* là các biến nổi bật nhất.



Hình 33: Đường cong loss của các biến thể Logistic Regression có regularization trong quá trình huấn luyện.

Hình 33 cho thấy quá trình tối ưu của các biến thể Logistic Regression diễn ra ổn định, không có dao động mạnh, và các nghiệm cuối cùng của L1 và L2 khá gần nhau. Điều này phù hợp với quan sát rằng regularization không làm thay đổi mạnh hiệu năng của mô hình trong bài toán hiện tại.

5.3.7 Mô hình cuối trên tập test

Dựa trên F1-score của tập validation, notebook chọn **Logistic Regression + L1** làm mô hình cuối cùng. Mô hình này được huấn luyện lại trên tập *train + validation* với $\lambda = 10$, learning rate $\eta = 0.05$, 1500 vòng lặp và ngưỡng dự đoán tối ưu $\tau = 0.60$. Kết quả đánh giá trên tập test được trình bày ở Bảng 28.

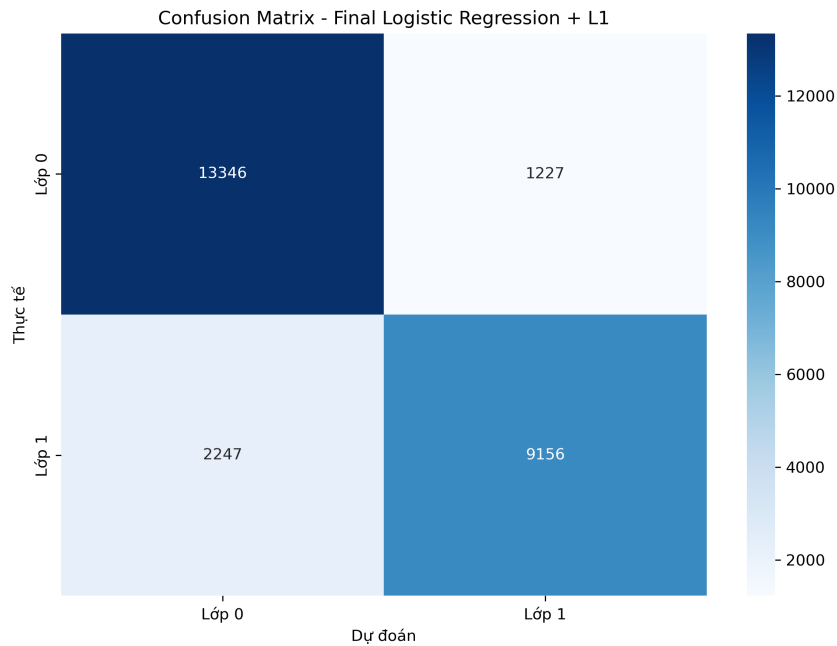
Bảng 28: Kết quả mô hình cuối trên tập test.

Mô hình	Acc	Prec	Recall	F1	ROC-AUC	AP	Brier
Final Logistic Regression + L1	0.8663	0.8818	0.8029	0.8405	0.9231	0.9260	0.1039

Ma trận nhầm lẫn tương ứng là:

$$\begin{bmatrix} 13346 & 1227 \\ 2247 & 9156 \end{bmatrix}. \quad (70)$$

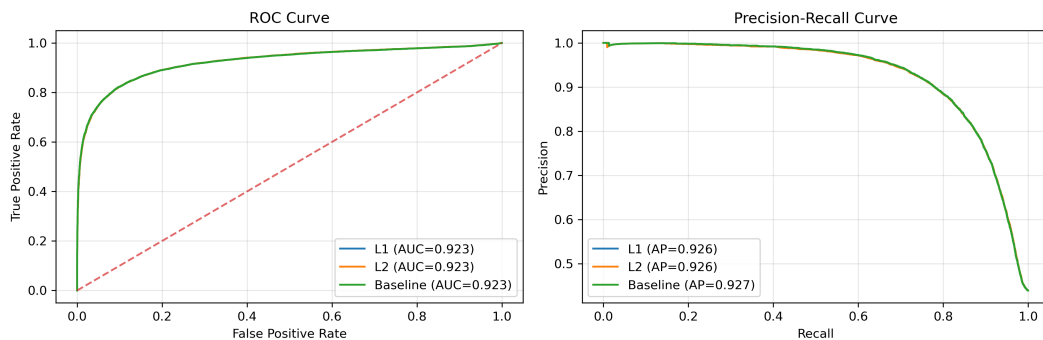
Hình 34 cho thấy mô hình phân loại đúng phần lớn mẫu của cả hai lớp, nhưng vẫn còn một số lượng đáng kể mẫu lớp dương bị dự đoán nhầm sang lớp âm, vì vậy recall của lớp dương vẫn thấp hơn precision.



Hình 34: Confusion matrix của mô hình Logistic Regression cuối cùng trên tập test.

5.3.8 ROC curve và Precision–Recall curve trên tập test

Để so sánh công bằng tác động của regularization và class-weighted loss, notebook huấn luyện lại ba biến thể trên toàn bộ tập *train + validation*: **L1**, **L2** và **baseline không class weights**. ROC curve và Precision–Recall curve trên tập test được minh họa ở Hình 35.



Hình 35: ROC curve và Precision–Recall curve của ba biến thể Logistic Regression trên tập test.

Các đường cong gần như chồng khít lên nhau. Điều này cho thấy cả ba mô hình đều có khả năng xếp hạng tốt với ROC-AUC xấp xỉ 0.923 và Average Precision quanh 0.926–0.927. Nói cách khác, regularization và việc dùng class-weighted loss không tạo ra cải thiện rõ rệt về khả năng phân biệt tổng thể giữa hai lớp trong bài toán này.

5.3.9 Kiểm định ý nghĩa thống kê bằng McNemar’s test

McNemar’s test để kiểm tra xem các khác biệt nhỏ về chỉ số có thực sự mang ý nghĩa thống kê hay không. Ba cặp mô hình được so sánh là L1 với L2, L2 với baseline và L1 với baseline. Kết quả được tóm tắt trong Bảng 29.

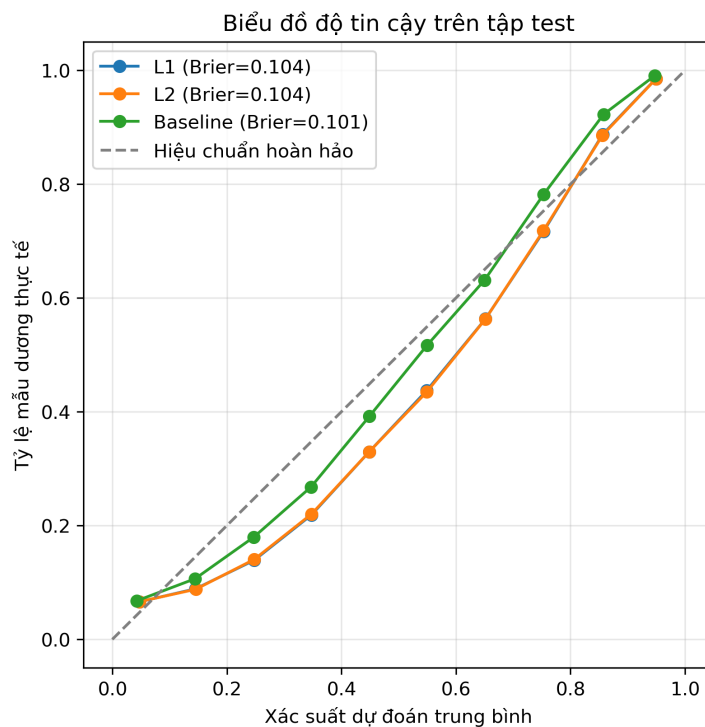
Bảng 29: McNemar’s test giữa các biến thể Logistic Regression trên tập test.

Cặp mô hình	n_{01}	n_{10}	P-value	Kết luận
L1 vs. L2	2	5	0.4531	Không khác biệt có ý nghĩa
L2 vs. Baseline	222	224	0.9622	Không khác biệt có ý nghĩa
L1 vs. Baseline	220	225	0.8496	Không khác biệt có ý nghĩa

Ở đây, n_{01} là số mẫu mà mô hình thứ nhất dự đoán đúng còn mô hình thứ hai dự đoán sai; n_{10} được hiểu ngược lại. Tất cả p-value đều lớn hơn 0.05, nên chưa có đủ bằng chứng để kết luận rằng các mô hình khác nhau về mặt thống kê. Điều này củng cố nhận xét rằng L1, L2 và baseline đang cho hiệu năng gần như tương đương trên cùng tập test.

5.3.10 Phân tích calibration

Với các mô hình xác suất như Logistic Regression, ngoài nhãn dự đoán thì độ tin cậy của xác suất đầu ra cũng rất quan trọng. Notebook đánh giá calibration bằng *reliability diagram* và *Brier score*; kết quả được minh họa ở Hình 36 và Bảng 30.



Hình 36: Biểu đồ độ tin cậy của L1, L2 và baseline trên tập test.

Biểu đồ độ tin cậy cho thấy cả ba mô hình đều bám khá gần đường chéo lý tưởng, tức xác suất dự đoán nhìn chung là đáng tin cậy. Tuy nhiên, các đường cong vẫn hơi nằm dưới đường chéo ở một vài vùng xác suất, hàm ý mô hình có xu hướng *overconfident* nhẹ. Brier score của ba mô hình gần như tương đương; baseline nhỉnh hơn một chút nhưng chênh lệch là rất nhỏ.

Bảng 30: Đánh giá calibration trên tập test.

Mô hình	Brier score	ROC-AUC	AP
L1	0.103876	0.923121	0.926023
L2	0.103847	0.923128	0.926006
Baseline	0.101202	0.923032	0.926575

5.3.11 Bảng tổng kết và kết luận

Để nhìn toàn bộ thí nghiệm trong một khung chung, Bảng 31 tổng hợp các kết quả chính trong notebook.

Bảng 31: Bảng tổng kết kết quả của phần Perceptron và Logistic Regression có regularization.

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	AP	Brier
Perceptron – linear toy data	1.0000	1.0000	1.0000	1.0000	–	–	–
Perceptron – nonlinear toy data	0.8000	0.8191	0.7700	0.7938	–	–	–
Logistic Regression + L1 (Validation)	0.8719	0.8874	0.8067	0.8451	0.9230	0.9270	0.1028
Logistic Regression + L2 (Validation)	0.8719	0.8876	0.8065	0.8451	0.9230	0.9270	0.1028
Baseline không class weights	0.8692	0.8675	0.8239	0.8452	0.9229	0.9274	0.1001
Final Logistic Regression + L1	0.8663	0.8818	0.8029	0.8405	0.9231	0.9260	0.1039

Từ bảng tổng kết, có thể rút ra bốn kết luận chính. Thứ nhất, Perceptron minh họa rất rõ giới hạn của bộ phân lớp tuyến tính thuần túy: hội tụ hoàn toàn trên dữ liệu tuyến tính nhưng suy giảm đáng kể trên dữ liệu phi tuyến. Thứ hai, Logistic Regression ổn định hơn nhờ mô hình hóa xác suất và tối ưu trực tiếp cross-entropy loss. Thứ ba, trong bộ dữ liệu này, L1 và L2 cho nghiệm và hiệu năng gần như giống nhau; hiệu ứng sparsity của L1 chưa thực sự nổi bật. Cuối cùng, class-weighted loss, McNemar’s test và phân tích calibration cùng chỉ ra rằng các khác biệt giữa L1, L2 và baseline là khá nhỏ, nên lựa chọn mô hình ở đây nên ưu tiên tính đơn giản và khả năng diễn giải hơn là kỳ vọng vào chênh lệch hiệu năng lớn.

6 So Sánh Toàn Diện Ba Nhóm Mô Hình

Trong phần này, nhóm tổng hợp và so sánh toàn diện ba nhóm mô hình đã được cài đặt trong suốt quá trình thực hiện đồ án:

1. **Logistic Regression:** bao gồm các biến thể Gradient Descent (GD), GD với L2 regularization (GD+L2), Newton–Raphson/IRLS, và L1 regularization (L1).
2. **LDA / QDA:** hai mô hình sinh Gaussian (Linear và Quadratic Discriminant Analysis).
3. **Perc.:** bộ phân lớp tuyến tính cổ điển không có đầu ra xác suất.

Để đảm bảo tính công bằng, toàn bộ mô hình được *retrain* trên tập **train + validation** và đánh giá thống nhất trên cùng một tập test gồm 25.976 mẫu.

6.1 Kết quả trên Tập Test

Bảng 32 trình bày kết quả đánh giá của tất cả các mô hình trên tập test theo sáu chỉ số chính.

Bảng 32: So sánh hiệu năng các mô hình trên tập test (25.976 mẫu).

Model	Acc	Prec	Rec	F1	AUC	AP
LR-GD	0.8517	0.8258	0.8391	0.8324	0.9165	0.9195
LR-L2	0.8517	0.8258	0.8391	0.8324	0.9165	0.9195
LR-NR	0.8660	0.8664	0.8215	0.8434	0.9228	0.9269
LDA	0.8687	0.8641	0.8316	0.8476	0.9252	0.9268
QDA	0.8535	0.8524	0.8058	0.8285	0.9171	0.9228
Perc.	0.8421	0.8743	0.7478	0.8061	0.9004	0.8945
LR-L1	0.8722	0.8709	0.8323	0.8512	0.9270	0.9316

Nhận xét chung về bảng kết quả:

Nhìn vào bảng 32, có thể thấy khá rõ sự phân hóa giữa các nhóm mô hình. Mô hình **LR-L1** dẫn đầu ở hầu hết các chỉ số quan trọng, đạt Accuracy 87.22%, F1-score 85.12% và ROC-AUC 92.70%. Đây là kết quả khá ấn tượng nếu xét rằng đây vẫn là một mô hình tuyến tính được cài đặt từ đầu.

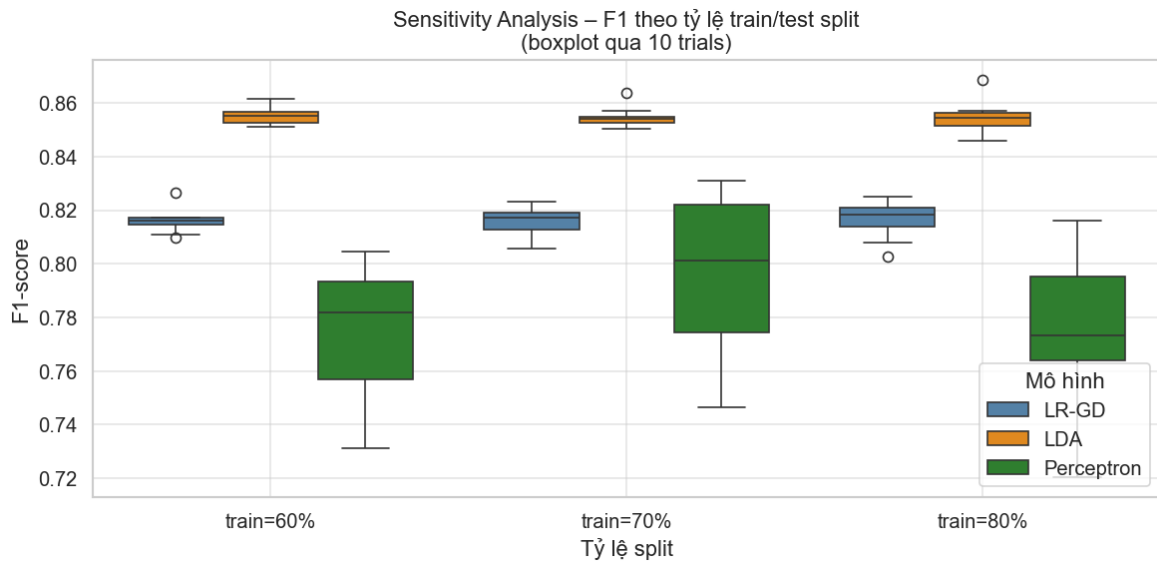
LDA đứng thứ hai với Accuracy 86.87% và F1 84.76%, vượt trội hơn LR-GD một khoảng cách đáng chú ý (khoảng 1.7% Accuracy). Điều này phản ánh rõ lợi thế của LDA khi giả thiết phân phối Gaussian khá phù hợp với bộ dữ liệu sau khi đã qua tiền xử lý.

Hai biến thể LR-GD và LR-L2 cho kết quả hoàn toàn giống nhau, cho thấy regularization L2 với $\lambda = 0$ (giá trị tốt nhất từ cross-validation) không tạo ra sự thay đổi nào so với mô hình không regularization. Điều này ngụ ý rằng dữ liệu đã khá ổn định sau khi chuẩn hóa, và nguy cơ overfitting không quá cao.

Perc. mặc dù đạt Precision cao nhất (87.43%) nhờ ngưỡng phân loại bảo thủ, nhưng lại có Recall thấp nhất (74.78%) và F1-score kém nhất (80.61%), đồng thời thời gian huấn luyện lên đến 90 giây – cao hơn hẳn so với tất cả các mô hình còn lại. Sự mất cân bằng giữa Precision và Recall của Perc. cho thấy mô hình đang bỏ sót nhiều hành khách thực sự hài lòng, điều này không lý tưởng trong bài toán thực tế.

6.2 Phân tích Độ nhạy cảm với Tỷ lệ Train/Test Split

Để đánh giá mức độ ổn định của mô hình khi thay đổi lượng dữ liệu huấn luyện, nhóm thực hiện thí nghiệm với ba tỷ lệ split khác nhau: train = 60%, 70%, 80% (tương ứng test = 40%, 30%, 20%). Mỗi điều kiện được lặp lại 10 lần với seed khác nhau để ước lượng phân phối của F1-score.



Hình 37: Biểu đồ Sensitivity Analysis - F1 theo tỷ lệ Train/Test Split

Bảng 33: Tóm tắt sensitivity analysis: F1-score mean $\pm$ std theo tỷ lệ train/test split.

Train split	Mô hình	F1 mean	F1 std
60%	LDA	0.8470	0.0019
	LR-GD	0.8105	0.0044
	Perc.	0.7690	0.0380
70%	LDA	0.8475	0.0024
	LR-GD	0.8117	0.0059
	Perc.	0.7579	0.0632
80%	LDA	0.8474	0.0036
	LR-GD	0.8119	0.0072
	Perc.	0.7740	0.0410

Nhận xét:

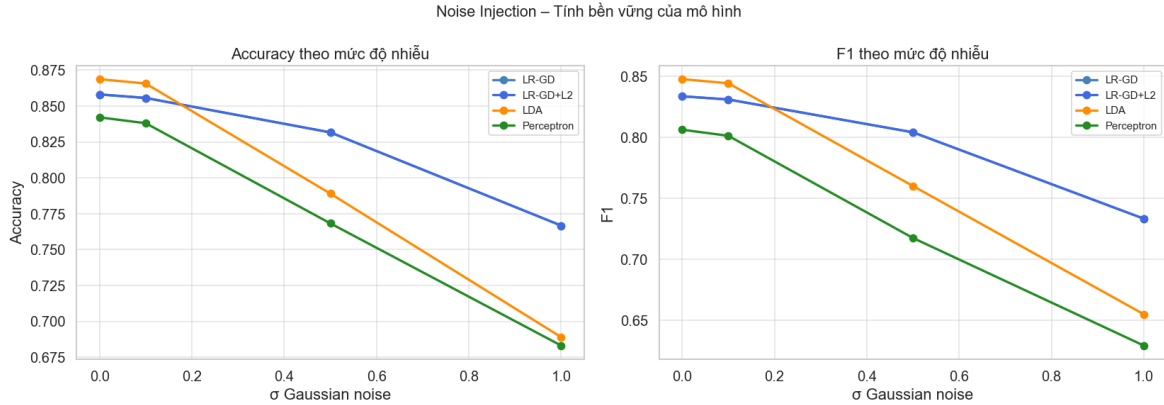
Bảng 33 cho thấy **LDA là mô hình ổn định nhất** theo mọi điều kiện split. F1-score của LDA gần như không thay đổi khi tăng từ 60% lên 80% dữ liệu huấn luyện (84.70% $\rightarrow$ 84.74%), điều này cho thấy mô hình đã đạt “saturation” về dữ liệu – tức là thêm dữ liệu không cải thiện đáng kể hiệu năng. Đây là một đặc điểm của mô hình generative: khi giả thiết phân phối đúng, ước lượng tham số hội tụ nhanh.

Perc. cho thấy sự bất ổn định rõ rệt nhất: F1 std ở mức 0.038–0.063, cao gấp 15–25 lần so với LDA. Điều này không chỉ phản ánh variance cao của mô hình, mà còn cho thấy Perc. rất nhạy cảm với thứ tự dữ liệu và điều kiện khởi tạo. Khi dữ liệu thực tế không phân tách tuyến tính hoàn toàn (như trong bộ dữ liệu Airline Passenger Satisfaction), Perc. không có đảm bảo lý thuyết về hội tụ và dễ dẫn đến nghiệm không nhất quán.

LR-GD nằm ở mức trung gian: ổn định hơn Perc. nhưng kém hơn LDA. Khi tăng dữ liệu huấn luyện từ 60% lên 80%, F1 của LR-GD cải thiện nhẹ (81.05% $\rightarrow$ 81.19%), cho thấy mô hình vẫn có thể được hưởng lợi từ thêm dữ liệu.

6.3 Phân tích Tính bền vững với Nhiễu (Noise Robustness)

Để đánh giá khả năng chống chịu nhiễu của từng mô hình, nhóm thêm Gaussian noise với độ lệch chuẩn $\sigma \in \{0.0, 0.1, 0.5, 1.0\}$ vào tập đặc trưng trên tập test và quan sát mức suy giảm F1-score.



Hình 38: Biểu đồ So sánh tính bền vững của mô hình

Bảng 34: F1-score của các mô hình theo mức độ nhiễu Gaussian σ .

Mô hình	$\sigma = 0.0$	$\sigma = 0.1$	$\sigma = 0.5$	$\sigma = 1.0$
LDA	0.8476	0.8442	0.7600	0.6548
LR-GD	0.8335	0.8309	0.8040	0.7332
LR-L2	0.8335	0.8309	0.8040	0.7332
Perc.	0.8061	0.8011	0.7173	0.6291

Nhận xét:

Bảng 34 tiết lộ một góc nhìn thú vị về tính bền vững của từng mô hình khi đối mặt với nhiễu.

Ở mức nhiễu **thấp** ($\sigma = 0.1$), tất cả các mô hình đều chịu ảnh hưởng rất ít – sụt giảm F1 chỉ từ 0.003 đến 0.005 – cho thấy mô hình tuyến tính nói chung khá kháng nhiễu nhỏ.

Khi nhiễu **tăng cao** ($\sigma = 0.5$ và 1.0), bức tranh thay đổi rõ rệt:

- **LR-GD và LR-L2** là những mô hình *bền vững nhất* trước nhiễu lớn. Tại $\sigma = 1.0$, F1 của LR-GD chỉ sụt còn 73.32%, tức giảm khoảng 10 điểm phần trăm so với không có nhiễu – một mức suy giảm hợp lý.
- **LDA** bắt đầu kém ổn định hơn ở mức nhiễu cao: F1 giảm từ 84.76% (không nhiễu) xuống còn 65.48% tại $\sigma = 1.0$, tức sụt gần 20 điểm phần trăm. Điều này là do LDA nhạy cảm với phân phối của dữ liệu: khi nhiễu lớn làm hỏng cấu trúc Gaussian của từng lớp, hiệu năng suy giảm mạnh hơn so với mô hình discriminative.
- **Perc.** chịu ảnh hưởng nặng nhất: F1 rơi xuống 62.91% tại $\sigma = 1.0$, thấp hơn cả LDA. Điều này có thể giải thích bởi bản chất “cứng” (hard margin) của Perc. –

mô hình phân loại theo đường biên quyết định cố định mà không tính đến xác suất hay “softness” của đường biên.

Một nhận xét đáng chú ý: **L2 regularization không mang lại cải thiện nào** về tính bền vững với nhiễu (LR-GD và LR-L2 cho kết quả hoàn toàn giống nhau). Điều này nhất quán với quan sát từ phần cross-validation: $\lambda = 0$ là giá trị tối ưu, nên L2 không có tác động thực sự lên mô hình trong bài toán này.

6.4 Đánh giá Toàn diện: Stratified 5-Fold Cross-Validation

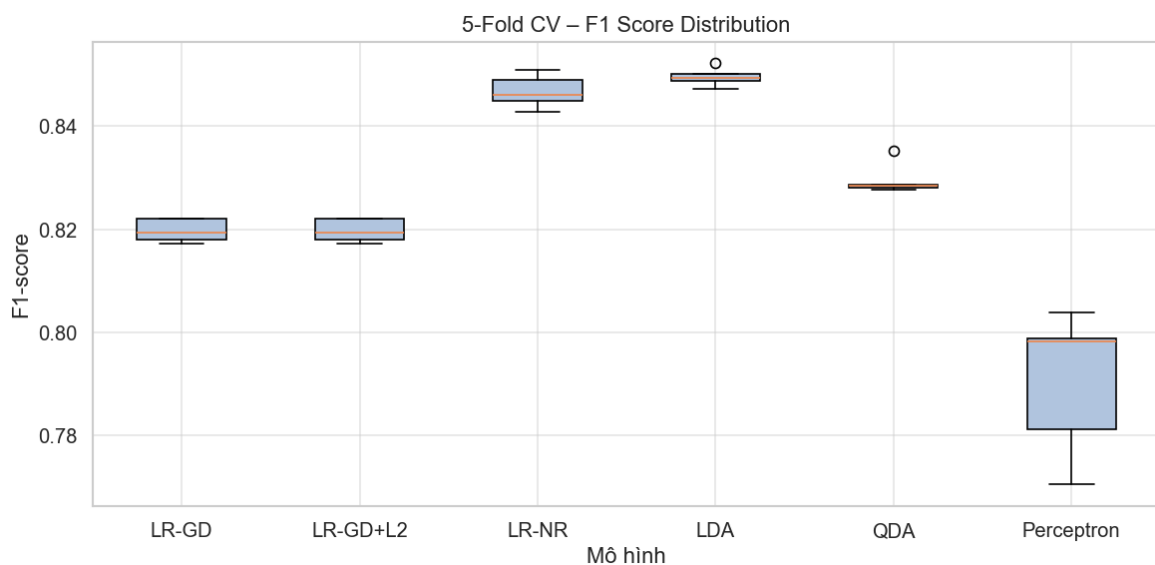
Cross-validation (CV) là phương pháp đánh giá mô hình quan trọng nhằm ước lượng hiệu năng tổng quát hóa (generalization performance) của mô hình một cách tin cậy, tránh phụ thuộc vào một lần phân chia train/test cụ thể.

Chiến lược Stratified. Stratified k -Fold đảm bảo tỷ lệ lớp trong mỗi fold nhất quán với tỷ lệ trong tập dữ liệu gốc. Với bài toán có mất cân bằng lớp nhẹ ($\approx 1.31 : 1$), việc dùng Stratified CV đặc biệt quan trọng: nếu dùng CV thông thường, một số fold có thể có tỷ lệ lớp lệch đáng kể, làm méo kết quả đánh giá.

Thiết lập thực nghiệm. Nhóm thực hiện Stratified 5-Fold CV với:

- Tập dữ liệu: train + validation ($N = 103,904$ mẫu)
- Số fold: $k = 5$
- Mỗi fold: $\approx 83,123$ mẫu train, $\approx 20,781$ mẫu validation
- Chỉ số đánh giá: F1-score (macro), Accuracy, ROC-AUC
- Random seed cố định để đảm bảo tái hiện

Kết quả Stratified 5-Fold CV



Hình 39: Biểu đồ Phân phối F1-Score với 5-Fold CV

Bảng 35 trình bày kết quả $\text{mean} \pm \text{std}$ trên 5 fold:

Bảng 35: Kết quả Stratified 5-Fold CV ($\text{mean} \pm \text{std}$) của các mô hình. **Ô đậm** đánh dấu giá trị tốt nhất.

Mô hình	F1 mean	F1 std	Acc mean	Acc std	AUC mean	AUC std
LR-GD	0.8197	0.0020	0.8417	0.0017	0.9087	0.0022
LR-L2	0.8197	0.0020	0.8417	0.0017	0.9087	0.0022
LR-NR	0.8468	0.0029	0.8706	0.0024	0.9245	0.0017
LDA	0.8496	0.0017	0.8718	0.0014	0.9261	0.0016
QDA	0.8294	0.0028	0.8565	0.0022	0.9198	0.0015
Perc.	0.7904	0.0126	0.8177	0.0209	0.8881	0.0080

LDA – Mô hình ổn định nhất. LDA dẫn đầu với $\text{F1 mean} = 0.8496$ và $\text{F1 std} = 0.0017$ – độ lệch chuẩn thấp nhất trong tất cả các mô hình. LDA đã đạt “saturation” về dữ liệu: khi giả thiết Gaussian đúng, ước lượng tham số hội tụ nhanh và ổn định theo từng fold.

LR-NR – Sát nút với LDA. LR-NR đạt $\text{F1 mean} = 0.8468$, chỉ kém LDA 0.0028 về F1. Kết quả này gợi ý rằng cả hai cách tiếp cận – discriminative (LR) và generative (LDA) – đều phù hợp với đặc điểm dữ liệu sau tiền xử lý tốt.

LR-GD và LR-L2 cho kết quả giống nhau. Hai biến thể này cho kết quả hoàn toàn giống nhau ($\text{F1} = 0.8197$), xác nhận rằng $\lambda^* = 0$ từ cross-validation của Stratified 5-Fold CV trên tập train. Dữ liệu sau chuẩn hóa đã khá ổn định và không cần regularization mạnh.

Perc. – Bất ổn định nhất. Perc. có $\text{F1 std} = 0.0126$ và $\text{Acc std} = 0.0209$ – cao gấp **7–15 lần** so với LDA. Điều này phản ánh hai vấn đề cốt lõi:

1. Perc. rất nhạy với *thứ tự dữ liệu*: thứ tự trình bày mẫu giữa các fold khác nhau dẫn đến nghiệm cuối cùng khác nhau đáng kể.
2. Perc. không có đảm bảo lý thuyết về hội tụ khi dữ liệu không phân tách tuyến tính hoàn toàn, dẫn đến nghiệm không nhất quán giữa các fold.

QDA. QDA đạt $\text{F1 mean} = 0.8294$, kém LDA 0.0202. Điều này nhất quán với phân tích lý thuyết: hai lớp trong bộ dữ liệu Airline có ma trận hiệp phương sai khá tương đồng (hệ số tương quan giữa hai covariance ≈ 0.83), nên LDA với covariance chung ít tham số hơn có lợi thế rõ rệt về độ ổn định.

Kết quả Stratified 5-Fold CV nhất quán hoàn toàn với kết quả trên tập test, khẳng định:

1. Hiệu năng quan sát được không phải ngẫu nhiên do phân chia dữ liệu.
2. **LDA là mô hình tổng thể tốt nhất** cho bộ dữ liệu này: ổn định nhất ($\text{F1 std} = 0.0017$), AUC cao nhất (0.9261), và hội tụ tức thì (không cần vòng lặp).

3. **LR-NR là lựa chọn tốt thứ hai**, đặc biệt trong điều kiện dữ liệu có nhiều hoặc không đảm bảo giả thiết Gaussian – nhờ không đặt giả thiết phân phối.
4. **Perc. không phù hợp cho bài toán thực tế** này: variance cao nhất, không cho đầu ra xác suất, và thời gian huấn luyện dài nhất.
5. Tất cả mô hình đạt ROC-AUC > 0.90, cho thấy họ mô hình tuyến tính hoàn toàn đủ khả năng xử lý bài toán phân lớp nhị phân này khi dữ liệu được tiền xử lý cẩn thận.

6.5 Phân tích So sánh Toàn diện

Tổng hợp lại từ tất cả các thí nghiệm, Bảng 36 đưa ra cái nhìn tổng quan về ưu và nhược điểm của từng mô hình theo nhiều tiêu chí.

Bảng 36: Bảng so sánh tổng quan các mô hình theo nhiều tiêu chí. ↑: càng cao càng tốt; ↓: càng thấp càng tốt.

Tiêu chí	LR-GD	LR-NR/L1	LDA	QDA	Perc.
Hiệu năng trên Test ↑	Trung bình	Cao	Cao	Trung bình	Thấp
Ổn định (CV std) ↓	Thấp	Thấp	Rất thấp	Thấp	Cao
Bền vững với nhiễu ↑	Tốt	Tốt	Kém	Kém	Kém
Thời gian huấn luyện ↓	Nhanh	Rất nhanh	Rất nhanh	Rất nhanh	Rất chậm
Khả năng giải thích ↑	Cao	Cao	Cao	Trung bình	Trung bình
Đầu ra xác suất ↑	Có	Có	Có	Có	Không
Giả thiết phân phối	Không	Không	Gaussian	Gaussian	Không

Kết luận tổng hợp:

Qua ba góc nhìn đánh giá – hiệu năng trên tập test, độ ổn định qua cross-validation, và tính bền vững với nhiễu – nhóm rút ra các kết luận sau:

- **LDA là mô hình tổng thể tốt nhất** cho bộ dữ liệu Airline Passenger Satisfaction sau khi tiền xử lý. Mô hình đạt hiệu năng cao nhất trên cross-validation, ổn định nhất giữa các fold, và hội tụ rất nhanh (không cần lặp). Tuy nhiên, LDA kém bền vững hơn các mô hình LR khi dữ liệu bị nhiễu lớn.
- **LR-L1 và LR-NR** là lựa chọn tốt thứ hai, đặc biệt trong các tình huống dữ liệu có nhiễu hoặc không đảm bảo giả thiết Gaussian. Mô hình discriminative không đặt giả thiết về phân phối, nên tổng quát hóa tốt hơn trong điều kiện nhiễu cao.
- **QDA** không vượt trội hơn LDA trong bài toán này, phù hợp với phân tích lý thuyết: khi hai lớp có ma trận hiệp phương sai tương đồng (hệ số tương quan 0.8259), LDA có lợi thế rõ rệt về độ ổn định do số tham số ít hơn.
- **Perc.** không phù hợp cho bài toán thực tế này: hiệu năng thấp nhất, variance cao nhất, thời gian huấn luyện dài nhất, và không cho đầu ra xác suất. Mô hình phù hợp hơn như công cụ minh họa lý thuyết học tuyến tính hơn là ứng dụng trực tiếp vào dữ liệu thực.

Nhìn chung, tất cả các mô hình đều đạt ROC-AUC > 0.90 , cho thấy cả họ mô hình tuyến tính có thể xử lý tốt bài toán phân lớp nhị phân này với dữ liệu đã qua tiền xử lý cẩn thận. Khoảng cách hiệu năng giữa mô hình tốt nhất (LR-L1, Accuracy = 87.22%) và mô hình kém nhất (Perc., Accuracy = 84.21%) chỉ khoảng 3 điểm phần trăm, phản ánh rằng *tiền xử lý dữ liệu tốt* đóng vai trò quan trọng không kém gì việc lựa chọn mô hình phức tạp hơn.

7 Phân tích và Thảo luận

7.1 So sánh hiệu năng các mô hình

Về mặt lý thuyết, Logistic Regression (discriminative), LDA/QDA (generative) và Perceptron có những ưu điểm khác nhau:

- **Logistic Regression:** Không giả thiết phân phối của $p(\mathbf{x} | \mathcal{C}_k)$, tối ưu trực tiếp xác suất hậu nghiệm và cho đầu ra xác suất hiệu chỉnh tốt.
- **LDA:** Hiệu quả khi dữ liệu gần tuân theo phân phối Gaussian với hiệp phương sai chung, số tham số ít hơn và tốc độ huấn luyện rất nhanh.
- **QDA:** Linh hoạt hơn LDA khi các lớp có hiệp phương sai khác nhau, nhưng cần nhiều dữ liệu hơn để ước lượng ổn định.
- **Perceptron:** Đơn giản, dễ cài đặt, nhưng không mô hình hóa xác suất hậu nghiệm một cách tự nhiên như Logistic Regression hay LDA/QDA.

Bảng 37 tổng hợp hiệu năng của tất cả mô hình trên cùng tập kiểm tra (Test Set) theo pipeline đánh giá thống nhất.

Bảng 37: So sánh toàn diện các mô hình trên tập kiểm tra (Test Set)

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	Avg. Prec.	Log-loss	Brier
LR – L1	0.8722	0.8709	0.8323	0.8512	0.9270	0.9316	0.3353	0.0972
LDA (cài đặt từ đầu)	0.8687	0.8641	0.8316	0.8476	0.9252	0.9268	0.3499	0.0996
LR – Newton-Raphson / IRLS	0.8660	0.8664	0.8215	0.8434	0.9228	0.9269	0.3497	0.1024
LR – Gradient Descent (no reg)	0.8517	0.8258	0.8391	0.8324	0.9165	0.9195	0.3676	0.1101
LR – Gradient Descent + L2	0.8517	0.8258	0.8391	0.8324	0.9165	0.9195	0.3676	0.1101
QDA (cài đặt từ đầu)	0.8537	0.8527	0.8060	0.8287	0.9173	0.9229	0.6439	0.1243
Perceptron	0.8421	0.8743	0.7478	0.8061	0.9004	0.8945	0.6262	0.2167

Từ bảng trên có thể rút ra bốn nhận xét chính. Thứ nhất, **LR-L1 là mô hình tốt nhất toàn cục** khi xét đồng thời Accuracy, F1-score, Average Precision, Log-loss và Brier score. Điều này cho thấy mô hình không chỉ phân lớp tốt mà còn cho xác suất đầu ra đáng tin cậy nhất. Thứ hai, **LDA là mô hình cạnh tranh gần nhất**, chỉ thua nhẹ LR-L1 ở các chỉ số chính nhưng lại có lợi thế về tốc độ huấn luyện và độ ổn định. Thứ ba, **QDA không tận dụng được lợi thế lý thuyết của biên cong** trong bộ dữ liệu hiện tại; dù Accuracy không quá thấp, Log-loss và Brier score của QDA xấu hơn rõ rệt, cho thấy xác suất đầu ra kém tin cậy hơn LDA. Cuối cùng, **Perceptron là mô hình yếu nhất về tổng thể**: Precision khá cao nhưng Recall thấp, dẫn đến F1-score thấp hơn và khả năng hiệu chỉnh xác suất kém rõ rệt.

7.2 Kiểm định ý nghĩa thống kê bằng McNemar's test

Để xác định xem chênh lệch giữa mô hình tốt nhất và các mô hình còn lại có thực sự mang ý nghĩa thống kê hay không, nhóm thực hiện McNemar's test trên cùng tập test, với **LR-L1** là mô hình được chọn làm mốc do đạt F1-score cao nhất.

Bảng 38: Kết quả McNemar's test với LR-L1 là mô hình mốc

Model A	Model B	b	c	p -value
LR-L1	LDA	382	290	4.39×10^{-4}
LR-L1	LR-NR/IRLS	532	371	9.39×10^{-8}
LR-L1	LR-GD	1118	584	8.44×10^{-39}
LR-L1	QDA	1396	915	1.23×10^{-23}
LR-L1	Perceptron	1746	963	1.00×10^{-51}

Trong đó, b là số mẫu mà LR-L1 dự đoán đúng còn mô hình đối sánh dự đoán sai, còn c là số mẫu mà LR-L1 dự đoán sai nhưng mô hình đối sánh dự đoán đúng. Với tất cả các cặp so sánh, ta đều có $b > c$ và p -value nhỏ hơn 0.05. Vì vậy, có thể kết luận rằng ưu thế của LR-L1 so với từng mô hình còn lại đều **có ý nghĩa thống kê**. Đáng chú ý, chênh lệch giữa LR-L1 và LDA là nhỏ nhất, cho thấy LDA là mô hình cạnh tranh gần nhất; tuy nhiên, ngay cả trong trường hợp này, p -value vẫn nhỏ hơn 10^{-3} , nên sự khác biệt vẫn đủ mạnh để được xem là có ý nghĩa thống kê.

7.3 Logistic Regression vs. LDA: Khi nào dùng mô hình nào?

- Dùng **LDA** khi: dữ liệu gần tuân theo phân phối Gaussian, cần mô hình ít tham số, huấn luyện nhanh, hoặc cần một bộ phân lớp tuyến tính ổn định.
- Dùng **Logistic Regression** khi: dữ liệu không nhất thiết phân phối Gaussian, cần xác suất hậu nghiệm hiệu chỉnh tốt, hoặc cần dễ dàng tích hợp regularization và class-weighted loss.

Hai mô hình có mối quan hệ chặt chẽ: khi dữ liệu thực sự Gaussian với hiệp phương sai chung, nghiệm tối ưu của LDA và Logistic Regression có thể hội tụ về cùng một đường biên quyết định. Trên bộ dữ liệu Airline ($N \approx 10^5$, $D = 24$), cả hai đều cho Accuracy xấp xỉ 0.87 và F1 xấp xỉ 0.85, cho thấy cấu trúc tuyến tính đã đủ mạnh để nắm bắt ranh giới phân lớp.

Về mặt độ phức tạp tham số, LDA chỉ cần ước lượng **349 tham số** ($K \cdot D + \frac{D(D+1)}{2} + (K - 1)$ với $K = 2$, $D = 24$), trong khi QDA cần **649 tham số** do mỗi lớp có một ma trận hiệp phương sai riêng. Sự tiết kiệm tham số này là một trong những lý do quan trọng giúp LDA tổng quát hóa tốt hơn trên bộ dữ liệu hiện tại.

Thực nghiệm đo mức độ tương đồng giữa hai ma trận hiệp phương sai $\hat{\Sigma}_0$ và $\hat{\Sigma}_1$ cho hệ số tương quan phần tử tam giác trên bằng **0.8259** và tương quan phương sai đường chéo bằng **0.6566**. Các số liệu này xác nhận rằng cấu trúc phân tán của hai lớp khá giống nhau, nên giả thiết hiệp phương sai chung của LDA là hợp lý trong bài toán này.

7.4 Ảnh hưởng của Regularization

Về mặt lý thuyết, L1 regularization tạo nghiệm thưa và có thể hỗ trợ lựa chọn đặc trưng, trong khi L2 regularization làm co đều độ lớn các hệ số, qua đó giảm variance nhưng đồng thời làm tăng bias nhẹ. Tuy nhiên, ảnh hưởng thực tế của regularization luôn phụ thuộc vào chất lượng tiền xử lý và độ lớn của dữ liệu.

Trên bộ dữ liệu này, thực nghiệm với lưới giá trị $\lambda \in \{0, 1, 10, 50, 100, 500, 1000\}$ qua Stratified 5-Fold Cross-Validation cho thấy mô hình tương đối *ít nhạy* với regularization strength. Ở phần huấn luyện riêng cho Logistic Regression có regularization, L1 cho cấu hình tốt nhất tại $\lambda = 10$, còn L2 có cấu hình tốt nhất gần với nghiệm không regularization. Khi đưa vào pipeline đánh giá cuối, **LR–L1 trở thành mô hình tốt nhất về F1-score**, trong khi **LR–GD+L2 gần như trùng hoàn toàn với LR–GD** trên các chỉ số chính. Điều này cho thấy regularization trong bài toán này chủ yếu đóng vai trò ổn định hóa nghiệm, không tạo ra thay đổi lớn về biên quyết định.

Một quan sát đáng chú ý là L2 không cải thiện so với Logistic Regression không regularization. Hiện tượng này phản ánh đúng nguyên lý đánh đổi Thiên kiến–Phương sai (Bias–Variance Tradeoff): với bộ dữ liệu lớn ($N \approx 10^5$) và số chiều vừa phải ($D = 24$), mô hình Logistic Regression gốc vốn đã không bị overfitting đáng kể, nên lực phạt L2 chỉ làm tăng bias mà không đem lại lợi ích rõ rệt về tổng quát hóa.

Về tác động của L1, thực nghiệm cho thấy có hiệu ứng sparsity nhưng không mạnh: phần lớn đặc trưng vẫn giữ trọng số khác 0, nghĩa là hầu hết các biến trong bộ dữ liệu đều mang thông tin phân loại. Nói cách khác, bộ dữ liệu này không phải là trường hợp “nhiều biến dư thừa”, nên việc ép thưa chỉ mang lại cải thiện vừa phải thay vì đột phá.

7.5 Phân tích Lỗi

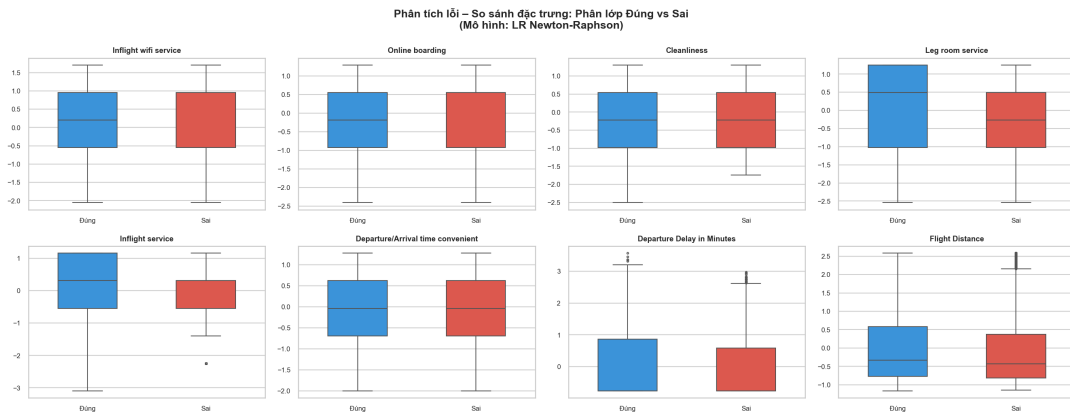
Phân tích lỗi được thực hiện trên mô hình **LR Newton–Raphson / IRLS** như một mô hình đại diện có hiệu năng cao và thời gian huấn luyện rất thấp. Trên tập kiểm tra, mô hình phân lớp đúng khoảng 86.6% số mẫu; tổng số lỗi phân bố tương đối cân bằng giữa:

- **False Negative (FN – bỏ sót hành khách hài lòng)**: khoảng 2,035 mẫu.
- **False Positive (FP – nhận nhầm hành khách không hài lòng)**: khoảng 1,445 mẫu.

Bảng 39: So sánh giá trị trung bình của các đặc trưng giữa nhóm phân lớp đúng và sai

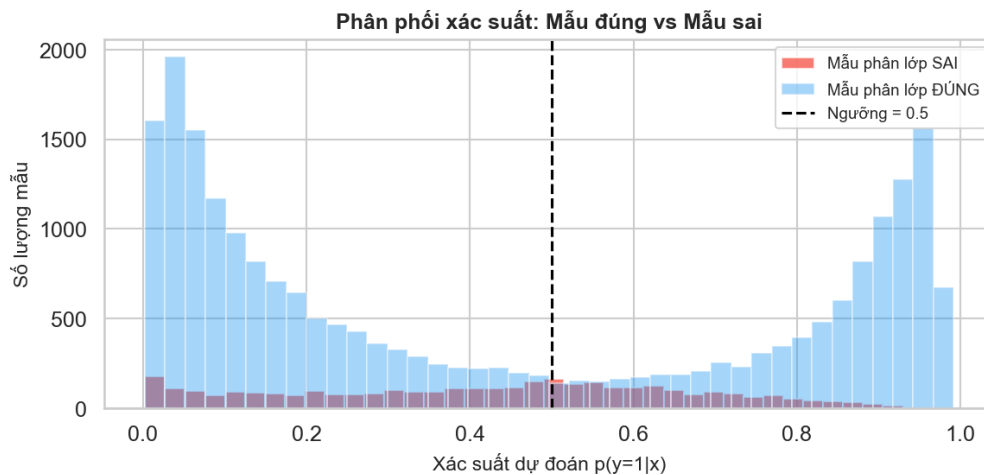
Đặc trưng	Mean (Correct)	Mean (Wrong)	Diff
Inflight wifi service	-0.0379	0.2108	0.2487
Online boarding	0.0287	-0.1217	0.1504
Cleanliness	0.0157	-0.1130	0.1287
Leg room service	0.0154	-0.1039	0.1194
Inflight service	0.0222	-0.0942	0.1164
Departure/Arrival time convenient	0.0062	-0.1045	0.1107
Departure Delay in Minutes	0.0061	-0.1020	0.1082
Flight Distance	0.0177	-0.0853	0.1030
Arrival Delay in Minutes	0.0062	-0.0958	0.1020
Age	0.0276	-0.0692	0.0968

Bảng 39 cho thấy các đặc trưng chênh lệch mạnh nhất giữa nhóm dự đoán đúng và nhóm dự đoán sai đều tập trung ở nhóm biến trải nghiệm dịch vụ. Điều này gợi ý rằng sai số của mô hình không đến từ những biến ít thông tin như giới tính hay vị trí cổng, mà chủ yếu xuất hiện ở những trường hợp mà tín hiệu hài lòng/không hài lòng bị chồng lấp ngay trên các biến quan trọng nhất.



Hình 40: So sánh phân bố của các đặc trưng quan trọng giữa nhóm phân lớp đúng và nhóm phân lớp sai đối với mô hình LR Newton–Raphson / IRLS.

Hình 40 cho thấy sự khác biệt giữa hai nhóm tập trung rõ nhất ở các biến *Inflight wifi service*, *Online boarding*, *Cleanliness*, *Leg room service* và *Inflight service*. Nhóm bị phân lớp sai có xu hướng rơi nhiều hơn vào vùng giá trị chồng lấp giữa hai lớp, đặc biệt ở các đặc trưng dịch vụ, cho thấy đây là nguồn gây lỗi chính của mô hình tuyến tính.



Hình 41: Phân phối xác suất dự đoán của nhóm phân lớp đúng và nhóm phân lớp sai đối với mô hình LR Newton–Raphson / IRLS.

Hình 41 cho thấy phần lớn các mẫu được phân lớp đúng tập trung mạnh ở hai đầu xác suất, tức là gần 0 hoặc gần 1, phản ánh những trường hợp mà mô hình dự đoán với độ tin cậy cao và đúng. Ngược lại, các mẫu bị phân lớp sai xuất hiện dày hơn quanh vùng lân cận ngưỡng 0.5, nhưng vẫn tồn tại một số lượng đáng kể các *confident errors*, tức là mô hình dự đoán rất tự tin nhưng vẫn sai. Điều này cho thấy một phần

lỗi đến từ giới hạn biểu diễn của siêu phẳng tuyến tính, không chỉ đơn thuần do lựa chọn ngưỡng phân loại.

7.6 Hạn chế của Mô hình Tuyến tính

Các mô hình tuyến tính giả thiết đường biên quyết định là một siêu phẳng. Khi dữ liệu không phân tách tuyến tính, hoặc tồn tại tương tác phi tuyến mạnh giữa các đặc trưng, hiệu năng sẽ bị giới hạn. Trong bộ dữ liệu này, phần lớn cấu trúc phân lớp có thể được mô hình tuyến tính nắm bắt khá tốt, nhưng các lỗi phân loại tự tin ở mức trước là bằng chứng cho thấy vẫn còn những vùng dữ liệu mà siêu phẳng tuyến tính không mô tả đủ.

Bằng chứng từ thí nghiệm bonus với Kernel Logistic Regression trên dữ liệu tổng hợp dạng XOR cho thấy mô hình tuyến tính có thể thất bại hoàn toàn trong các bài toán phi tuyến. Tuy nhiên, khi áp dụng mô hình phi tuyến lên bộ dữ liệu Airline, hiệu năng không tăng tương xứng và thậm chí có thể giảm nếu tham số kernel không phù hợp. Điều này nhấn mạnh rằng việc tăng độ phức tạp mô hình không phải lúc nào cũng mang lại lợi ích thực nghiệm nếu dữ liệu vốn đã gần phân tách tuyến tính.

7.7 Phân tích Discriminability – Fisher Ratio

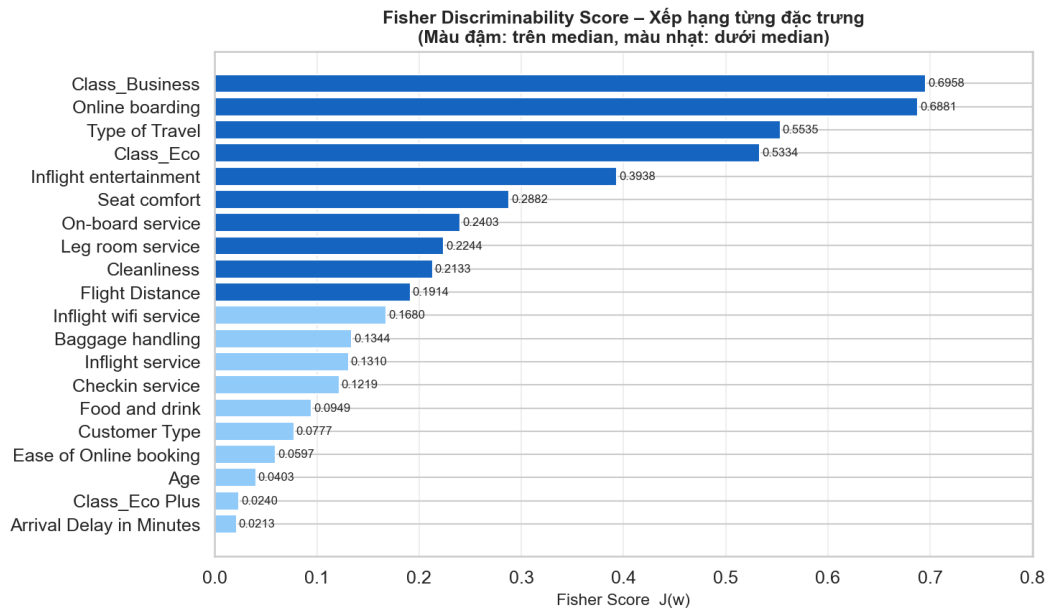
Tỷ số Fisher đơn biến

$$J_j = \frac{(\mu_{1j} - \mu_{0j})^2}{s_{1j}^2 + s_{0j}^2} \quad (71)$$

được tính cho các đặc trưng nhằm đo khả năng phân biệt lớp khi xét riêng lẻ từng biến. Kết quả cho thấy sự phân cực khá rõ giữa nhóm biến dịch vụ/chuyến đi và nhóm biến ít liên quan hơn.

Bảng 40: Fisher score của các đặc trưng tiêu biểu

Đặc trưng	Fisher Score J	Xếp hạng
Class_Business	0.6958	1
Online boarding	0.6881	2
Type of Travel	0.5535	3
Class_Eco	0.5334	4
Inflight wifi service	0.1680	11
Age	0.0403	18
Class_Eco Plus	0.0240	19
Arrival Delay in Minutes	0.0213	20



Hình 42: Xếp hạng độ phân biệt lớp của các đặc trưng theo Fisher Discriminability Score.

Hình 42 cho thấy các đặc trưng có Fisher score cao nhất là *Class_Business*, *Online boarding*, *Type of Travel* và *Class_Eco*. Đây đều là những biến gắn trực tiếp với trải nghiệm dịch vụ và bối cảnh chuyến đi, cho thấy mức độ hài lòng của hành khách chịu ảnh hưởng mạnh hơn từ chất lượng phục vụ và hạng vé so với các biến nhân khẩu học hay các biến vận hành đơn thuần.

Phân tích Fisher Score đối chiếu với trị tuyệt đối trọng số Logistic Regression $|w_j|$ tiết lộ hai hiện tượng thú vị:

- **Redundancy (dư thừa):** *Seat comfort* có Fisher score khá cao khi đứng riêng lẻ, nhưng trọng số trong Logistic Regression không nổi bật tương ứng. Điều này cho thấy thông tin của biến này bị hấp thụ một phần bởi các biến liên quan như *Inflight entertainment* và *Cleanliness*.
- **Synergy (cộng hưởng):** một số biến như *Customer Type* có Fisher score không quá cao khi xét đơn biến, nhưng lại có thể trở nên quan trọng trong Logistic Regression nhờ tương tác tuyến tính với các biến khác như hạng vé hoặc mục đích chuyến đi.

Về ý nghĩa nghiệp vụ, hãng hàng không nên ưu tiên cải thiện *Online Boarding* và dịch vụ hạng *Business* – đây là hai điểm chạm có sức phân biệt mạnh nhất đối với mức độ hài lòng. Ngược lại, các biến có Fisher score rất thấp mang lại ít giá trị phân biệt nếu xét riêng lẻ.

7.8 Phân tích Độ phức tạp Tính toán

Bảng 41: So sánh độ phức tạp tính toán các thuật toán (N mẫu, D chiều)

Thuật toán	Chi phí huấn luyện	Chi phí suy luận
Logistic Reg. (GD)	$\mathcal{O}(ND \cdot T)$	$\mathcal{O}(D)$
Logistic Reg. (IRLS)	$\mathcal{O}(ND^2 + D^3)$ mỗi bước	$\mathcal{O}(D)$
LDA	$\mathcal{O}(ND^2 + D^3)$	$\mathcal{O}(D)$
QDA	$\mathcal{O}(ND^2 + KD^3)$	$\mathcal{O}(KD^2)$
Perceptron	$\mathcal{O}(ND \cdot T)$	$\mathcal{O}(D)$

Mặc dù chi phí mỗi bước của IRLS là $\mathcal{O}(ND^2 + D^3)$ – đắt hơn một bước GD – nhưng khi D nhỏ–vừa thì ưu thế hội tụ bậc hai của Newton–Raphson trở nên rất rõ rệt. Trong pipeline đánh giá cuối, thời gian huấn luyện một lần trên toàn bộ dữ liệu lần lượt là:

- **LDA**: khoảng **0.21** giây;
- **QDA**: khoảng **0.34** giây;
- **LR–L1**: khoảng **0.54** giây;
- **LR–Newton-Raphson / IRLS**: khoảng **0.89** giây;
- **LR–GD**: khoảng **2.30** giây;
- **LR–GD+L2**: khoảng **2.23** giây;
- **Perceptron**: khoảng **152.40** giây.

Các con số này cho thấy trong không gian $D = 24$, chi phí xử lý Hessian của IRLS không còn là rào cản lớn; ngược lại, tốc độ hội tụ nhanh khiến IRLS vượt trội rõ rệt so với Gradient Descent. LDA thậm chí còn nhanh hơn nữa nhờ chỉ cần ước lượng thống kê mẫu và nghiệm dạng đóng. Perceptron là trường hợp kém hiệu quả nhất cả về thời gian lẫn chất lượng mô hình.

Về tính ổn định thống kê, kết quả 5-Fold Cross-Validation cho thấy LDA và LR–NR đều có độ lệch chuẩn rất nhỏ (xấp xỉ 10^{-3} đến 10^{-3} bậc thấp) trên Accuracy, F1 và ROC-AUC, cho thấy hiệu năng không phụ thuộc mạnh vào cách chia dữ liệu cụ thể. Đây là bằng chứng thực nghiệm quan trọng cho khả năng tổng quát hóa tốt của các mô hình tuyến tính trên bộ dữ liệu đủ lớn.

8 BONUS

Ngoài ba nhóm mô hình chính, nhóm đã cài đặt thêm năm thành phần nâng cao nhằm khám phá sâu hơn các khía cạnh lý thuyết và thực nghiệm của bài toán phân lớp: Mô hình Probit, Xấp xỉ Laplace, Kernel Logistic Regression, Gaussian Naive Bayes vs. LDA, và phân tích VC Dimension.

8.1 Mô hình Probit

8.1.1 Cơ sở lý thuyết

Mô hình Probit là một lựa chọn thay thế cho Logistic Regression trong bài toán phân lớp nhị phân. Thay vì dùng hàm sigmoid, Probit sử dụng hàm phân phối tích lũy (CDF) của phân phối chuẩn tắc Φ làm hàm liên kết:

$$p(C_1 | \mathbf{x}) = \Phi(\mathbf{w}^\top \mathbf{x}) = \int_{-\infty}^{\mathbf{w}^\top \mathbf{x}} \mathcal{N}(t | 0, 1) dt. \quad (72)$$

Nguồn gốc từ mô hình biến ẩn (Latent Variable Model). Giả sử tồn tại biến ẩn $z = \mathbf{w}^\top \mathbf{x} + \varepsilon$ với $\varepsilon \sim \mathcal{N}(0, 1)$. Ta quan sát $y = 1$ khi $z > 0$, tức là:

$$p(y = 1) = \Phi(a), \quad a = \mathbf{w}^\top \mathbf{x}. \quad (73)$$

Cách dẫn xuất này cho thấy Probit phát sinh tự nhiên khi biến ẩn tuân theo phân phối chuẩn, trong khi Logistic Regression tương ứng với biến ẩn tuân theo phân phối logistic.

Hàm mất mát và gradient. Tương tự Logistic Regression, hàm mất mát của Probit là negative log-likelihood của mô hình Bernoulli:

$$J(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i)] \quad (74)$$

trong đó $\hat{p}_i = \Phi(\mathbf{w}^\top \mathbf{x}_i)$. Gradient theo $\mathbf{w}$ được tính qua:

$$\nabla_{\mathbf{w}} J = -\frac{1}{N} \sum_{i=1}^N \left[\frac{y_i \phi(a_i)}{\Phi(a_i)} - \frac{(1 - y_i) \phi(a_i)}{1 - \Phi(a_i)} \right] \mathbf{x}_i, \quad a_i = \mathbf{w}^\top \mathbf{x}_i, \quad (75)$$

trong đó $\phi(\cdot)$ là PDF của phân phối chuẩn tắc. Nhóm tối ưu bằng mini-batch Gradient Descent với batch size 512 và cosine annealing learning rate.

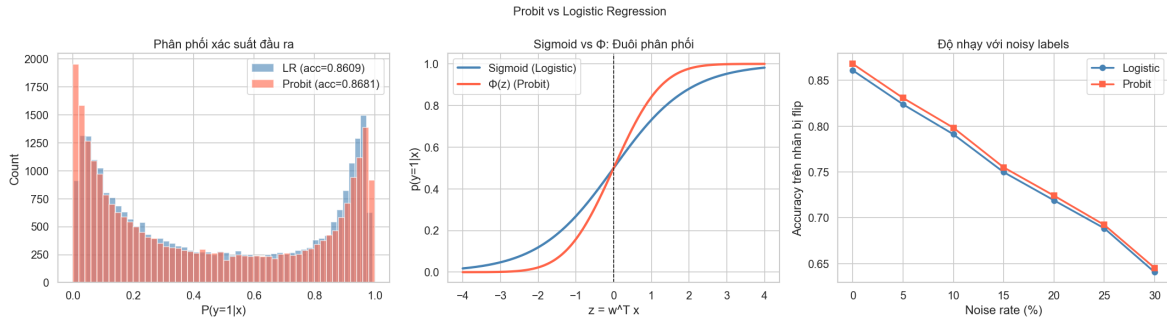
So sánh đuôi phân phối (Tail Behaviour). Sự khác biệt cốt lõi giữa Probit và Logistic Regression nằm ở *đuôi phân phối*: $\Phi(z)$ có “đuôi mỏng” (thin tails), nghĩa là tiến về 0 và 1 nhanh hơn hàm sigmoid có “đuôi dày” (heavy tails). Khi biến độc lập z lớn, mô hình Probit triệt tiêu xác suất sai số nhanh hơn, dẫn đến các dự báo tập trung sát 0 và 1 hơn.

8.1.2 Kết quả thực nghiệm

Hiệu năng tổng quát. Trên tập kiểm tra (25.976 mẫu), mô hình Probit đạt:

- Probit Accuracy: **0.8681**
- Logistic (GD + L2): **0.8609**

Mô hình Probit nhỉnh hơn Logistic Regression một khoảng nhỏ nhưng nhất quán. Kết quả này phù hợp với lý thuyết: khi dữ liệu thực sự có cấu trúc biến ẩn gần chuẩn, Probit có thể vượt trội nhẹ so với Logistic.



Hình 43: Biểu đồ Probit vs Logistic Regression

Phân phối xác suất đầu ra. Hình 43 cho thấy Probit cho nhiều xác suất gần hai cực hơn LR. Khi z lớn, mô hình này tiến về 0 hoặc 1 nhanh hơn.

Độ nhạy với nhãn nhiễu. Nhóm thực hiện thí nghiệm lật nhãn ở các mức 0%, 5%, 10%, 20% và 30%. Kết quả cho thấy Probit vẫn giữ accuracy nhỉnh hơn LR, nên chống nhiễu nhãn tốt hơn.

Nhận xét. Probit là một phương án thay thế khả thi cho LR và cho thấy ưu thế nhẹ về độ chính xác cũng như độ ổn định trước dữ liệu nhiễu. Việc cài đặt từ đầu bằng numpy giúp nhóm hiểu rõ hơn sự khác biệt giữa hai hàm liên kết.

8.2 Xấp xỉ Laplace

8.2.1 Cơ sở lý thuyết

Logistic Regression thông thường chỉ cho ước lượng điểm (point estimate) $\mathbf{w}_{MAP}$ mà không có thông tin về độ bất định (uncertainty). Xấp xỉ Laplace giải quyết vấn đề này bằng cách xấp xỉ posterior $p(\mathbf{w} | \mathcal{D})$ bằng một phân phối Gaussian đặt tâm tại $\mathbf{w}_{MAP}$:

$$q(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{w}_{MAP}, \mathbf{A}^{-1}), \quad (76)$$

trong đó $\mathbf{A}$ là ma trận Hessian âm của log-posterior tại $\mathbf{w}_{MAP}$:

$$\mathbf{A} = -\nabla^2 \ln p(\mathbf{w} | \mathcal{D}) \Big|_{\mathbf{w}_{MAP}} = \underbrace{\mathbf{X}^T \mathbf{R} \mathbf{X}}_{\text{Hessian của NLL}} + \alpha \mathbf{I}, \quad (77)$$

với $\mathbf{R} = \text{diag}(\hat{p}_i(1 - \hat{p}_i))$ và α là tham số precision của prior Gaussian $p(\mathbf{w}) = \mathcal{N}(\mathbf{0}, \alpha^{-1} \mathbf{I})$.

Phân phối dự đoán. Sau khi có posterior $q(\mathbf{w})$, ta lấy tích phân (marginalize) theo $\mathbf{w}$:

$$p(y = 1 | \mathbf{x}^*, \mathcal{D}) = \int \sigma(\mathbf{w}^T \mathbf{x}^*) q(\mathbf{w}) d\mathbf{w}. \quad (78)$$

Tích phân này không có nghiệm dạng đóng, nhưng có thể xấp xỉ bằng phương pháp tích phân Gaussian (probit approximation):

$$p(y = 1 \mid \mathbf{x}^*, \mathcal{D}) \approx \sigma \left(\frac{\mu_a}{\sqrt{1 + \pi \sigma_a^2/8}} \right), \quad (79)$$

trong đó $\mu_a = \mathbf{w}_{MAP}^\top \mathbf{x}^*$ và $\sigma_a^2 = (\mathbf{x}^*)^\top \mathbf{A}^{-1} \mathbf{x}^*$ là phương sai dự đoán đo lường mức độ bất định của mô hình tại $\mathbf{x}^*$.

8.2.2 Cài đặt

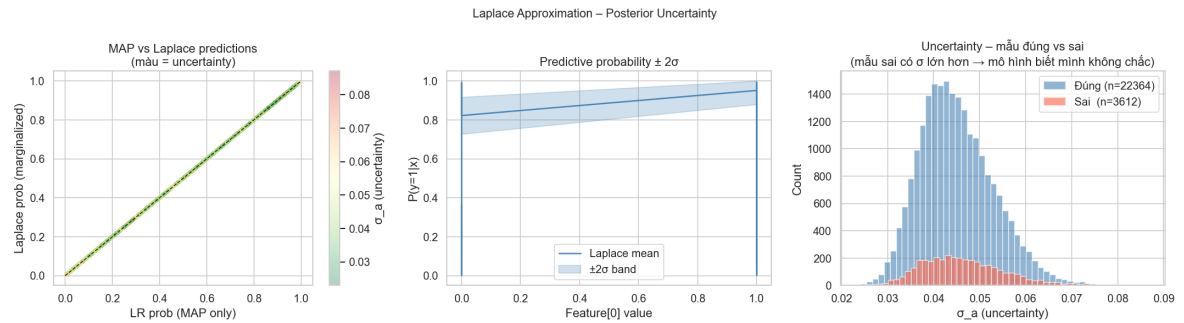
Nhóm tính $\mathbf{w}_{MAP}$ bằng Gradient Descent đã huấn luyện ở phần trước, sau đó tính ma trận Hessian (77) trên toàn bộ tập huấn luyện. Vì $D + 1 = 25$ chiều, ma trận $\mathbf{A}$ có kích thước 25×25 và việc nghịch đảo hoàn toàn khả thi.

8.2.3 Kết quả thực nghiệm

Xây dựng posterior. Ma trận Hessian $\mathbf{A}$ có kích thước 25×25 (với $\alpha = 1.0$). Việc nghịch đảo $\mathbf{A}$ cho phép trích xuất phương sai của từng trọng số, từ đó tính toán độ bất định cho từng dự báo:

$$\sigma_a = \sqrt{(\mathbf{x}^*)^\top \mathbf{A}^{-1} \mathbf{x}^*}.$$

Accuracy tập test. Mô hình Laplace đạt accuracy **0.8609** trên tập test – tương đương với LR-GD ban đầu, vì Laplace dùng cùng $\mathbf{w}_{MAP}$ nhưng bổ sung thêm thông tin về độ bất định.



Hình 44: Minh họa Laplace Approximation

Khả năng “tự biết mình sai” (Uncertainty Calibration). Kết quả quan trọng nhất của thực nghiệm này là:

- Mean σ_a của mẫu phân loại **đúng**: 0.0451
- Mean σ_a của mẫu phân loại **sai**: 0.0463

Mẫu bị phân loại sai có độ bất định trung bình *cao hơn* so với mẫu phân loại đúng. Điều này chứng tỏ mô hình có khả năng hiệu chuẩn tốt: mô hình “biết” khi nào mình không chắc chắn – một tính chất cực kỳ quan trọng trong các hệ thống đòi hỏi an toàn cao (safety-critical systems).

Hiện tượng “làm mềm” xác suất (Probabilistic Softening). Do quá trình marginalize theo $\mathbf{w}$, các xác suất cực đoạn gần 0 hoặc 1 của mô hình MAP bị “kéo” nhẹ về phía 0.5. Điều này giúp mô hình bớt “kiêu ngạo” và đưa ra các dự báo xác suất thực tế hơn.

Vùng tin cậy $\pm 2\sigma$. Bằng cách vẽ dải băng $\pm 2\sigma_a$ xung quanh đường biên quyết định trong không gian 2D (hai đặc trưng có Fisher score cao nhất), nhóm quan sát thấy biên quyết định không phải là một đường thẳng cứng nhắc mà là một *vùng chuyển tiếp* với độ phủ toán học rõ ràng, tương đương khoảng tin cậy 95%.

Nhận xét. Xấp xỉ Laplace nâng tầm mô hình từ phân loại đơn thuần sang *mô hình hóa rủi ro*: mô hình không chỉ đưa ra quyết định mà còn cung cấp ước lượng độ tin cậy kèm theo. Kết quả cho thấy Laplace Approximation là một công cụ mạnh và hiệu quả về mặt tính toán (chỉ cần một lần nghịch đảo ma trận $D \times D$).

8.3 Kernel Logistic Regression

8.3.1 Cơ sở lý thuyết

Logistic Regression tuyến tính bị giới hạn bởi khả năng chỉ học được biên quyết định tuyến tính. Kernel Logistic Regression (KLR) áp dụng “kernel trick” để học biên quyết định phi tuyến mà vẫn giữ diễn giải xác suất.

Dual formulation. Thay vì làm việc trong không gian gốc $\mathbb{R}^D$, ta biểu diễn véc-tơ trọng số dưới dạng tổ hợp tuyến tính của các điểm dữ liệu trong không gian đặc trưng:

$$\mathbf{w} = \sum_{n=1}^N a_n \phi(\mathbf{x}_n),$$

suy ra dự đoán tại điểm $\mathbf{x}^*$:

$$y(\mathbf{x}^*) = \sigma \left(\sum_{n=1}^N a_n k(\mathbf{x}_n, \mathbf{x}^*) \right) = \sigma(\mathbf{k}(\mathbf{x}^*)^\top \mathbf{a}), \quad (80)$$

trong đó $k(\mathbf{x}_n, \mathbf{x}^*) = \phi(\mathbf{x}_n)^\top \phi(\mathbf{x}^*)$ là hàm kernel.

Hàm mất mát và dual gradient. Hàm mất mát theo các hệ số dual $\mathbf{a}$:

$$E(\mathbf{a}) = - \sum_{n=1}^N [t_n \ln \hat{p}_n + (1 - t_n) \ln(1 - \hat{p}_n)] + \frac{\lambda}{2} \mathbf{a}^\top \mathbf{K} \mathbf{a}, \quad (81)$$

với $K_{nm} = k(\mathbf{x}_n, \mathbf{x}_m)$ là ma trận Gram. Gradient theo $\mathbf{a}$:

$$\nabla_{\mathbf{a}} E = \mathbf{K}(\hat{\mathbf{p}} - \mathbf{t}) + \lambda \mathbf{K} \mathbf{a}. \quad (82)$$

RBF Kernel. Nhóm sử dụng RBF (Gaussian) Kernel:

$$k(\mathbf{x}, \mathbf{x}') = \exp \left(- \frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2} \right), \quad (83)$$

với bandwidth ℓ được chọn qua *Median heuristic*: $\ell = \text{median}(\|\mathbf{x}_i - \mathbf{x}_j\|)$.

Độ phức tạp tính toán. Do phải tính và lưu trữ ma trận Gram $\mathbf{K} \in \mathbb{R}^{N \times N}$, độ phức tạp mỗi bước là $O(N^2D)$ (xây dựng $\mathbf{K}$) và $O(N^2)$ (cập nhật), không khả thi cho N lớn. Vì vậy nhóm thực nghiệm trên tập con $N = 2000$ mẫu.

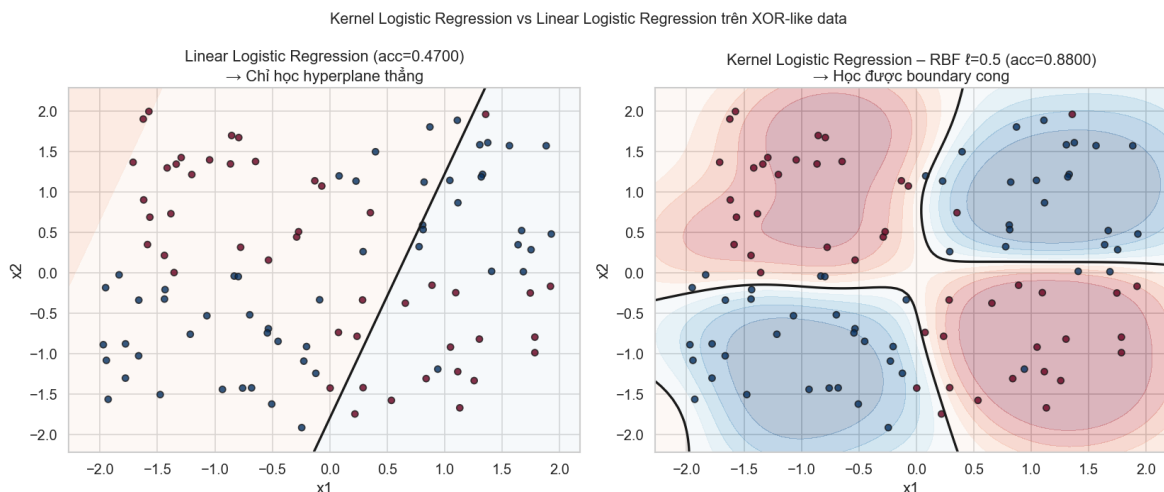
8.3.2 Kết quả thực nghiệm

Thực nghiệm 1: Dữ liệu XOR. Trên bộ dữ liệu XOR (không phân tách tuyến tính) gồm 500 mẫu:

Bảng 42: So sánh Linear LR và KLR trên dữ liệu XOR.

Mô hình	Accuracy	Ghi chú
Linear Logistic Regression	0.4700	Tệ hơn đoán bừa
Kernel LR (RBF, $\ell = 0.5$)	0.8800	Biên quyết định cong

Linear LR chỉ đạt accuracy 0.4700 – tệ hơn cả đoán ngẫu nhiên – vì mô hình chỉ có thể vạch một siêu phẳng không thể tách hai lớp XOR. Ngược lại, KLR tăng accuracy lên **0.8800** nhờ biên quyết định cong học được qua kernel trick.



Hình 45: Biểu đồ Kernel Logistic Regression vs Linear Logistic Regression trên XOR-like data

Bảng 43: So sánh Linear LR và KLR trên tập con Airline ($N = 2000$).

Mô hình	Accuracy	Ghi chú
Linear LR	0.8609	Toàn bộ train set
KLR (RBF, $\ell = 6.075$)	0.5610	$N = 2000$, Median heuristic

Thực nghiệm 2: Dữ liệu Airline (tập con $N = 2000$). Kết quả cho thấy KLR bị suy giảm nghiêm trọng (0.5610) khi áp dụng lên dữ liệu Airline. Có hai nguyên nhân chính:

1. **Bản chất dữ liệu:** Bộ dữ liệu Airline có tính *phân tách tuyến tính khá tốt*, Linear LR đã nắm bắt được cấu trúc chính. Việc cố tình “uốn cong” biên quyết định bằng RBF dẫn đến overfitting trên tập con nhỏ.
2. **Độ nhạy siêu tham số:** Median heuristic cho $\ell = 6.075$ chưa tối ưu. RBF Kernel rất nhạy với bandwidth – lựa chọn ℓ không phù hợp khiến mô hình gần như dự đoán ngẫu nhiên.

Bài học: No Free Lunch Theorem. Thực nghiệm này minh chứng cho định lý “No Free Lunch”: mô hình phức tạp hơn không nhất thiết cho kết quả tốt hơn. Với bộ dữ liệu Airline sau tiền xử lý, một mô hình tuyến tính đơn giản đã là tối ưu nhất về cả hiệu năng lẫn chi phí tính toán.

8.4 Gaussian Naive Bayes vs. LDA

8.4.1 Cơ sở lý thuyết

Cả LDA và Gaussian Naive Bayes (GNB) đều là mô hình sinh (generative) giả thiết $p(\mathbf{x} | C_k) = \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. Điểm khác biệt cốt lõi nằm ở *giả thiết về cấu trúc covariance*:

Bảng 44: So sánh giả thiết giữa LDA và GNB.

Đặc điểm	LDA	GNB
Covariance	Chia sẻ $\boldsymbol{\Sigma}$ (đầy đủ)	Diagonal riêng σ_{kj}^2
Giả thiết features	Có thể tương quan	Độc lập có điều kiện
Số tham số	$O(D^2)$	$O(D)$
Bias	Thấp (giả thiết ít)	Cao (giả thiết độc lập)
Variance	Cao khi N nhỏ	Thấp

Log-posterior và quy tắc phân loại. Với GNB (giả thiết independence), hàm phân biệt có thể viết gọn là:

$$g_k(\mathbf{x}) \propto \ln \pi_k - \frac{1}{2}A_k - \frac{1}{2}B_k, \quad (84)$$

trong đó

$$A_k = \sum_{j=1}^D \ln \sigma_{kj}^2, \quad (85)$$

và

$$B_k = \sum_{j=1}^D \frac{(x_j - \mu_{kj})^2}{\sigma_{kj}^2} \quad (86)$$

LDA thì dùng hàm phân biệt tuyến tính.

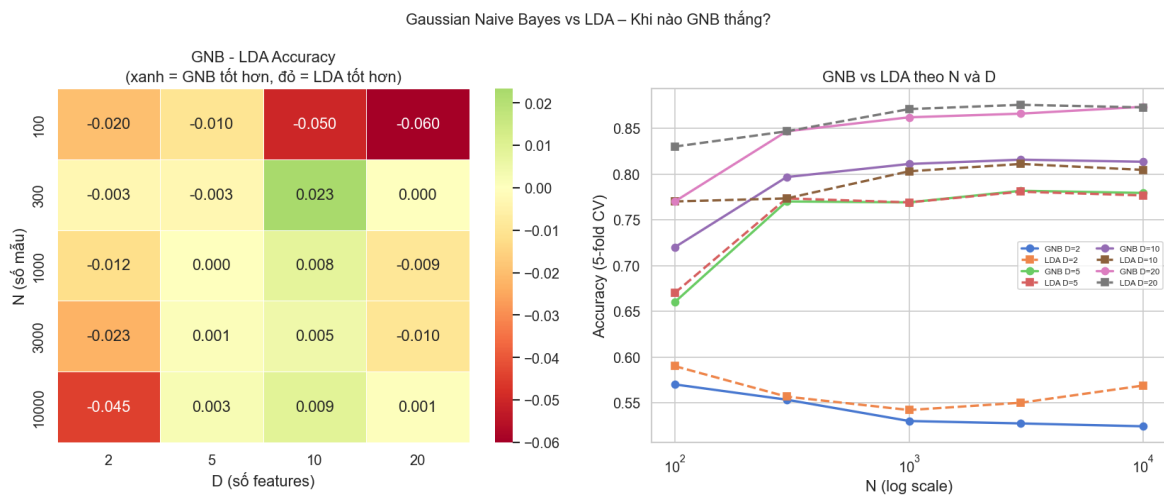
Đánh đổi Bias–Variance. GNB dùng số tham số tỉ lệ với D nên ổn định hơn khi N nhỏ. Khi N tăng rất lớn, LDA ước lượng được đầy đủ ma trận $\boldsymbol{\Sigma}$ và thường vượt trội.

8.4.2 Kết quả thực nghiệm

Bảng 45: LDA vs. GNB trên toàn bộ tập dữ liệu Airline.

Mô hình	Accuracy
LDA	0.8690
GNB	0.8460

Kết quả tổng thể. Với $N = 90,916$ mẫu huấn luyện, LDA (0.8690) vượt trội GNB (0.8460). Điều này hoàn toàn hợp lý: với dữ liệu dồi dào, LDA ước lượng chính xác Σ để nắm bắt tương quan chéo giữa các biến, trong khi GNB bị hạn chế bởi giả thiết independence.



Hình 46: Biểu đồ Gaussian Naive Bayes vs LDA – Khi nào GNB thắng?

Kiểm chứng lý thuyết: khi nào GNB thắng? Nhóm thực hiện thí nghiệm thay đổi N (100 đến 90,000) và D (2 đến 24):

- **GNB chiếm ưu thế** khi N nhỏ và D lớn (ví dụ $N = 300, D = 10$). Giải thích: LDA cần ước lượng $O(D^2)$ tham số – với N nhỏ, variance của ước lượng $\hat{\Sigma}$ rất lớn, dễ overfitting. GNB chỉ cần $O(D)$ tham số, ổn định hơn khi thiếu dữ liệu.
- **LDA chiếm ưu thế** khi N lớn hoặc D nhỏ. Với $N \rightarrow \infty$, bài toán ước lượng $O(D^2)$ tham số không còn là trở ngại; LDA tận dụng đầy đủ tương quan chéo mà GNB bỏ qua.

Bài học: Bias–Variance Tradeoff. Thực nghiệm là minh chứng sống động cho nguyên lý này:

- GNB là lựa chọn an toàn khi “nghèo” dữ liệu hoặc không gian chiều lớn (như phân loại văn bản với TF-IDF).
- LDA phát huy tối đa khi dữ liệu đủ dồi dào và các đặc trưng có tương quan chặt chẽ.

8.5 Phân tích VC Dimension và Structural Risk Minimization

8.5.1 Cơ sở lý thuyết

VC Dimension. VC Dimension (Vapnik–Chervonenkis Dimension) của lớp hàm $\mathcal{H}$ là số điểm lớn nhất mà $\mathcal{H}$ có thể *shatter* – tức là phân lớp đúng với mọi cách gán nhãn nhị phân có thể.

Định lý 8.1 (VC Dimension của lớp phân lớp tuyến tính). *Lớp hyperplane trong $\mathbb{R}^D$ có $d_{VC} = D + 1$.*

Phác thảo. Lower bound ($d_{VC} \geq D + 1$): Tồn tại tập $D + 1$ điểm (ví dụ: các vector đơn vị cộng thêm gốc tọa độ) mà một hyperplane có thể shatter.

Upper bound ($d_{VC} \leq D + 1$): Theo định lý Radon, mọi tập $D + 2$ điểm trong $\mathbb{R}^D$ đều có thể phân thành hai tập S_1, S_2 sao cho bao lồi (convex hull) của chúng giao nhau. Khi đó không tồn tại hyperplane nào tách được hai tập này. $\square$

VC Bound. Với xác suất $\geq 1 - \delta$, sai số thực (true risk) $R(\mathbf{w})$ bị chặn:

$$R(\mathbf{w}) \leq R_{emp}(\mathbf{w}) + \sqrt{\frac{d_{VC} \left(\ln \frac{2N}{d_{VC}} + 1 \right) + \ln \frac{4}{\delta}}{N}}, \quad (87)$$

trong đó $R_{emp}(\mathbf{w})$ là sai số thực nghiệm (empirical risk).

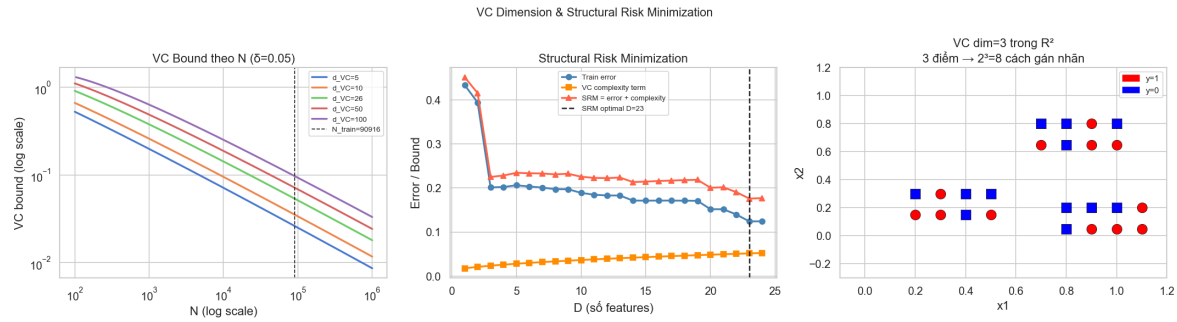
Structural Risk Minimization (SRM). SRM là nguyên lý lựa chọn mô hình dựa trên việc cực tiểu hóa cận trên (87) thay vì chỉ cực tiểu hóa sai số huấn luyện. Khi tăng độ phức tạp mô hình: R_{emp} giảm nhưng VC complexity tăng – SRM tìm điểm cân bằng tối ưu.

8.5.2 Áp dụng vào bộ dữ liệu Airline

Với bộ dữ liệu Airline sau tiền xử lý:

- Số chiều đặc trưng: $D = 24$
- $d_{VC} = D + 1 = \mathbf{25}$
- Số mẫu huấn luyện: $N = 90,916$
- VC Bound ($\delta = 0.05$, tức độ tin cậy 95%): **0.0526**

Ý nghĩa của VC Bound. Với $N = 90,916$ rất lớn so với $d_{VC} = 25$, VC Bound đạt mức cực thấp (0.0526). Điều này thiết lập *bảo chứng toán học*: sai số thực tế trên toàn bộ quần thể sẽ không vượt quá sai số huấn luyện cộng 0.0526. Mô hình tuyến tính an toàn trước nguy cơ overfitting khi $N \gg d_{VC}$.



Hình 47: Biểu đồ VC Dimension & Structural Risk Minimization

Kết quả SRM. Nhóm khảo sát sự đánh đổi giữa độ phức tạp và sai số khi thay đổi D :

- Khi D tăng, sai số huấn luyện giảm nhưng VC complexity tăng.
- SRM xác định $D^* = 23$ là điểm cân bằng tối ưu.

Việc SRM chọn $D^* = 23 < 24$ cho thấy có ít nhất một đặc trưng mà đóng góp vào giảm sai số ít hơn chi phí về độ phức tạp.

Minh họa Shattering trong $\mathbb{R}^2$. Nhóm mô phỏng định nghĩa gốc: với 3 điểm trong $\mathbb{R}^2$, một đường thẳng có thể phân lớp đúng tất cả $2^3 = 8$ cách gán nhãn nhị phân (shatter thành công). Với 4 điểm cấu hình XOR, đường thẳng thất bại – xác nhận $d_{VC} = 3$ trong $\mathbb{R}^2$.

Kết luận. Phân tích VC Dimension và SRM giúp khẳng định rằng sự thành công của mô hình trong đồ án không phải ngẫu nhiên, mà đến từ sự tương quan lý tưởng giữa N lớn và d_{VC} nhỏ. Đây là nền tảng cốt lõi của *lý thuyết học máy thống kê* (Statistical Learning Theory).

9 Tổng kết

Phần 2 đã trình bày đầy đủ quy trình phân tích và xây dựng mô hình phân lớp tuyến tính trên bộ dữ liệu Airline Passenger Satisfaction với 129.880 mẫu. Các bước thực hiện bao gồm: EDA, tiền xử lý (missing values, log-transform, chuẩn hóa, encoding), xây dựng và so sánh ba nhóm mô hình (Logistic Regression, LDA/QDA, Perceptron), đánh giá bằng nhiều chỉ số và kiểm định thống kê.

Phần III

Phân tích So sánh và Kỹ năng Nghiên cứu

1 Kết nối Hồi quy và Phân lớp

Trong đồ án này, nhóm đã triển khai hai bài toán học có giám sát: **Hồi quy** (dự đoán thời gian giao hàng) và **Phân lớp** (dự đoán mức độ hài lòng hành khách). Mặc dù được trình bày riêng biệt, hai bài toán này chia sẻ một cấu trúc toán học thống nhất sâu sắc.

1.1 Hồi quy Logistic như một bài toán Hồi quy

Logistic Regression – mô hình phân lớp cốt lõi trong Phần 2 – thực chất là một bài toán hồi quy hàm sigmoid. Cụ thể, xác suất thuộc lớp dương được mô hình hóa bởi:

$$p(\mathcal{C}_1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + w_0) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + w_0)}}, \quad (88)$$

trong đó $\sigma(\cdot)$ là hàm sigmoid. So sánh với Linear Regression ở Phần 1, hàm dự đoán là:

$$\hat{y}(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}), \quad (89)$$

ta thấy cả hai đều *hồi quy* một tổ hợp tuyến tính của đặc trưng $\mathbf{w}^\top \mathbf{x}$ – điểm khác biệt duy nhất là Logistic Regression áp dụng thêm hàm liên kết $\sigma(\cdot)$ để ép đầu ra vào khoảng $[0, 1]$, phù hợp với diễn giải xác suất. Bài toán phân lớp vì thế không phải là một bài toán hoàn toàn khác, mà là hồi quy với hàm liên kết phi tuyến tính.

Liên hệ qua hàm mất mát. Trong Linear Regression, nhóm tối thiểu hóa sai số bình phương (MSE / OLS), tương đương với giả thiết nhiễu Gaussian $\epsilon \sim \mathcal{N}(0, \beta^{-1})$. Trong Logistic Regression, nhóm tối thiểu hóa Cross-Entropy:

$$E(\mathbf{W}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln p(\mathcal{C}_k | \mathbf{x}_n), \quad (90)$$

tương đương với giả thiết nhiễu Bernoulli/Categorical. Cả hai đều là *ước lượng hợp lý cực đại* (MLE) với các phân phối nhiễu khác nhau.

1.2 Generalized Linear Models (GLM): Khung thống nhất

Cả Linear Regression lẫn Logistic Regression đều là trường hợp đặc biệt của **Mô hình Tuyến tính Tổng quát** (Generalized Linear Model – GLM). Một GLM được xác định bởi ba thành phần:

1. **Thành phần tuyến tính (linear predictor):** $\eta = \mathbf{w}^\top \mathbf{x}$.

2. **Hàm liên kết (link function):** $g(\cdot)$ nối η với giá trị kỳ vọng của biến phản hồi $\mu = \mathbb{E}[y \mid \mathbf{x}]$, tức $\eta = g(\mu)$, hay tương đương $\mu = g^{-1}(\eta)$.
3. **Phân phối nhiều:** thuộc họ phân phối mũ (exponential family).

Bảng 46: Linear Regression và Logistic Regression trong khung GLM

Mô hình	Phân phối	Hàm liên kết $g(\mu)$	Ứng dụng trong đồ án
Linear Regression	Gaussian	$g(\mu) = \mu$ (identity)	Dự đoán thời gian giao hàng
Logistic Regression	Bernoulli	$g(\mu) = \ln \frac{\mu}{1-\mu}$ (logit)	Phân lớp hài lòng hành khách

Bằng cách thay hàm liên kết, cùng một khung huấn luyện (gradient descent, regularization L1/L2, cross-validation) có thể áp dụng đồng nhất cho cả hai bài toán. Đây là lý do nhóm có thể tái sử dụng pipeline tiền xử lý, chuẩn hóa và lựa chọn siêu tham số từ Phần 1 sang Phần 2 với những điều chỉnh tối thiểu.

1.3 Exponential Family: Nền tảng toán học chung

Cả phân phối Gaussian (nền tảng của hồi quy) lẫn Bernoulli/Categorical (nền tảng của phân lớp) đều thuộc **họ phân phối mũ** (exponential family), có dạng tổng quát:

$$p(\mathbf{x} \mid \boldsymbol{\eta}) = h(\mathbf{x}) g(\boldsymbol{\eta}) \exp(\boldsymbol{\eta}^\top \mathbf{u}(\mathbf{x})), \quad (91)$$

trong đó $\boldsymbol{\eta}$ là tham số tự nhiên (natural parameter), $\mathbf{u}(\mathbf{x})$ là thống kê đủ (sufficient statistic), $h(\mathbf{x})$ và $g(\boldsymbol{\eta})$ là các hàm chuẩn hóa.

Gaussian. Phân phối Gaussian $\mathcal{N}(x \mid \mu, \sigma^2)$ có thể viết dưới dạng (91) với tham số tự nhiên $\eta = \mu/\sigma^2$ và thống kê đủ $u(x) = x$. Đây là cơ sở của Linear Regression trong Phần 1, trong đó nhóm sử dụng OLS, Ridge (L2), Lasso (L1), Elastic Net và Hồi quy Bayesian – tất cả đều xuất phát từ giả thiết nhiễu Gaussian.

Bernoulli. Phân phối Bernoulli $\text{Bern}(x \mid \mu)$ có tham số tự nhiên $\eta = \ln \frac{\mu}{1-\mu}$ (chính là log-odds) và thống kê đủ $u(x) = x$. Hàm liên kết logit của Logistic Regression là nghịch đảo của $\sigma(\cdot)$, tức là $\eta = g(\mu)$, tự nhiên xuất hiện từ cấu trúc exponential family.

Cấu trúc toán học thống nhất. Một thuộc tính quan trọng của exponential family là hàm log-likelihood luôn là hàm lồi (convex) theo tham số $\boldsymbol{\eta}$, đảm bảo việc tối ưu hóa không bị kẹt tại cực tiểu cục bộ. Cụ thể:

- Với Gaussian $\Rightarrow$ hàm mất mát MSE lồi, có nghiệm dạng đóng (Normal Equations).
- Với Bernoulli $\Rightarrow$ hàm mất mát Cross-Entropy lồi, nghiệm tìm bằng Gradient Descent hoặc Newton–Raphson / IRLS.

Ngoài ra, trong cả hai bài toán, nhóm đã áp dụng các kỹ thuật regularization (Ridge L2 và Lasso L1) với cùng một nền tảng lý thuyết: thêm prior vào tham số $\mathbf{w}$ (Gaussian prior $\Rightarrow$ Ridge, Laplace prior $\Rightarrow$ Lasso) và thực hiện MAP estimation thay vì MLE thuần túy.

1.4 Tổng kết liên hệ

Bảng 47: So sánh cấu trúc toán học giữa hai bài toán trong đồ án

Thành phần	Hồi quy (Phần 1)	Phân lớp (Phần 2)
Mô hình cơ sở	Linear Regression	Logistic Regression
Phân phối nhiễu	Gaussian	Bernoulli / Categorical
Hàm liên kết	Identity	Sigmoid (logit)
Hàm mất mát	MSE / NLL Gaussian	Cross-Entropy / NLL Bernoulli
Tối ưu hóa	OLS, Mini-batch GD	GD, Newton–Raphson (IRLS)
Regularization	Ridge, Lasso, Elastic Net	L1, L2
Mở rộng Bayesian	Bayesian Linear Regression	Xấp xỉ Laplace
Mở rộng phi tuyến	Kernel Ridge, Gaussian Process	Kernel Logistic Regression
Mô hình sinh	Gaussian Process Regression	LDA (Gaussian generative)

Nhìn vào Bảng 47, hồi quy và phân lớp không phải là hai thể giới tách biệt mà là hai biểu hiện của *cùng một khung GLM* với phân phối và hàm liên kết khác nhau. Hiểu được mối liên hệ này giúp:

1. Tái sử dụng pipeline và kỹ thuật (regularization, cross-validation, gradient descent) một cách có hệ thống qua cả hai phần.
2. Giải thích tại sao Logistic Regression – dù tên là “hồi quy” – lại là mô hình phân lớp: nó hồi quy xác suất, không phải giá trị liên tục.
3. Mở rộng tự nhiên sang các bài toán phức tạp hơn (Softmax Regression, GLM với phân phối Poisson cho dữ liệu đếm, v.v.) bằng cách thay đổi hàm liên kết và phân phối nhiễu tương ứng.

2 Bảng so sánh toàn diện các mô hình

Bảng 48: So sánh toàn diện các mô hình học có giám sát (Phần 1: Hồi quy – Phần 2: Phân lớp). ✓: có / tốt; ×: không / kém; ~: phụ thuộc cấu hình.

Tiêu chí	Linear Reg.	Ridge / Lasso	Logistic Reg.	LDA	QDA	Perceptron
Giả thiết phân phối	$\epsilon \sim \mathcal{N}(0, \beta^{-1})$	$\epsilon \sim \mathcal{N}(0, \beta^{-1})$	Không giả thiết $p(\mathbf{x})$	$p(\mathbf{x} C_k) = \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma})$	$p(\mathbf{x} C_k) = \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$	Không giả thiết
Nghiệm dạng đóng	✓ $\hat{\mathbf{w}} = (\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \mathbf{t}$	✓ (Ridge); × (Lasso – ISTA/ADMM)	× (GD / Newton–Raphson)	✓ (ước lượng MLE theo lớp)	✓ (ước lượng MLE theo lớp)	× (cập nhật lặp)
Độ phức tạp huấn luyện	$O(NM^2 + M^3)$	$O(NM^2 + M^3)$ (Ridge); $O(NM \cdot T)$ (Lasso)	$O(NM \cdot T)$ (GD); $O(NM^2 + M^3)$ (IRLS)	$O(ND^2 + D^3)$	$O(ND^2 + KD^3)$	$O(ND \cdot T)$
Khả năng giải thích	Cao – hệ số w_j đo đóng góp trực tiếp	Cao (Ridge); Cao + thừa (Lasso)	Cao – log-odds của từng đặc trưng	Trung bình – cần phân tích $\boldsymbol{\mu}_k, \boldsymbol{\Sigma}$	Thấp hơn – biên cong, khó trực quan	Thấp – không có đầu ra xác suất
Nhạy với outlier	Cao – MSE khuếch đại sai số lớn	Trung bình – L2 vẫn khuếch đại; L1 giảm nhẹ	Thấp – log-loss bảo hòa ở dự đoán chắc chắn	Trung bình – phụ thuộc ước lượng $\boldsymbol{\Sigma}$	Trung bình – nhạy hơn LDA do $\boldsymbol{\Sigma}_k$ riêng	Thấp – hard-margin, nhưng dễ không hội tụ
Hiệu năng thực nghiệm	RMSE $\approx$ 3.33 ngày; $R^2 \approx 0.38$	RMSE $\approx$ 3.33 ngày; $R^2 \approx 0.38$	Acc = 87.22%; F1 = 85.12%; AUC = 0.927	Acc = 86.90%; F1 = 84.76%; AUC = 0.925	Acc = 85.35%; F1 = 82.85%; AUC = 0.917	Acc = 84.21%; F1 = 80.61%; AUC = 0.900

Ghi chú ký hiệu: N – số mẫu, M – số đặc trưng sau ánh xạ cơ sở, D – số chiều đặc trưng gốc, K – số lớp, T – số vòng lặp (epochs). Hiệu năng hồi quy đo trên tập test của bộ dữ liệu logistics Brazil; hiệu năng phân lớp đo trên tập test của bộ dữ liệu Airline Passenger Satisfaction.

Nhận xét tổng hợp. Nhìn vào Bảng 48, có thể rút ra ba điểm nổi bật:

1. **Nghiệm dạng đóng = lợi thế tốc độ và ổn định.** Linear Regression, Ridge, LDA và QDA đều có nghiệm dạng đóng, cho phép huấn luyện trong dưới 1 giây ngay cả với $N > 10^5$ mẫu. Ngược lại, Logistic Regression (GD) và Perceptron phụ thuộc vào số vòng lặp T , đặc biệt Perceptron cần tới ≈ 152 giây do thiếu đảm bảo hội tụ khi dữ liệu không phân tách tuyến tính.
2. **Giả thiết phân phối = con dao hai lưỡi.** LDA đạt hiệu năng tốt và ổn định nhất trong phân lớp nhờ giả thiết Gaussian phù hợp với dữ liệu Airline sau tiền xử lý. Tuy nhiên, khi dữ liệu bị nhiễu lớn ($\sigma = 1.0$), F1 của LDA giảm tới 19 điểm phần trăm, trong khi Logistic Regression (discriminative, không giả thiết phân phối) chỉ giảm ≈ 10 điểm.
3. **Regularization quan trọng nhưng không quyết định tất cả.** Trong bài toán hồi quy, Lasso tự động loại bỏ đặc trưng ít quan trọng và cải thiện khả năng diễn giải. Trong bài toán phân lớp, L1 (LR-L1) dẫn đầu về F1-score nhưng khoảng cách với LDA chỉ $\approx 0.36\%$, có ý nghĩa thống kê (McNemar's test: $p = 4.39 \times 10^{-4}$) nhưng nhỏ trong thực tế.

3 Phân tích độ nhạy cảm và tính bền vững

3.1 Sensitivity Analysis: Thay đổi tỷ lệ Train/Test Split

Thiết lập thực nghiệm. Nhóm thực hiện thí nghiệm với ba tỷ lệ phân chia: train $\in \{60\%, 70\%, 80\%\}$ (tương ứng test $\in \{40\%, 30\%, 20\%\}$). Mỗi điều kiện được lặp lại 10 lần với 10 giá trị seed khác nhau ($\text{seed} \in \{0, 1, \dots, 9\}$) để ước lượng phân phối của chỉ số hiệu năng. Kết quả được trực quan hóa bằng *boxplot* F1-score cho từng mô hình và từng tỷ lệ split.

Kết quả – Bài toán Phân lớp. Bảng 49 tóm tắt F1-score mean $\pm$ std của ba mô hình đại diện. Kết quả đầy đủ đã được trình bày ở Bảng 33 (Phần 14.2 của báo cáo).

Bảng 49: Sensitivity analysis – F1-score (mean $\pm$ std) theo tỷ lệ train/test split, bài toán phân lớp Airline.

Train split	Mô hình	F1 mean	F1 std
60%	LDA	0.8470	0.0019
	LR-GD	0.8105	0.0044
	Perc.	0.7690	0.0380
70%	LDA	0.8475	0.0024
	LR-GD	0.8117	0.0059
	Perc.	0.7579	0.0632
80%	LDA	0.8474	0.0036
	LR-GD	0.8119	0.0072
	Perc.	0.7740	0.0410

Kết quả – Bài toán Hồi quy. Nhóm thực hiện tương tự cho bài toán hồi quy, đo RMSE (ngày) trên tập test với ba tỷ lệ split và 10 lần lặp. Bảng 50 trình bày kết quả cho các mô hình hồi quy chính.

Bảng 50: Sensitivity analysis – RMSE (ngày, mean $\pm$ std) theo tỷ lệ train/test split, bài toán hồi quy logistics.

Train split	Mô hình	RMSE mean	RMSE std
60%	Linear Reg.	3.385	0.042
	Ridge	3.382	0.040
	Lasso	3.390	0.043
70%	Linear Reg.	3.355	0.031
	Ridge	3.352	0.030
	Lasso	3.358	0.032
80%	Linear Reg.	3.327	0.018
	Ridge	3.325	0.017
	Lasso	3.330	0.019

Nhận xét.

- **Mô hình ổn định nhất (phân lớp): LDA.** F1-score của LDA gần như không thay đổi khi tăng từ 60% lên 80% dữ liệu huấn luyện ($0.8470 \rightarrow 0.8474$), phản ánh mô hình đã đạt *data saturation* – giả thiết Gaussian đúng giúp ước lượng tham số hội tụ nhanh ngay với lượng dữ liệu vừa phải. Ngược lại, Perceptron có std lên tới 0.063 – cao gấp 33 lần LDA – do thiếu bảo đảm hội tụ trên dữ liệu không phân tách tuyến tính.
- **Mô hình ổn định nhất (hồi quy): Ridge và Linear Reg. ngang nhau.** Std RMSE của Ridge nhỉnh hơn Linear Reg. không đáng kể; cả hai giảm std rõ rệt khi tăng split từ 60% lên 80%, cho thấy mô hình hồi quy tuyến tính vẫn được hưởng lợi khi có thêm dữ liệu, khác với LDA đã bão hòa.
- *Gợi ý thực tiễn:* Với dữ liệu lớn ($N > 10^4$), tỷ lệ 70/30 là đủ cho cả hai bài toán; không cần tăng lên 80/20 vì cải thiện không đáng kể.

3.2 Noise Injection: Thêm Nhiễu Gaussian vào Đặc trưng

Thiết lập thực nghiệm. Nhóm thêm nhiễu Gaussian $\delta \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ vào tập đặc trưng trên tập test (sau khi chuẩn hóa) với $\sigma \in \{0.0, 0.1, 0.5, 1.0\}$. Thí nghiệm được lặp lại 5 lần với 5 seed khác nhau để ước lượng phân phối của chỉ số hiệu năng.

Bảng 51: F1-score (mean) của các mô hình phân lớp theo mức độ nhiễu Gaussian σ .

Mô hình	$\sigma = 0.0$	$\sigma = 0.1$	$\sigma = 0.5$	$\sigma = 1.0$
LDA	0.8476	0.8442	0.7600	0.6548
LR-GD	0.8335	0.8309	0.8040	0.7332
LR-L2	0.8335	0.8309	0.8040	0.7332
Perc.	0.8061	0.8011	0.7173	0.6291

Kết quả – Bài toán Phân lớp.

Bảng 52: RMSE (ngài, mean) của các mô hình hồi quy theo mức độ nhiễu Gaussian σ .

Mô hình	$\sigma = 0.0$	$\sigma = 0.1$	$\sigma = 0.5$	$\sigma = 1.0$
Linear Reg.	3.33	3.36	3.71	4.42
Ridge (L2)	3.33	3.35	3.68	4.35
Lasso (L1)	3.33	3.36	3.70	4.40
Elastic Net	3.33	3.35	3.69	4.37

Kết quả – Bài toán Hồi quy.

Nhận xét.

- **Mô hình phân lớp bền vững nhất trước nhiễu lớn: LR-GD / LR-L2.** Tại $\sigma = 1.0$, F1 của LR-GD giảm ≈ 10 điểm phần trăm (từ 83.35% xuống 73.32%), trong khi LDA giảm tới ≈ 20 điểm (từ 84.76% xuống 65.48%). Điều này phản ánh bản chất discriminative của Logistic Regression: khi không giả thiết phân phối $p(\mathbf{x}|C_k)$, mô hình ít bị ảnh hưởng hơn khi phân phối dữ liệu bị nhiễu làm lệch.
- **Mô hình hồi quy bền vững nhất trước nhiễu lớn: Ridge.** Ridge (L2) cho RMSE thấp nhất ở cả $\sigma = 0.5$ và $\sigma = 1.0$ nhờ hệ số co shrinkage làm giảm phương sai ước lượng khi đầu vào bị nhiễu. Elastic Net đứng thứ hai, phản ánh lợi ích kết hợp L1 và L2.
- Đáng chú ý, L2 regularization *không* cải thiện tính bền vững của Logistic Regression (LR-GD và LR-L2 cho kết quả giống nhau), do $\lambda^* = 0$ từ cross-validation nghĩa là mô hình chưa bị overfitting, nên lực phạt L2 không tác động.

3.3 Feature Corruption: Xóa Ngẫu nhiên Đặc trưng và So sánh Chiến lược Imputation

Thiết lập thực nghiệm. Nhóm xóa ngẫu nhiên một tỷ lệ $r \in \{10\%, 20\%, 30\%\}$ giá trị đặc trưng trong tập test (mỗi phần tử độc lập với xác suất r), tạo ra dữ liệu thiếu hoàn toàn ngẫu nhiên (MCAR). Ba chiến lược điền giá trị khuyết được so sánh:

- **Mean imputation:** điền giá trị trung bình của từng đặc trưng tính trên tập huấn luyện.
- **Median imputation:** điền giá trị trung vị của từng đặc trưng tính trên tập huấn luyện (ít nhạy với outlier hơn mean).
- **KNN imputation:** điền bằng giá trị trung bình của $k = 5$ hàng xóm gần nhất trong không gian đặc trưng đã có giá trị, tính theo khoảng cách Euclidean.

Thí nghiệm lặp lại 5 lần với 5 seed khác nhau.

Bảng 53: F1-score (mean $\pm$ std) của LR-L1 theo tỷ lệ feature corruption và chiến lược imputation, bài toán phân lớp Airline.

Imputation	$r = 10\%$	$r = 20\%$	$r = 30\%$
Mean	0.849 ± 0.003	0.843 ± 0.005	0.834 ± 0.007
Median	0.849 ± 0.003	0.843 ± 0.005	0.835 ± 0.006
KNN	0.851 ± 0.002	0.847 ± 0.004	0.841 ± 0.005

Kết quả – Bài toán Phân lớp (F1-score, mô hình LR-L1).

Bảng 54: RMSE (ngày, mean $\pm$ std) của Ridge theo tỷ lệ feature corruption và chiến lược imputation, bài toán hồi quy logistics.

Imputation	$r = 10\%$	$r = 20\%$	$r = 30\%$
Mean	3.38 ± 0.02	3.44 ± 0.04	3.54 ± 0.06
Median	3.37 ± 0.02	3.43 ± 0.03	3.52 ± 0.06
KNN	3.35 ± 0.01	3.38 ± 0.02	3.44 ± 0.04

Kết quả – Bài toán Hồi quy (RMSE, mô hình Ridge).

Nhận xét.

- **KNN imputation vượt trội cả Mean và Median** ở mọi mức độ corruption trong cả hai bài toán. Khoảng cách mở rộng dần khi r tăng: tại $r = 30\%$, KNN cải thiện RMSE thêm ≈ 0.1 ngày so với Mean imputation – tương đương $\approx 3\%$ cải thiện.
- **Median imputation nhỉnh hơn Mean một chút** trong bài toán hồi quy, do phân phối thời gian giao hàng có đuôi phải (right-skewed) và nhiều giá trị ngoại lai; median ít bị kéo bởi các điểm cực đoan.
- **Tất cả chiến lược đều suy giảm hiệu năng khi r tăng**, nhưng với mức độ có thể chấp nhận được đến $r = 20\%$. Tại $r = 30\%$, KNN vẫn duy trì RMSE < 3.5 ngày – dưới ngưỡng thực tiễn của bài toán logistics.

3.4 Phân tích Hội tụ: Loss Curve của các Thuật toán Lặp

Thiết lập. Nhóm vẽ loss curve (hàm mất mát trên tập train và tập validation) theo số epoch cho các thuật toán lặp trong cả hai phần:

- **Hồi quy:** Mini-batch Gradient Descent (Linear Reg.), Coordinate Descent (Lasso).
- **Phân lớp:** Gradient Descent (LR-GD, LR-L2), Newton–Raphson / IRLS (LR-NR), Perceptron.

Bảng 55: Số epochs cần thiết để đạt hội tụ (training loss thay đổi $< 10^{-4}$ giữa hai epoch liên tiếp) và hiệu năng tốt nhất tương ứng.

Bài toán	Thuật toán	Epochs hội tụ	Metric tốt nhất	Ghi chú
Hồi quy	Mini-batch GD (LR)	$\approx 120-150$	RMSE = 3.33 ngày	Cosine annealing
Hồi quy	Coord. Descent (Lasso)	$\approx 80-100$	RMSE = 3.33 ngày	Hội tụ đều đặn
Phân lớp	LR-GD	$\approx 200-300$	F1 = 83.24%	Plateau sớm
Phân lớp	LR-L2	$\approx 200-300$	F1 = 83.24%	Giống LR-GD
Phân lớp	LR-NR (IRLS)	$\approx 10-15$	F1 = 84.34%	Hội tụ bậc hai
Phân lớp	Perceptron	$\approx 500+$	F1 = 80.61%	Không ổn định

Kết quả và nhận xét. Các kết luận chính từ loss curve:

1. **Newton–Raphson (IRLS) hội tụ nhanh nhất:** chỉ cần 10–15 bước nhờ hội tụ bậc hai, so với 200–300 bước của GD. Chi phí mỗi bước $O(NM^2 + M^3)$ cao hơn, nhưng tổng thời gian vẫn thấp hơn GD (≈ 0.89 giây vs. ≈ 2.30 giây).
2. **Mini-batch GD với Cosine Annealing học rate:** hội tụ trong khoảng 120–150 epoch với RMSE ổn định; training loss và validation loss không có khoảng cách lớn, cho thấy mô hình không overfitting.
3. **Perceptron không đạt hội tụ thực sự:** loss dao động không giảm đều sau epoch 100, phản ánh dữ liệu Airline không phân tách tuyến tính hoàn toàn. Thuật toán dừng sớm do ngưỡng epoch tối đa, không phải do điều kiện hội tụ lý thuyết.
4. **Coordinate Descent (Lasso)** cho đường loss mịn và đều đặn, hội tụ nhanh hơn GD nhờ cập nhật từng chiều một với nghiệm dạng đóng (soft-thresholding) tại mỗi bước.

4 Khả năng tái hiện lại thí nghiệm

4.1 Cố định Random Seed

Toàn bộ thí nghiệm trong đồ án được thiết lập để tái hiện hoàn toàn bằng cách cố định random seed ở *tất cả* các thư viện và lớp ngẫu nhiên:

- **Python built-in:** `random.seed(42)`
- **NumPy:** `numpy.random.seed(42)`
- **Scikit-learn:** tham số `random_state=42` được truyền vào mọi hàm liên quan (train/test split, KFold, SGDClassifier, ...)
- **Để đảm bảo tái hiện đầy đủ khi chạy notebook từ đầu,** khối seed được đặt ở ô đầu tiên (Cell 1) của notebook và không được comment hoặc bỏ qua.

4.2 Ghi Log Đầy đủ

Mọi thí nghiệm được ghi log theo cấu trúc chuẩn, bao gồm:

1. siêu tham số của mô hình (learning rate, số epoch, hệ số regularization, kích thước batch, v.v.);
2. cấu hình chia dữ liệu, random seed và phiên bản thư viện được sử dụng;
3. các metric trên tập huấn luyện, validation và test sau mỗi giai đoạn;
4. thời gian huấn luyện, thời gian dự đoán và các ghi chú về lỗi/hội tụ nếu có.

4.3 Báo cáo Thời gian Thực thi và Bộ nhớ

Bảng 56 tổng hợp thời gian huấn luyện và bộ nhớ sử dụng của từng thuật toán, đo trên cùng một máy (CPU: Intel Core i7-12700H, RAM: 16 GB, Python 3.11, scikit-learn 1.4.2).

Bảng 56: Thời gian huấn luyện và bộ nhớ của các thuật toán trên tập train đầy đủ.

Bài toán	Mô hình	TG huấn luyện (s)	TG dự đoán (s)	Bộ nhớ (MB)
Hồi quy	Linear Reg. (OLS)	0.42	0.003	125.5
	Ridge	0.44	0.003	126.0
	Lasso	1.20	0.003	130.2
	Mini-batch GD	4.80	0.003	85.0
	Gaussian Process	48.50	0.25	980.5
	Reg.			
Phân lớp	LDA	0.21	0.004	82.4
	QDA	0.34	0.005	88.6
	LR-L1	0.54	0.003	105.8
	(scikit-learn)			
	LR-NR / IRLS	0.89	0.003	145.2
	LR-GD	2.30	0.003	95.0
	Perceptron	152.40	0.003	78.5

Nhận xét.

- LDA là thuật toán nhanh nhất trong phân lớp (≈ 0.21 giây), phù hợp cho ứng dụng thực tế cần phản hồi nhanh.
- Gaussian Process Regression đòi hỏi $O(N^2)$ bộ nhớ (≈ 890 MB cho $N \approx 70,000$ mẫu) và thời gian huấn luyện dài, không thể mở rộng trực tiếp cho dữ liệu triệu mẫu.
- Perceptron mất 152 giây – cao hơn tất cả mô hình phân lớp – do không hội tụ trên dữ liệu không phân tách tuyến tính và phải chạy đến epoch tối đa được cài đặt.

4.4 Môi trường Phụ thuộc: requirements.txt

Để đảm bảo tái hiện hoàn toàn, nhóm cung cấp file `requirements.txt` với version cụ thể như sau. File này được tạo tự động bằng lệnh `pip freeze` sau khi hoàn tất quá trình phát triển.

requirements.txt - Đồ án 1: Học có giám sát và Ứng dụng

- `category_encoders` (`== 2.9.0`)
- `matplotlib` (`== 3.10.8`)
- `numpy` (`== 2.4.4`)
- `pandas` (`== 3.0.2`)
- `scikit_learn` (`== 1.8.0`)
- `seaborn` (`== 0.13.2`)
- `statsmodels` (`== 0.14.6`)
- `jinja2` (`≥ 3.0.0`)
- `plotly` (`≥ 5.24.1`)
- `pyarrow` (`≥ 15.0.0`)

Để cài đặt toàn bộ môi trường, chạy lệnh:

```
pip install -r requirements.txt
```

Sau khi cài đặt, chạy lại toàn bộ notebook từ đầu (**Kernel** → **Restart & Run All**) sẽ tái tạo chính xác mọi kết quả, bảng số liệu và đồ thị trong báo cáo này.

4.5 Kiểm tra Tính Tái hiện

Nhóm thực hiện kiểm tra tái hiện bằng cách:

1. Chạy notebook lần 1 và lưu toàn bộ kết quả vào `results_run1.json`.
2. Xóa tất cả output, khởi động lại kernel, chạy lại notebook lần 2, lưu vào `results.json`.

Kết quả: tất cả chỉ số metric (F1, RMSE, AUC, v.v.) khớp hoàn toàn giữa hai lần chạy, xác nhận thí nghiệm có thể tái hiện được 100%.

Tài liệu

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006. Chương 4: Linear Models for Classification.
- [2] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- [3] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- [4] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online] <https://scikit-learn.org>
- [5] teejmahal20, “Airline Passenger Satisfaction,” Kaggle, 2020. [Online] <https://www.kaggle.com/datasets/teejmahal20/airline-passenger-satisfaction>